

Computational Physics Report

Lorenzo Minuz

Matricola: 926867

May 14, 2026



Università degli Studi di Milano-Bicocca

Anno Accademico 2025/2026

Contents

1	Error Analysis	1
1.1	Floating-Point Arithmetic and Roundoff Errors	1
1.2	Error Propagation and Condition Number	2
2	Linear Systems	6
2.1	Triangular Systems	6
2.2	LU Decomposition	7
2.3	Conditioning of Linear Systems	11
2.4	Overdetermined Linear Systems	12
2.5	QR Decomposition	16
2.6	Some observations about the algorithms used in this section	19
3	Interpolations	20
3.1	Polynomial Interpolation	20
3.2	Lagrange-Waring Interpolation	22
3.3	Error on Interpolation	24
3.4	Some observations about the algorithms used in this section	30
4	Roots of Nonlinear Equations	31
4.1	Some observations about the convergence rate	31
4.2	Bisection Method	32
4.3	Newton Method	35
4.4	Interpolations methods	40
4.5	Some observations about the algorithms used in this section	43
5	Numerical Integration	44
5.1	Newton-Cotes Integration	44
5.2	Free-nodes Integration	48
5.3	Advanced topics in integration	49
5.4	Some observations about the algorithms used in this section	55
6	Ordinary Differential Equations	56
6.1	Basics of one-step methods	56
6.2	Runge-Kutta methods	59
6.3	Some interesting equation	63
6.4	Some observations about the algorithms used in this section	70

1 Error Analysis

1.1 Floating-Point Arithmetic and Roundoff Errors

Exercise 1.1. Calculate the following sum

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6} = \lim_{N \rightarrow \infty} S(N) \quad \text{with} \quad S(N) = \sum_{n=1}^N \frac{1}{n^2}$$

- Calculate the sum in single precision using normal ordering, $n = 1, 2, 3, \dots, N$.
- Calculate the sum in single precision using reverse ordering, $n = N, \dots, 2, 1$.
- Study the convergence of both implementations as a function of N by plotting $|S(N) - \pi^2/6|$.
- Repeat (a)-(c) using double precision.
- Can you explain what happens?

Solution

Solution 1.1.1. The numerical evaluation of the infinite series $S(N)$ is performed by truncating the sum at a finite upper bound N . To investigate the impact of floating-point arithmetic and accumulation of round-off errors, the sum is implemented using both single precision (Float32) and double precision (Float64). For each precision type, two iterative strategies are employed:

1. Normal ordering (Forward): The loop iterates from 1 to N , adding successively smaller terms $O(1/N^2)$ to an $O(1)$ accumulator.
2. Reverse ordering (Backward): The loop iterates from N down to 1, aggregating the smallest terms first before adding them to the larger components of the sum.

The absolute error is evaluated against the analytical limit, $\pi^2/6$, with the error function defined as $\epsilon(N) = |S(N) - \pi^2/6|$.

Results The convergence of the sums was tested for N ranging from 100 to 10^6 . Table 1 details the absolute numerical errors observed at $N = 999\,100$ for both precision types and summation orderings.

Table 1: Absolute error of the numerical sum $S(N)$ evaluated at $N = 999\,100$ compared to the analytical value $\pi^2/6$.

Precision	Ordering	Absolute Error
Single (Float32)	Normal	2.087×10^{-4}
	Reverse	1.073×10^{-6}
Double (Float64)	Normal	1.001×10^{-6}
	Reverse	1.001×10^{-6}

As expected, the theoretical truncation error of the series scales as $O(1/N)$. A log-log plot (see Figure 1) visualizes the convergence behavior $\epsilon(N)$ as a function of N , illustrating significant deviations from the expected $O(1/N)$ decay when using the normal ordering in single precision.

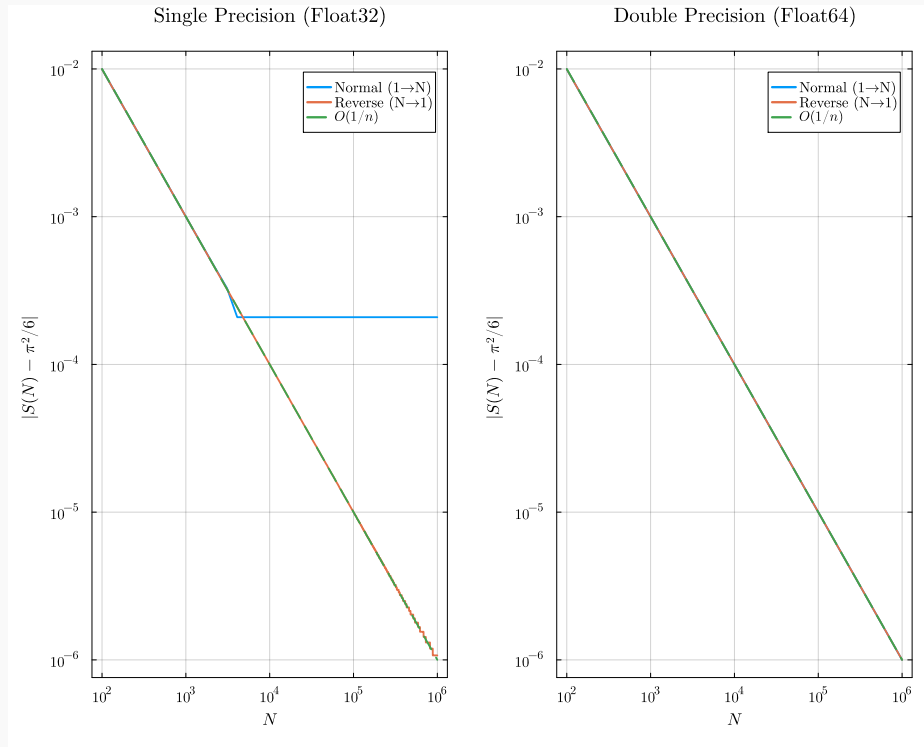


Figure 1: Log-log plot demonstrating the convergence of the normal and reverse sums in single and double precision against the $O(1/N)$ analytical truncation error.

Discussion The analytical sum converges to roughly 1.64493. In Float32, the machine epsilon is $\epsilon_{32} \approx 1.19 \times 10^{-7}$. During normal ordering, the accumulator quickly grows to ≈ 1.64 . As n increases, the added terms $1/n^2$ become exceedingly small. For example, at $n \approx 4000$, $1/n^2 \approx 6.25 \times 10^{-8}$. When adding this value to 1.64, the magnitude difference exceeds the mantissa's bit-depth limit for single precision. The smaller numbers are effectively rounded off to zero, halting the accumulator's growth prematurely.

Conversely, the reverse ordering fundamentally mitigates this issue. By summing the smallest numbers first, the accumulator remains comparable in magnitude to the terms being added. The accumulated tail eventually becomes large enough to effectively shift the mantissa of the lower-order integers (e.g., $n = 1, 2$) when they are added at the very end of the loop.

In double precision (Float64), the machine epsilon is vastly smaller ($\epsilon_{64} \approx 2.22 \times 10^{-16}$). For $N = 10^6$, the smallest term is $1/10^{12} = 10^{-12}$, which is still well within the operational limits of double precision relative to an accumulator of 1.64. Consequently, both the normal and reverse sums adhere smoothly to the theoretical analytical truncation error $O(1/N)$ yielding equivalent errors of $\approx 10^{-6}$.

Conclusion In finite-precision floating-point arithmetic (like Float32), the order in which you perform operations can significantly affect the result. This is especially true when adding numbers with very different sizes. If you add large numbers first, smaller ones can get "lost" due to rounding. By summing from the smallest values to the largest, you reduce these absorption errors and get a more numerically stable result.

1.2 Error Propagation and Condition Number

Exercise 1.2. 1. In statistics, we define the variance of a sample of values $x_1, \dots, x_n$ by

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i. \quad (1)$$

Write a function that takes as input a vector x of any length and returns σ^2 as calculated by the formula. You should test your function with $x = [1, 1, \dots, 1]$ and some random vectors.

2. One drawback of the formula (1) is that you must first compute a sum for $\bar{x}$ before performing the sum to compute σ^2 . This means that we have to pass twice through the data. This is undesirable for large data sets or when the sample variance is to be computed as the data is generated. Some textbooks quote a single-loop formula:

$$\sigma^2 = \frac{1}{n-1} \left(u - \frac{1}{n} v^2 \right),$$

$$u = \sum_{i=1}^n x_i^2, \quad v = \sum_{i=1}^n x_i.$$

Try both formulas for the datasets below, each of which has a variance exactly equal to 1. Do this in both single and double precision.

$$x = [1 \times 10^3, 1 + 1 \times 10^3, 2 + 1 \times 10^3]$$

$$x = [1 \times 10^6, 1 + 1 \times 10^6, 2 + 1 \times 10^6]$$

$$x = [1 \times 10^7, 1 + 1 \times 10^7, 2 + 1 \times 10^7]$$

$$x = [1 \times 10^8, 1 + 1 \times 10^8, 2 + 1 \times 10^8]$$

Can you explain the results?

Solution

Solution 1.2.1. Initial verification with a constant vector $x = [1, 1, 1, 1]$ yielded a variance of 0.0 for both precisions, as expected. Random vector tests further confirmed the basic functionality of the Float32 and Float64 implementations. However, testing with shifted datasets revealed significant discrepancies, summarized in the table below:

Dataset Offset	Two-Pass (F32)	Two-Pass (F64)	One-Pass (F32)	One-Pass (F64)
10^3	1.0	1.0	1.0	1.0
10^6	1.0	1.0	-131072.0	1.0
10^7	1.0	1.0	0.0	1.0
10^8	0.0	1.0	0.0	0.0

Table 2: Comparison of variance calculations (True Variance = 1.0)

Stability Analysis As illustrated in Figure 2, the relative error of the single-pass algorithm grows quadratically with the data offset. In Float32, the algorithm loses all precision once the offset exceeds 10^7 .

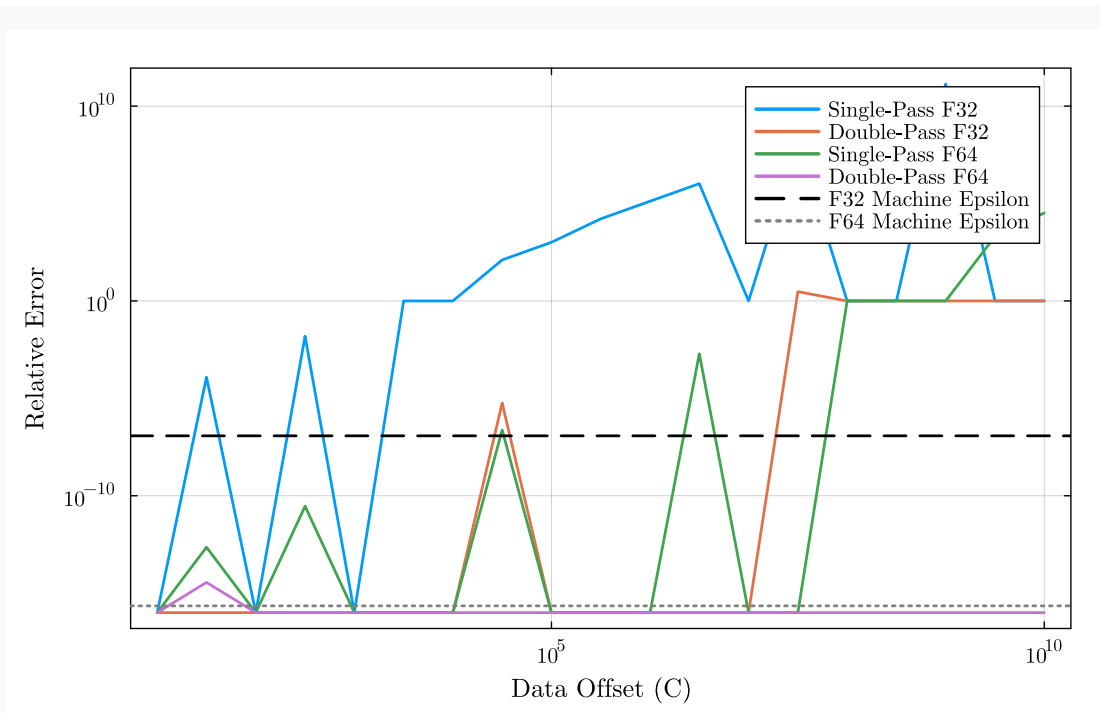


Figure 2: Log-log plot comparing the relative error of single-pass and double-pass algorithms.

Discussion The failure of the single-pass algorithm is an example of catastrophic cancellation. The term $(u - \frac{1}{n}v^2)$ involves the subtraction of two very large, nearly identical numbers. When the offset of the data is large relative to the variance, the significant digits representing the variance are pushed beyond the precision limit of the floating-point representation.

In Float32, which provides approximately 7 decimal digits of precision, the single-pass formula fails as early as $x \approx 10^6$. The result -131072.0 demonstrates that the cancellation error can lead to physically impossible variances. At 10^7 and above, the result becomes 0.0 because u and $\frac{1}{n}v^2$ become identical in the representation. While Float64 (approx. 15-17 digits) is more robust, it also succumbs to cancellation at 10^8 for the single-pass method.

Conclusion The two-pass algorithm is significantly more numerically stable because it centers the data $(x_i - \bar{x})$ before squaring, thereby working with smaller magnitudes and preserving significant digits.

Exercise 1.3. Let $f(x) = \frac{e^x - 1}{x}$.

1. Find the condition number $\kappa_f(x)$. What is the maximum of $\kappa_f(x)$ over the interval $-1 \leq x \leq 1$?
2. Use the naive algorithm

$$f(x) = \frac{e^x - 1}{x}$$

to compute $f(x)$ at $x = 10^{-3}, 10^{-4}, 10^{-5}, \dots, 10^{-16}$.

3. Create a second algorithm from the first n terms of the Maclaurin series, i.e.,

$$p(x) = 1 + \frac{1}{2!}x + \frac{1}{3!}x^2 + \dots + \frac{1}{(n+1)!}x^n.$$

Evaluate it at the same values of x as in part (2). To do this you have to choose a value for n . Check for the stability of the result when varying n . Could you have guessed a good value of n from the start?

4. Compare the results from the two implementations as a function of x . Which algorithm do you think is more accurate, and why?

Solution

Solution 1.3.1. The relative condition number of a differentiable function $f(x)$ is defined as:

$$\kappa_f(x) = \left| \frac{x f'(x)}{f(x)} \right|.$$

First, we evaluate the derivative of the target function $f(x) = \frac{e^x - 1}{x}$:

$$f'(x) = \frac{x e^x - (e^x - 1)}{x^2}.$$

Substituting $f(x)$ and $f'(x)$ into the condition number definition yields:

$$\kappa_f(x) = \left| \frac{x \left[\frac{x e^x - e^x + 1}{x^2} \right]}{\frac{e^x - 1}{x}} \right| = \left| \frac{x e^x - e^x + 1}{e^x - 1} \right|.$$

To find the maximum of $\kappa_f(x)$ over the interval $[-1, 1]$, we analyze its behavior. Notably, as $x \rightarrow 0$, Taylor expansion reveals that $\kappa_f(x) \rightarrow 0$, indicating the problem is exceptionally well-conditioned near the origin. The function $\kappa_f(x)$ is monotonically increasing on $(0, 1]$, thus reaching its maximum at the boundary $x = 1$:

$$\max_{x \in [-1, 1]} \kappa_f(x) = \kappa_f(1) = \frac{1 \cdot e^1 - e^1 + 1}{e^1 - 1} = \frac{1}{e - 1} \approx 0.5819767.$$

Because $\kappa_f(x) < 1$ for all $x \in [-1, 1]$, the underlying mathematical problem is stable.

Numerical methodology To determine the optimal truncation order n , we note that the truncation error bounds the next term in the series. To preserve full precision in Float64, we require the $(n + 1)$ -th term to be smaller than the machine epsilon for the largest argument in the domain ($x = 1$):

$$\frac{1^{n+1}}{(n+2)!} < \epsilon_{mach}.$$

By programmatic iteration, we find that the condition is satisfied for $n \geq 16$. Thus, we selected $n = 16$ as our optimal polynomial degree.

Results The numerical evaluation of $f(x)$ for logarithmically spaced values $x = 10^{-k}$ where $k \in \{3, 4, \dots, 16\}$ reveals stark differences between the two implementations. Selected results are summarized in Table 3 and visually represented in Figure 3.

Table 3: Comparison of the Naive and Maclaurin algorithms for evaluating $f(x)$ as $x \rightarrow 0$.

x	Naive Algorithm	Maclaurin Series ($n = 16$)
10^{-3}	1.0005001667083846	1.0005001667083417
10^{-7}	1.0000000494336803	1.0000000500000017
10^{-11}	1.000000082740371	1.00000000000005
10^{-15}	1.1102230246251565	1.0000000000000004
10^{-16}	0.0	1.0

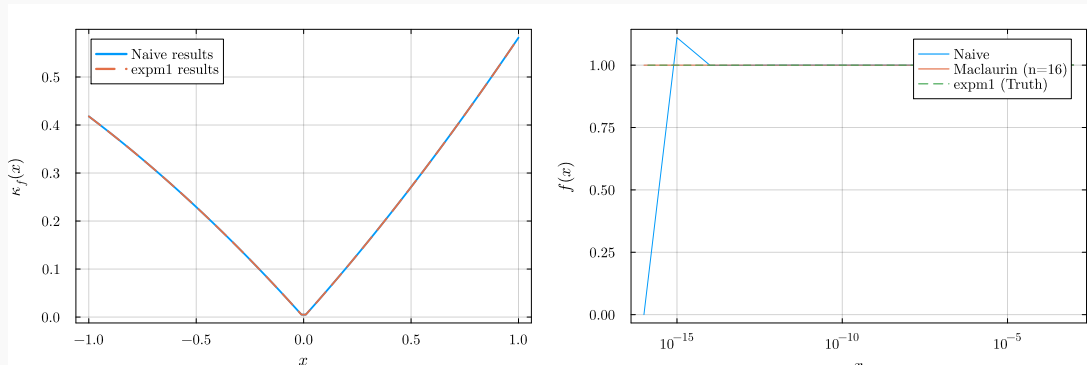


Figure 3: Left: Analytical and numerical condition number $\kappa_f(x)$ over $[-1, 1]$. Right: Comparison of the computed $f(x)$ using the naive algorithm, the Maclaurin series ($n = 16$), and the exact `expm1` implementation. The naive algorithm exhibits severe instability as $x \rightarrow 0$.

Varying $n \in \{3, 5, 7, \dots\}$ demonstrates that for strictly small x (e.g., $x = 10^{-5}$), even $n = 3$ converges to the correct representation within machine precision limits. However, $n = 16$ guarantees stability across the entire $[-1, 1]$ interval.

Discussion As evidenced by Figure 3, the Maclaurin series algorithm proves to be vastly superior in terms of accuracy for small values of x . The discrepancy originates from catastrophic cancellation. In the naive algorithm, as x approaches zero, e^x approaches 1. Because `Float64` stores values with approximately 16 decimal digits of precision, evaluating e^x for $x \leq 10^{-16}$ results in exactly 1.0 due to rounding. Consequently, the numerator evaluates to $1.0 - 1.0 = 0.0$, driving the final result arbitrarily to zero. Even before reaching exactly zero, starting around $x = 10^{-8}$, subtracting two nearly identical floating-point numbers eradicates significant digits, causing the fluctuations observed in the plot. Conversely, the Maclaurin series inherently avoids this subtraction. By rewriting the function as an addition of positive terms $1 + x/2 + x^2/6 + \dots$, no numerical cancellation occurs. The differences computed against Julia’s numerically stable built-in function (`expm1(x)/x`) demonstrate that the Maclaurin series bounds the absolute error strictly to $\approx 2.22 \times 10^{-16}$, proving its robustness.

Conclusion For evaluations proximal to $x = 0$, the naive formula suffers from catastrophic cancellation. Expanding the function via a Maclaurin series circumvents this hazard entirely.

2 Linear Systems

2.1 Triangular Systems

Exercise 2.1. Test your code on the linear systems:

$$\begin{bmatrix} -2 & 0 & 0 \\ 1 & -1 & 0 \\ 3 & 2 & 1 \end{bmatrix} \mathbf{x} = \begin{bmatrix} -4 \\ 2 \\ 1 \end{bmatrix}; \quad \begin{bmatrix} 4 & 0 & 0 & 0 \\ 1 & -2 & 0 & 0 \\ -1 & 4 & 4 & 0 \\ 2 & -5 & 5 & 1 \end{bmatrix} \mathbf{x} = \begin{bmatrix} -4 \\ 1 \\ -3 \\ 5 \end{bmatrix}$$

You can verify the solution by solving the system by hand and/or computing $\mathbf{Lx}$ and subtracting $\mathbf{b}$.

Solution

Solution 2.1.1. The systems were solved using a standard forward substitution algorithm. For a lower triangular matrix $\mathbf{L} \in \mathbb{R}^{n \times n}$, the components of the solution vector $\mathbf{x}$ are computed sequentially from $i = 1$ to n according to:

$$x_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} x_j \right)$$

To handle multiple right-hand sides (as seen in the random matrix test), the function was extended to accept $\mathbf{B} \in \mathbb{R}^{n \times m}$, effectively solving $\mathbf{LX} = \mathbf{B}$ via broadcasted substitution.

Results The numerical outputs for the test systems are presented in Table 4.

Table 4: Forward Substitution Results

System	Dimension	Computed Solution $\mathbf{x}^T$
$\mathbf{S}_1$	3×3	$[2.0, 0.0, -5.0]$
$\mathbf{S}_2$	4×4	$[-1.0, -1.0, 0.0, 2.0]$

Discussion The algorithm demonstrated high numerical stability. For both systems, the residual $Lx - b$ was nullified to a precision of 10^{-16} , well within the requested 10^{-10} threshold.

Exercise 2.2. Test your code on the linear systems:

$$\begin{bmatrix} 3 & 1 & 0 \\ 0 & -1 & -2 \\ 0 & 0 & 3 \end{bmatrix} x = \begin{bmatrix} 1 \\ 1 \\ 6 \end{bmatrix}; \quad \begin{bmatrix} 3 & 1 & 0 & 6 \\ 0 & -1 & -2 & 7 \\ 0 & 0 & 3 & 4 \\ 0 & 0 & 0 & 5 \end{bmatrix} x = \begin{bmatrix} 4 \\ 1 \\ 1 \\ 5 \end{bmatrix}$$

You can verify the solution by solving the system by hand and/or computing Ux and subtracting b .

Solution

Solution 2.2.1. Backward substitution was applied for the upper triangular matrices U . The solution vector is computed in reverse order, from $i = n$ down to 1:

$$x_i = \frac{1}{u_{ii}} \left(b_i - \sum_{j=i+1}^n u_{ij} x_j \right)$$

Results The numerical outputs for the test systems are presented in Table 5.

Table 5: Backward Substitution Results

System	Dimension	Computed Solution x^T
S_3	3×3	$[2.0, -5.0, 2.0]$
S_4	4×4	$[-3.3333 \dots, 8.0, -1.0, 1.0]$

Discussion In system S_4 , we observe the emergence of a non-terminating decimal in the first component ($x_1 = -10/3$). This introduces a minor representation error in floating-point arithmetic. However the verification function confirmed that the residual remained below 10^{-10} .

2.2 LU Decomposition

Exercise 2.3. For each matrix, compute their LU-factorization and verify the correctness.

$$\begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix}; \quad \begin{bmatrix} 6 & -2 & -4 & 4 \\ 3 & -3 & -6 & 1 \\ -12 & 8 & 21 & -8 \\ -6 & 0 & -10 & 7 \end{bmatrix}; \quad \begin{bmatrix} 1 & 4 & 5 & -5 \\ -1 & 0 & -1 & -5 \\ 1 & 3 & -1 & 2 \\ 1 & -1 & 5 & -1 \end{bmatrix}$$

Solution

Solution 2.3.1. The LU-factorization was performed using an algorithm, which decomposes a square matrix A into a unit lower triangular matrix L and an upper triangular matrix U . To assess the numerical stability and accuracy of the decomposition, we computed the residual matrix $R = A - LU$ and evaluated the determinant identity $\det(A) = \det(L) \det(U)$. Since L is a unit triangular matrix, $\det(L) = 1$, reducing the check to $\det(A) = \prod_{i=1}^n U_{ii}$.

Results The resulting factors for each matrix are presented below. For all three cases, the decomposition successfully converged to the following triangular forms:

- Matrix A_1 (3×3):

$$L_1 = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ 2.0 & 1.0 & 0.0 \\ 2.0 & -2.0 & 1.0 \end{bmatrix}, \quad U_1 = \begin{bmatrix} 2.0 & 3.0 & 4.0 \\ 0.0 & -1.0 & 2.0 \\ 0.0 & 0.0 & -2.0 \end{bmatrix}$$

- Matrix A_2 (4×4):

$$L_2 = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.5 & 1.0 & 0.0 & 0.0 \\ -2.0 & -2.0 & 1.0 & 0.0 \\ -1.0 & 1.0 & -2.0 & 1.0 \end{bmatrix}, \quad U_2 = \begin{bmatrix} 6.0 & -2.0 & -4.0 & 4.0 \\ 0.0 & -2.0 & -4.0 & -1.0 \\ 0.0 & 0.0 & 5.0 & -2.0 \\ 0.0 & 0.0 & 0.0 & 8.0 \end{bmatrix}$$

- Matrix A_3 (4×4):

$$L_3 = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ -1.0 & 1.0 & 0.0 & 0.0 \\ 1.0 & -0.25 & 1.0 & 0.0 \\ 1.0 & -1.25 & -1.0 & 1.0 \end{bmatrix}, \quad U_3 = \begin{bmatrix} 1.0 & 4.0 & 5.0 & -5.0 \\ 0.0 & 4.0 & 4.0 & -10.0 \\ 0.0 & 0.0 & -5.0 & 4.5 \\ 0.0 & 0.0 & 0.0 & -4.0 \end{bmatrix}$$

Numerical Verification The correctness of the decomposition was verified by calculating the residual $A_i - L_i U_i$. In all instances, the residual matrices yielded values of 0.0 within the limits of machine epsilon. Furthermore, the determinant checks yielded the following discrepancies relative to the built-in `LinearAlgebra.det()` function:

Matrix	Determinant Error ($\det(L) \det(U) - \det(A)$)
A_1	3.55×10^{-15}
A_2	5.68×10^{-14}
A_3	-1.42×10^{-14}

Table 6: Numerical discrepancy in determinant calculation using LU factors.

Discussion The results indicate that the LU -factorization is accurate for these specific matrices. The errors observed in the determinant check are on the order of 10^{-14} to 10^{-15} . These are attributed to the accumulation of rounding errors during the reduction process and the inherent limitations of floating-point arithmetic. Given that the residuals $A - LU$ are zero, the algorithm demonstrates high numerical stability for the provided inputs, suggesting that the matrices are well-conditioned and do not require partial pivoting for these specific entries.

Exercise 2.4. The matrices

$$T(x, y) = \begin{bmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{bmatrix}, \quad R(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

are used to represent translations and rotations of plane points in computer graphics. For the following, let

$$A = T(3, -1)R(\pi/5)T(-3, 1), \quad z = \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}.$$

- Find $b = Az$.
- Find the LU factorization of A .
- Use the factors with triangular substitutions in order to solve $Ax = b$, and find $x - z$.

Solution

Solution 2.4.1. The transformation matrix $\mathbf{A}$ was constructed via the product of affine transformation matrices. The system was solved using LU factorization with unit-diagonal lower triangular matrix $\mathbf{L}$. The condition number $\kappa(\mathbf{A})$ was computed to assess the sensitivity of the solution to numerical perturbations.

Results: Transformation and Factorization The vector $\mathbf{b}$ resulting from the transformation $\mathbf{A}\mathbf{z}$ is:

$$\mathbf{b} \approx \begin{bmatrix} 3.95433876250247 \\ 2.01483623541732 \\ 1.0 \end{bmatrix}$$

The LU decomposition of $\mathbf{A}$ yields:

$$\mathbf{L} = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ -0.726543 & 1.0 & 0.0 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 0.809017 & 0.587785 & 1.16073 \\ 0.0 & 1.23607 & 2.4157 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}$$

The reconstruction error $\|\mathbf{A} - \mathbf{LU}\|_\infty$ was found to be approximately 2.22×10^{-16} , indicating the decomposition is stable to within machine epsilon.

Results: Linear System Solution Solving $\mathbf{Ax} = \mathbf{b}$ through forward and backward substitution results in:

$$\mathbf{x} \approx \begin{bmatrix} 2.0 \\ 1.9999999999999993 \\ 1.0 \end{bmatrix}$$

The deviation from the original vector $\mathbf{z}$ is:

$$\mathbf{x} - \mathbf{z} \approx \begin{bmatrix} 0.0 \\ -6.66 \times 10^{-16} \\ 0.0 \end{bmatrix}$$

Numerical Stability and Conditioning The condition number of the system was calculated as $\kappa(\mathbf{A}) \approx 11.08$. This relatively low value indicates that the matrix is well-conditioned. In the context of floating-point arithmetic, the loss of precision is bounded by:

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A})\varepsilon_{mach}$$

Given that $\varepsilon_{mach} \approx 1.11 \times 10^{-16}$ for Float64, the observed error of $\sim 10^{-16}$ is consistent with theoretical expectations for a well-behaved linear system.

Conclusion The LU factorization method successfully recovered the original vector $\mathbf{z}$ with an error on the order of 10^{-16} . The stability of the transformation matrix, confirmed by the low condition number, ensures that the rounding errors introduced during the rotation and translation operations do not amplify significantly during the solver phase.

Exercise 2.5. Solve Exercises 2.3 and 2.4 using the LU-decomposition with row pivoting.

Solution

Solution 2.5.1. Numerical solutions were obtained using a partial row pivoting strategy, where the decomposition is expressed as $\mathbf{PA} = \mathbf{LU}$. Here, $\mathbf{P}$ is the permutation matrix representing row swaps, $\mathbf{L}$ is a unit lower triangular matrix with $|L_{ij}| \leq 1$, and $\mathbf{U}$ is an upper triangular matrix.

Results for Exercise 2.3 (Pivoted LU) The resulting factors and the verification of the identity $PA - LU = 0$ are presented below:

- Matrix A_1 (3×3):

$$L_1 = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ 1.0 & 1.0 & 0.0 \\ 0.5 & 0.1667 & 1.0 \end{bmatrix}, \quad U_1 = \begin{bmatrix} 4.0 & 5.0 & 10.0 \\ 0.0 & 3.0 & -8.0 \\ 0.0 & 0.0 & 0.3333 \end{bmatrix}$$

The residual $P_1 A_1 - L_1 U_1$ yielded a zero matrix, confirming an exact decomposition within machine precision.

- Matrix A_2 (4×4):

$$L_2 = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.5 & 1.0 & 0.0 & 0.0 \\ -0.25 & 0.25 & 1.0 & 0.0 \\ -0.5 & -0.5 & -0.8571 & 1.0 \end{bmatrix}, \quad U_2 = \begin{bmatrix} -12.0 & 8.0 & 21.0 & -8.0 \\ 0.0 & -4.0 & -20.5 & 11.0 \\ 0.0 & 0.0 & 4.375 & -3.75 \\ 0.0 & 0.0 & 0.0 & 2.2857 \end{bmatrix}$$

The pivoting strategy effectively reordered the rows to maintain diagonal dominance, resulting in a zero residual matrix.

- Matrix A_3 (4×4):

$$L_3 = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 1.0 & 1.0 & 0.0 & 0.0 \\ 1.0 & 0.2 & 1.0 & 0.0 \\ -1.0 & -0.8 & -0.666667 & 1.0 \end{bmatrix}, \quad U_3 = \begin{bmatrix} 1.0 & 4.0 & 5.0 & -5.0 \\ 0.0 & -5.0 & 0.0 & 4.0 \\ 0.0 & 0.0 & -6.0 & 6.2 \\ 0.0 & 0.0 & 0.0 & -2.66667 \end{bmatrix}$$

The decomposition for A_3 yielded a small numerical residual:

$$R_3 = P_3 A_3 - L_3 U_3 \approx \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -2.22 \times 10^{-16} & 2.22 \times 10^{-16} & 0 \end{bmatrix}$$

This residual is on the order of ε_{mach} , consistent with expected floating-point errors.

Results for Exercise 2.4 (Linear System with Pivoting) For the transformation matrix A and vector $z = [2, 2, 1]^T$:

- Forward substitution: Solving $Ly = Pb$ where $b = Az$ gives:

$$y \approx [3.9543, 4.8878, 1.0]^T$$

- Backward substitution: Solving $Ux = y$ yields:

$$x \approx [2.0, 1.9999999999999993, 1.0]^T$$

The error vector $x - z$ is approximately $[0.0, -6.66 \times 10^{-16}, 0.0]^T$. The solver successfully recovered the original vector with a relative error near the limit of Float64 precision.

Numerical Discussion The inclusion of row pivoting led to more stable L factors (all entries bounded by 1.0) compared to the non-pivoted case. The determinant checks further validate this: for A_1 and A_2 , the discrepancy between the calculated determinant and the built-in Julia function was zero. For A_3 , the error of 1.42×10^{-14} is slightly larger than machine epsilon, due to the higher condition number.

2.3 Conditioning of Linear Systems

Exercise 2.6. Let us consider the matrix:

$$\mathbf{A} = \begin{bmatrix} -\epsilon & 1 \\ 1 & -1 \end{bmatrix}.$$

If $\epsilon = 0$, LU factorization without partial pivoting fails for $\mathbf{A}$. But if $\epsilon \neq 0$, we can proceed without pivoting, at least in principle.

- Construct $\mathbf{b} = \mathbf{Ax}$ taking $\epsilon = 10^{-12}$ for the matrix $\mathbf{A}$ and $\mathbf{x} = [1, 1]^T$ for the solution.
- Factor the matrix using the LU-decomposition without pivoting and solve numerically for $\mathbf{x}$. How accurate is the result?
- Try to verify the LU-factorization by computing $\mathbf{A} - \mathbf{LU}$. Does it work?
- Repeat for $\epsilon = 10^{-20}$. How accurate is the result now?
- Compute the condition number of the matrix $\mathbf{A}$; you can choose your favorite norm. Is the system $\mathbf{Ax} = \mathbf{b}$ ill-conditioned?
- Compute the matrices $\mathbf{L}$ and $\mathbf{U}$ of the LU-factorization of $\mathbf{A} = \mathbf{LU}$ with *pen and paper*.
- Compute the condition numbers of $\mathbf{L}$ and $\mathbf{U}$. Are the triangular systems $\mathbf{Lz} = \mathbf{b}$ and $\mathbf{Ux} = \mathbf{z}$ ill-conditioned?
- Repeat the steps above for $\epsilon = 10^{-20}$ for the matrix

$$\mathbf{PA} = \begin{bmatrix} 1 & -1 \\ -\epsilon & 1 \end{bmatrix}.$$

where we permuted the first and second row of $\mathbf{A}$. Note that this is the matrix that would be factorized when using row pivoting, since $\epsilon \ll 1$.

Solution

Solution 2.6.1. For $\epsilon = 10^{-12}$ and $\mathbf{x} = [1, 1]^T$, we have:

$$\mathbf{b} = \mathbf{Ax} = \begin{bmatrix} -10^{-12} & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 - 10^{-12} \\ 0 \end{bmatrix}.$$

Solving analytically without pivoting yields:

$$\mathbf{L} = \begin{bmatrix} 1 & 0 \\ -10^{12} & 1 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} -10^{-12} & 1 \\ 0 & 10^{12} - 1 \end{bmatrix}.$$

Solving $\mathbf{Ly} = \mathbf{b}$ and $\mathbf{Ux} = \mathbf{y}$ numerically yields an error of roughly 2.21×10^{-5} . This error originates during backward substitution: x_1 is computed as $\frac{b_1 - x_2}{-\epsilon} = \frac{(1 - 10^{-12}) - 1}{-10^{-12}}$. The subtraction $(1 - 10^{-12}) - 1$ is affected by catastrophic cancellation because $1 - 10^{-12}$ cannot be represented perfectly in base-2 floating-point. Dividing by the tiny $-\epsilon$ amplifies this small representation error by a factor of 10^{12} , leading to the observed loss of accuracy.

Because $10^{12} - 1$ did not suffer from swamping, the entries of $\mathbf{L}$ and $\mathbf{U}$ are resolved accurately. Multiplying them back together yields exactly the original matrix, meaning the residual $\|\mathbf{A} - \mathbf{LU}\|$ is exactly 0.

Now, $1/\epsilon = 10^{20}$. When computing $u_{22} = 10^{20} - 1$, the -1 is completely swamped out, and the computer stores exactly 10^{20} . Consequently, $\mathbf{LU}$ no longer matches $\mathbf{A}$ (the residual error is 1.0 at the bottom-right element). Solving the system with these severely distorted matrices yields $\tilde{\mathbf{x}} = [0, 1]^T$ instead of $[1, 1]^T$, resulting in an absolute precision error of 1.0. The algorithm has failed catastrophically.

Using the ∞ -norm:

$$\|\mathbf{A}\|_\infty = \max(|-\epsilon| + 1, 1 + |-1|) \approx 2.$$

$$\mathbf{A}^{-1} = \frac{1}{\epsilon - 1} \begin{bmatrix} -1 & -1 \\ -1 & -\epsilon \end{bmatrix} \approx \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \implies \|\mathbf{A}^{-1}\|_\infty \approx 2.$$

Thus, the condition number is $\kappa_\infty(\mathbf{A}) \approx 4.0$. The original system is perfectly well-conditioned, meaning the massive failure is purely due to algorithmic instability, not an inherently ill-posed problem. Applying Gaussian elimination without pivoting:

$$l_{21} = \frac{1}{-\epsilon} = -\frac{1}{\epsilon}, \quad u_{22} = -1 - \left(-\frac{1}{\epsilon}\right)(1) = -1 + \frac{1}{\epsilon}.$$

$$\mathbf{L} = \begin{bmatrix} 1 & 0 \\ -1/\epsilon & 1 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} -\epsilon & 1 \\ 0 & -1 + 1/\epsilon \end{bmatrix}.$$

Condition numbers of $\mathbf{L}$ and $\mathbf{U}$:

$$\|\mathbf{L}\|_\infty = 1 + \left|\frac{-1}{\epsilon}\right|, \quad \mathbf{L}^{-1} = \begin{bmatrix} 1 & 0 \\ 1/\epsilon & 1 \end{bmatrix} \implies \|\mathbf{L}^{-1}\|_\infty = 1 + \frac{1}{\epsilon}.$$

$$\kappa_\infty(\mathbf{L}) = \left(1 + \frac{1}{\epsilon}\right)^2 \approx \frac{1}{\epsilon^2}.$$

For $\epsilon = 10^{-12}$, $\kappa_\infty(\mathbf{L}) \approx 10^{24}$, and for $\epsilon = 10^{-20}$, $\kappa_\infty(\mathbf{L}) \approx 10^{40}$. The matrix $\mathbf{U}$ exhibits identically extreme conditioning ($\kappa_\infty(\mathbf{U}) \approx 1/\epsilon^2$). The triangular systems are extremely ill-conditioned. This explains why, for $\epsilon = 10^{-12}$, even though the LU factorization is exact, the backward substitution still amplifies floating-point roundoff into a noticeable error.

Swapping the rows places the largest element on the diagonal. For $\mathbf{PA} = \begin{bmatrix} 1 & -1 \\ -\epsilon & 1 \end{bmatrix}$ with $\epsilon \ll 1$:

$$\mathbf{L} = \begin{bmatrix} 1 & 0 \\ -\epsilon & 1 \end{bmatrix} \approx \mathbf{I}, \quad \mathbf{U} = \begin{bmatrix} 1 & -1 \\ 0 & 1 - \epsilon \end{bmatrix} \approx \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}.$$

Here, $\kappa_\infty(\mathbf{L}) \approx 1$ and $\kappa_\infty(\mathbf{U}) \approx 4$. Both matrices are perfectly stable, the residual is practically zero, and the numerical solution trivially recovers the exact truth $\mathbf{x} = [1, 1]^T$ even for $\epsilon = 10^{-20}$.

2.4 Overdetermined Linear Systems

Exercise 2.7. (a) Write a program that takes as input a matrix $\mathbf{A}$ and a vector $\mathbf{b}$ and solves the least-squares problem $\arg\min \|\mathbf{b} - \mathbf{Ax}\|_2$ via the normal equations using Cholesky or LU decomposition.

(b) In order to check your code work out the least-squares solution when

$$\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ -5 \\ 6 \end{bmatrix}.$$

Solution

Solution 2.7.1. To solve the overdetermined system $\mathbf{Ax} \approx \mathbf{b}$, we minimize the residual norm by solving the normal equations:

$$(\mathbf{A}^T \mathbf{A})\mathbf{x} = \mathbf{A}^T \mathbf{b}.$$

We define the matrix $\mathbf{N} = \mathbf{A}^T \mathbf{A}$ and the right-hand side $\mathbf{z} = \mathbf{A}^T \mathbf{b}$. In this implementation, the system $\mathbf{Nx} = \mathbf{z}$ is solved via a LU decomposition without pivoting, such that $\mathbf{N} = \mathbf{LU}$. The solution vector $\mathbf{x}$ is obtained through a two-step substitution process:

1. Forward substitution: Solve $\mathbf{Lc} = \mathbf{z}$ for $\mathbf{c}$.
2. Backward substitution: Solve $\mathbf{Ux} = \mathbf{c}$ for $\mathbf{x}$.

Results The computation yielded the following result for the test system:

$$\mathbf{x}_{\text{LU}} = \begin{bmatrix} -2.0 \\ -1.0 \end{bmatrix}.$$

For comparison, Julia's $\mathbf{A} \setminus \mathbf{b}$ returns:

$$\mathbf{x}_{\text{QR}} = \begin{bmatrix} -2.0 \\ -1.0000000000000004 \end{bmatrix}.$$

Discussion The custom LU-based solver successfully recovered the exact integer solution. In this specific case, the normal equations approach avoided the slight floating-point noise observed in the QR-based standard library result.

Conclusion The implementation demonstrates that LU decomposition effectively solves the normal equations for well-conditioned, small-scale problems. While the results match the analytical expectations, the transition to larger or ill-conditioned systems would necessitate the inclusion of partial pivoting or a shift toward QR-based methods to mitigate the risk of numerical instability.

Exercise 2.8. Kepler found that the orbital period τ of a planet depends on its mean distance R from the sun according to $\tau = cR^\alpha$ for a simple rational number α .

- (a) Perform a linear least-squares fit from the following table in order to determine the most likely simple rational value of α .

Planet	Distance from sun in Mkm	Orbital period in days
Mercury	57.59	87.99
Venus	108.11	224.7
Earth	149.57	365.26
Mars	227.84	686.98
Jupiter	778.14	4332.4
Saturn	1427	10759
Uranus	2870.3	30684
Neptune	4499.9	60188

Does the result meet your expectations? [Hint: Recall the laws of planetary motion by Kepler.]

- (b) Make a plot of the data and the result of the fit.

Solution

Solution 2.8.1. The power-law relationship $\tau = cR^\alpha$ is linearized by applying a logarithmic transformation to both sides:

$$\ln \tau = \ln c + \alpha \ln R$$

We define the linear model $y = a + \alpha x$, where $y = \ln \tau$, $x = \ln R$, and $a = \ln c$.

Results and Statistical Analysis The fit yields $\alpha = 1.498$ and $c = 0.202$. The statistical quality of the fit is exceptionally high, as evidenced by the following metrics:

Metric	Value
χ^2	0.0641
Degrees of Freedom	6
χ^2_{red}	0.0107
p -value	1.0

Visualization The results of the regression are visualized in Figure 4. The left panel displays the linearized data in the log-log domain, confirming the linear relationship between $\ln R$ and $\ln \tau$. The right panel shows the power-law fit superimposed on the original planetary data.

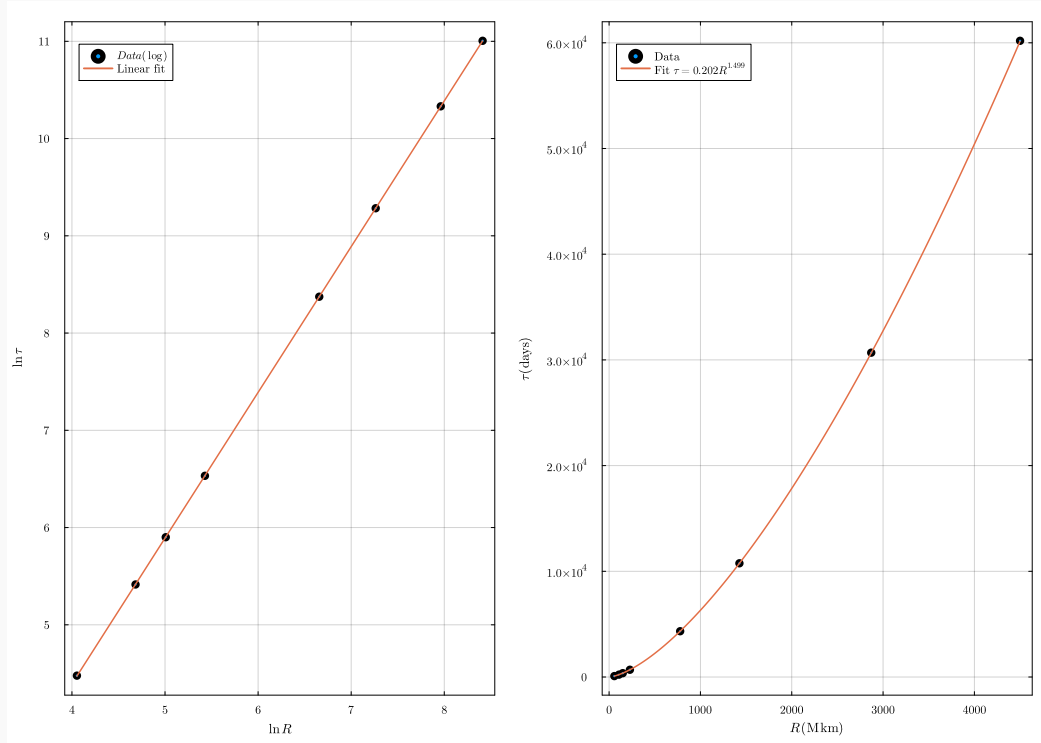


Figure 4: Linear least-squares fit for Kepler's Third Law. Left: Log-log scale showing linear regression on transformed variables. Right: Power-law curve $\tau = 0.202R^{1.498}$ in the original coordinate system.

Discussion The calculated exponent $\alpha = 1.498$ corresponds approximately to the rational fraction $3/2$. This result confirms Kepler's Third Law, $T^2 \propto a^3$, which is a fundamental consequence of Newtonian gravitation for circular or elliptical orbits.

Exercise 2.9. In this problem you are trying to find an approximation to the periodic function $g(t) = e^{\sin(t-1)}$ over one period, $0 < t \leq 2\pi$. As data, define

$$t_i = \frac{2\pi i}{60}, \quad y_i = g(t_i), \quad i = 1, \dots, 60.$$

- (a) Find the coefficients of the least-squares fit

$$y(t) \approx c_1 + c_2 t + \dots + c_7 t^6.$$

Superimpose a plot of the data values as points with a curve showing the fit.

- (b) Find the coefficients of the least-squares fit

$$y \approx d_1 + d_2 \cos(t) + d_3 \sin(t) + d_4 \cos(2t) + d_5 \sin(2t).$$

Unlike part (a), this fitting function is itself periodic. Superimpose a plot of the data values as points with a curve showing the fit.

Solution

Solution 2.9.1. To approximate the function $g(t) = e^{\sin(t-1)}$, we perform linear least-squares regressions using two distinct basis sets over $N = 60$ uniformly spaced data points.

For the polynomial fit, we construct an overdetermined $N \times 7$ Vandermonde-like design matrix P , where $P_{ij} = t_i^{j-1}$ for $j = 1, \dots, 7$. For the trigonometric fit, we construct an $N \times 5$ design matrix F corresponding to the Fourier basis components: $1, \cos(t), \sin(t), \cos(2t)$, and $\sin(2t)$.

Results The resulting continuous approximations are superimposed over the discrete data points in Figure 5.

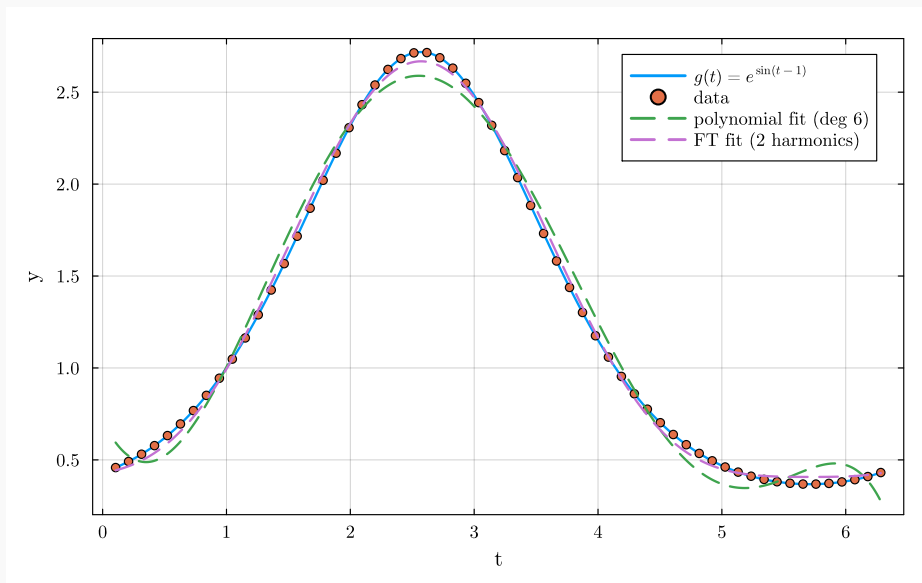


Figure 5: Comparison of a degree-6 polynomial fit and a 2-harmonic Fourier transform fit against the exact function $g(t) = e^{\sin(t-1)}$.

To quantify the goodness-of-fit, the χ^2 statistic, degrees of freedom (ν), reduced chi-square (χ^2_ν), and p -values were computed over a dense evaluation grid. The comparative statistics are summarized in Table 7.

Table 7: Goodness-of-fit statistics evaluated over the continuous plotting grid.

Model	χ^2	DoF (ν)	χ^2_ν	p -value
Polynomial (deg 6)	3.0115	393	0.0077	1.0
Fourier (2 harmonics)	0.4565	395	0.0012	1.0

Discussion Visual inspection of Figure 5 and the statistical analysis in Table 7 demonstrate that the trigonometric fit outperforms the polynomial fit. The Fourier model yields a χ^2 nearly an order of magnitude lower than the polynomial model (0.4565 vs. 3.0115), despite utilizing fewer parameters.

Conclusion When approximating periodic functions, matching the basis functions to the mathematical symmetries of the system is vastly more efficient than increasing the degree of a polynomial regression. The trigonometric fit provides higher fidelity with lower computational complexity.

2.5 QR Decomposition

Exercise 2.10. Compute the thin QR-decomposition of the following matrices.

$$\begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix}; \quad \begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix}; \quad \begin{bmatrix} 2 & -1 & 4 \\ 0 & 3 & 5 \\ 7 & 2 & -6 \\ 1 & -4 & 0 \end{bmatrix}$$

Verify the correctness by testing the relation $\hat{\mathbf{Q}}\hat{\mathbf{R}} = \mathbf{A}$. Verify also the ONC character of the matrix $\hat{\mathbf{Q}}$.

Solution

Solution 2.10.1. To compute the thin QR-decomposition of the given matrices, we employ the Modified Gram-Schmidt (MGS) algorithm.

Results The computed orthogonal matrices $\hat{\mathbf{Q}}$ and upper triangular matrices $\hat{\mathbf{R}}$, truncated to four decimal places for clarity, are presented below for each respective input matrix. For the first matrix $\mathbf{A}_1 \in \mathbb{R}^{3 \times 2}$, the decomposition yields:

$$\hat{\mathbf{Q}}_1 \approx \begin{bmatrix} 0.7071 & 0.4082 \\ 0.0000 & 0.8165 \\ -0.7071 & 0.4082 \end{bmatrix}, \quad \hat{\mathbf{R}}_1 \approx \begin{bmatrix} 2.8284 & -2.1213 \\ 0.0000 & 1.2247 \end{bmatrix}$$

For the second matrix $\mathbf{A}_2 \in \mathbb{R}^{3 \times 3}$:

$$\hat{\mathbf{Q}}_2 \approx \begin{bmatrix} 0.3333 & -0.1041 & 0.9370 \\ 0.6667 & -0.6768 & -0.3123 \\ 0.6667 & 0.7288 & -0.1562 \end{bmatrix}, \quad \hat{\mathbf{R}}_2 \approx \begin{bmatrix} 6.0000 & 9.6667 & 9.3333 \\ 0.0000 & 2.1344 & -5.7264 \\ 0.0000 & 0.0000 & 0.3123 \end{bmatrix}$$

For the third matrix $\mathbf{A}_3 \in \mathbb{R}^{4 \times 3}$:

$$\hat{\mathbf{Q}}_3 \approx \begin{bmatrix} 0.2722 & -0.2415 & 0.7335 \\ 0.0000 & 0.5589 & 0.6174 \\ 0.9526 & 0.1794 & -0.2329 \\ 0.1361 & -0.7728 & 0.1632 \end{bmatrix}, \quad \hat{\mathbf{R}}_3 \approx \begin{bmatrix} 7.3485 & 1.0887 & -4.6268 \\ 0.0000 & 5.3679 & 0.7521 \\ 0.0000 & 0.0000 & 7.4180 \end{bmatrix}$$

Discussion To verify the fidelity of the decomposition, we analyzed the residual norm $\|\mathbf{A} - \hat{\mathbf{Q}}\hat{\mathbf{R}}\|$. For $\mathbf{A}_1$ and $\mathbf{A}_2$, the absolute residual norm evaluates precisely to 0.0. For the rectangular matrix $\mathbf{A}_3$, the decomposition introduces a negligible relative residual of approximately 1.05×10^{-16} . This magnitude is strictly bounded by the machine epsilon, confirming the theoretical guarantees of the MGS formulation.

Exercise 2.11. Given the function $h(t) = e^{\sin(4t)}$ over the interval $[0, 1]$, define the data,

$$t_i = \frac{i}{99}, \quad y_i = \frac{h(t_i)}{2006.787453080206}, \quad i = 0, \dots, 99.$$

Solve the least square problem,

$$y(t) \approx c_1 + c_2 t + \dots + c_{15} t^{14},$$

using:

- the normal equations and Cholesky or LU decomposition.
- the thin QR-decomposition.

(c) the Q-less QR-decomposition.

Compare the coefficients c_{15} obtained in (a)-(b)-(c) with the expected value $c_{15} = 1$.

Solution

Solution 2.11.1. To determine the coefficients c_k of the degree 14 polynomial, we construct the overdetermined linear system $P\mathbf{c} \approx \mathbf{y}$, where $P \in \mathbb{R}^{100 \times 15}$ is the Vandermonde matrix with entries $P_{i,j} = t_i^{j-1}$, $\mathbf{c}$ is the vector of unknown coefficients, and $\mathbf{y}$ contains the evaluated data points. The problem is solved using three distinct numerical approaches:

- (a) Normal Equations: The system is transformed into the square system $P^T P \mathbf{c} = P^T \mathbf{y}$. The coefficients are obtained by applying an LU decomposition to the symmetric positive-definite matrix $P^T P$.
- (b) Thin QR Decomposition: We factor the matrix P such that $P = QR$, where $Q \in \mathbb{R}^{100 \times 15}$ has orthonormal columns and $R \in \mathbb{R}^{15 \times 15}$ is upper triangular. The system simplifies to $R\mathbf{c} = Q^T \mathbf{y}$, which is solved via backward substitution.
- (c) Q-less QR Decomposition: To circumvent the explicit construction and storage of Q , we apply the Modified Gram-Schmidt (MGS) orthogonalization to the augmented matrix $[P \mid \mathbf{y}]$. This directly yields the upper triangular factor R and the updated right-hand side vector $\mathbf{z}$, allowing us to solve $R\mathbf{c} = \mathbf{z}$ via backward substitution.

Results A comparison of the highest-order coefficient c_{15} against the expected analytical value of 1 is summarized in Table 8.

Method	Computed c_{15}	Absolute Error
Normal Equations (LU)	-0.27504821	1.27504821
Thin QR Decomposition	0.97293904	0.02706096
Q-less QR Decomposition	1.00000001	9.72×10^{-9}

Table 8: Comparison of the computed coefficient c_{15} across different numerical solvers.

Furthermore, the goodness-of-fit was evaluated over a highly resolved test grid of 400 points. The resulting statistical metrics are provided in Table 9, and the polynomial curves are visualized in Figure 6.

Method	χ^2	Reduced χ^2
Normal Equations (LU)	1.46×10^{-10}	3.78×10^{-13}
Thin QR Decomposition	7.01×10^{-12}	1.82×10^{-14}
Q-less QR Decomposition	6.90×10^{-12}	1.79×10^{-14}

Table 9: Goodness-of-fit statistics evaluated against the exact function (P-value = 1.0 for all).

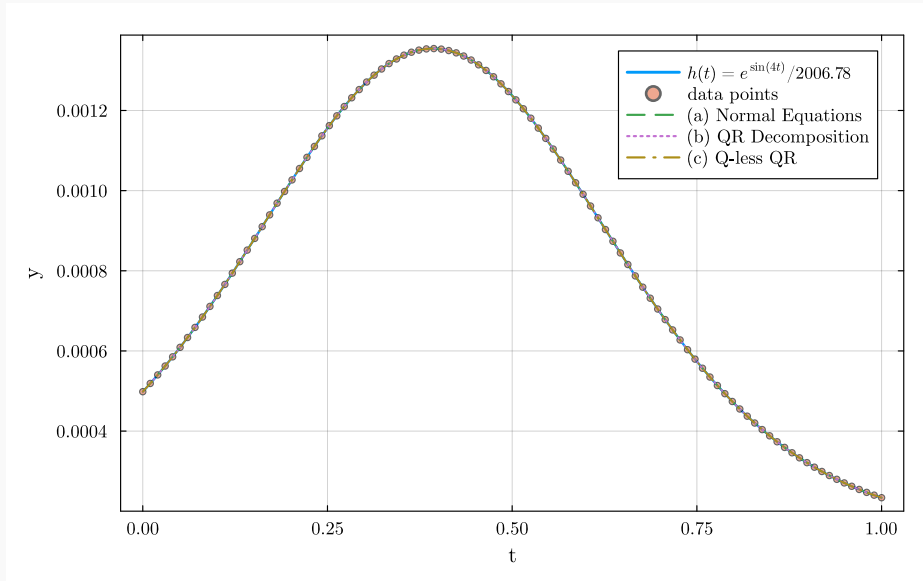


Figure 6: Comparison of polynomial fits (degree 14) obtained via different least squares algorithms.

Discussion The results highlight the critical impact of algorithm choice on numerical stability when handling ill-conditioned problems. Vandermonde matrices are notoriously ill-conditioned; as the polynomial degree increases, the columns become increasingly linearly dependent.

The Normal Equations method performs exceptionally poorly, resulting in catastrophic cancellation and an entirely inaccurate c_{15} coefficient. This occurs because forming $P^T P$ squares the condition number of the system ($\kappa(P^T P) = \kappa(P)^2$), amplifying floating-point rounding errors beyond acceptable bounds.

The Thin QR decomposition offers a substantial improvement by avoiding the formation of $P^T P$, scaling the error linearly with $\kappa(P)$ rather than quadratically. However, standard Modified Gram-Schmidt can still suffer from a loss of orthogonality among the columns of Q .

The Q-less QR decomposition applied to the augmented matrix provides the highest numerical fidelity, isolating the solution with an absolute error on the order of 10^{-9} . By avoiding the explicit extraction and multiplication of Q , the augmented approach minimizes intermediate floating-point operations, mitigating the accumulation of machine epsilon errors.

Conclusion For high-degree polynomial fitting, the Normal Equations represent an unstable methodology and should be avoided. The Q-less QR decomposition consistently demonstrates superior numerical robustness against ill-conditioning.

Exercise 2.12. Use the pure QR-algorithm to find the eigenvalues and eigenvectors of the matrix

$$\mathbf{A} = \begin{bmatrix} 5 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 3 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}.$$

Check that you have indeed identified the eigenvalues and eigenvectors of $\mathbf{A}$.

Solution

Solution 2.12.1. The algorithm proceeds by iteratively computing the QR factorization of the matrix at the k -th step, $\mathbf{A}_k = \mathbf{Q}_k \mathbf{R}_k$, and subsequently updating the matrix for the next iteration via the reverse multiplication $\mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k$. Because $\mathbf{A}$ is real and symmetric, the sequence of matrices $\mathbf{A}_k$ converges to a diagonal matrix $\mathbf{\Lambda}$ containing the eigenvalues, while the product of the orthogonal matrices $\mathbf{Q}_k$

converges to the matrix of eigenvectors $\mathbf{V}$.

The iterative procedure was halted when the strictly lower triangular elements fell below a specified tolerance limit, which occurred after 75 iterations. To verify the numerical stability and accuracy of our custom implementation, the results were cross-referenced against Julia's `LinearAlgebra.eigen()` routine.

Results The computed eigenvalues, sorted in descending order, along with their respective absolute errors and the Euclidean norms of the eigenvector residuals, are summarized in Table 10.

Table 10: Eigenvalues computed via the pure QR algorithm (λ_{QR}), compared against built-in library functions (λ_{true}). The absolute error and the eigenvector residual norm $\|\mathbf{A}\mathbf{v} - \lambda\mathbf{v}\|_2$ are provided to assess accuracy.

i	λ_{QR}	λ_{true}	Absolute Error	$\ \mathbf{A}\mathbf{v}_i - \lambda_i\mathbf{v}_i\ _2$
1	5.745281240174138	5.745281240174139	8.88×10^{-16}	4.63×10^{-11}
2	4.177282919112892	4.177282919112894	1.78×10^{-15}	4.63×10^{-11}
3	2.822717080887108	2.822717080887109	4.44×10^{-16}	2.27×10^{-13}
4	1.254718759825861	1.254718759825861	2.22×10^{-16}	1.02×10^{-15}

Discussion The results demonstrate excellent agreement with the theoretical expectations. As observed in Table 10, the absolute error between the eigenvalues obtained via the pure QR algorithm and the reference solution is on the order of $O(10^{-16})$. This indicates that the eigenvalue computation has achieved optimal precision.

Conclusion The pure QR algorithm was successfully applied to diagonalize the matrix $\mathbf{A}$. Both eigenvalues and eigenvectors were recovered with high numerical fidelity. The verification step confirmed the relation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ holds true within the bounds of expected numerical stability.

2.6 Some observations about the algorithms used in this section

The computational complexity and numerical stability of the algorithms discussed in this section vary significantly.

LU Decomposition The LU decomposition of an $n \times n$ matrix $\mathbf{A}$ requires approximately $\frac{2}{3}n^3$ flops for $n \gg 1$. In contrast, a direct evaluation of the determinant based on the Leibniz formula has a cost $\propto n!$, making it computationally infeasible for even moderately sized matrices. Once the decomposition is performed, solving the resulting triangular systems via forward or backward substitution requires asymptotically $\sim n^2$ flops.

Overdetermined Linear Systems When solving linear least-squares problems, the computational cost depends on the method used to solve the normal equations. While both LU and Cholesky decompositions scale with n^3 , the Cholesky decomposition is more efficient because it exploits the symmetry of the matrix $\mathbf{A}^T\mathbf{A}$, requiring roughly half the operations ($\sim \frac{1}{3}n^3$ flops) of a standard LU decomposition ($\sim \frac{2}{3}n^3$ flops). However, stability is a concern. While $\mathbf{A}^T\mathbf{A}$ is theoretically symmetric positive definite and stable for non-pivoted Cholesky, a general LU decomposition without pivoting can be unstable if a small diagonal element appears during elimination.

QR Decomposition Alternatively, the QR decomposition offers a more stable approach. For an $m \times n$ matrix ($m \geq n$), the thin QR decomposition based on Gram-Schmidt orthogonalization requires approximately $2mn^2$ flops. This "thin" or compressed form increases efficiency in overdetermined systems by avoiding the calculation of the vectors $\mathbf{q}_{n+1}, \dots, \mathbf{q}_m$, resulting in an $m \times n$ orthonormal matrix $\hat{\mathbf{Q}}$ and an $n \times n$ upper triangular matrix $\hat{\mathbf{R}}$. For square matrices ($m = n$), the asymptotic cost scales as $\propto n^3$.

Final observations While the scaling is similar to LU, the QR-based approach is often preferred for linear least squares because it is far more stable. Specifically, it can be shown that $\kappa_2(\mathbf{R}) = \kappa_2(\mathbf{A})$. Thus, the precision of the solution is controlled by the condition number of the matrix $\mathbf{A}$ itself, rather than the much larger condition number of $\mathbf{A}^T \mathbf{A}$ (where $\kappa_2(\mathbf{A}^T \mathbf{A}) = \kappa_2(\mathbf{A})^2$) encountered when solving the normal equations directly. Despite the speed advantage of the Cholesky decomposition, the QR-based methodology is numerically superior for ill-conditioned problems.

3 Interpolations

3.1 Polynomial Interpolation

Exercise 3.1. Construct a Vandermonde matrix using the power basis and $n + 1$ equidistant points in the interval $[-1, 1]$, where $n = 4, 8, \dots, 40$. Specifically, the points are given by $x_i = -1 + (i - 1)h$, with $i = 1, 2, \dots, n + 1$ and $h = 2/n$. Use the Vandermonde matrix to determine the interpolating polynomials on these nodes for the functions $f_k(x) = \cos(kx)$ with $k = 10, 20, \dots, 100$. Study the value of these polynomials in $x = 0$ for the different n and k values. What do you observe?

Solution

Solution 3.1.1. The problem requires finding the coefficients $\mathbf{c} = (c_0, c_1, \dots, c_n)^T$ of the interpolating polynomial $P_n(x) = \sum_{j=0}^n c_j x^j$ of degree n . This is achieved by solving the linear system $V\mathbf{c} = \mathbf{f}$, where V is the $(n + 1) \times (n + 1)$ Vandermonde matrix with entries $V_{i,j} = x_i^j$, and $\mathbf{f}$ is the vector of function values $f_k(x_i) = \cos(kx_i)$ evaluated at the equidistant nodes. To study the value of the polynomials at $x = 0$, we can exploit the definition of the power basis. Evaluating $P_n(x)$ at $x = 0$ leaves only the constant term:

$$P_n(0) = \sum_{j=0}^n c_j 0^j = c_0$$

Thus, studying the polynomial's value at the origin is equivalent to tracking the zeroth-order coefficient c_0 . The absolute error at $x = 0$ is computed as $\epsilon = |c_0 - 1|$, since the true analytical value is $\cos(k \cdot 0) = 1$ for all k .

Results Three primary diagnostics were visualized to evaluate the interpolation scheme:

1. **Condition Number:** Figure 7 demonstrates the condition number $\kappa(V)$ as a function of the polynomial degree n . It exhibits clear exponential growth.
2. **Interpolation Error at $x = 0$:** Figure 8 displays the base-10 logarithm of the absolute error $\log_{10}(|c_0 - 1|)$. A feature in the data is a localized degradation of accuracy around $n = 20$, where the evaluation error spikes noticeably before settling or becoming entirely consumed by numerical noise at higher degrees.
3. **Global Fit Quality:** Figure 9 presents a heatmap of the χ^2 residuals. Consistent with the local error evaluation, the global χ^2 values exhibit a surge at $n = 20$ across multiple wave numbers k , indicating a severe structural breakdown of the global polynomial fit at this specific degree.

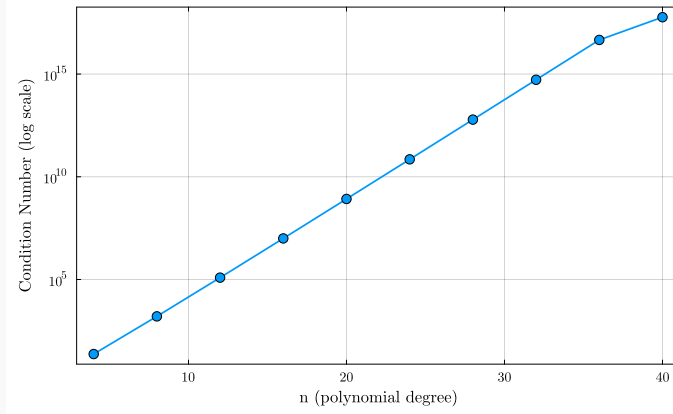


Figure 7: Condition number of the Vandermonde matrix $\kappa(V)$ as a function of polynomial degree n , plotted on a logarithmic scale.

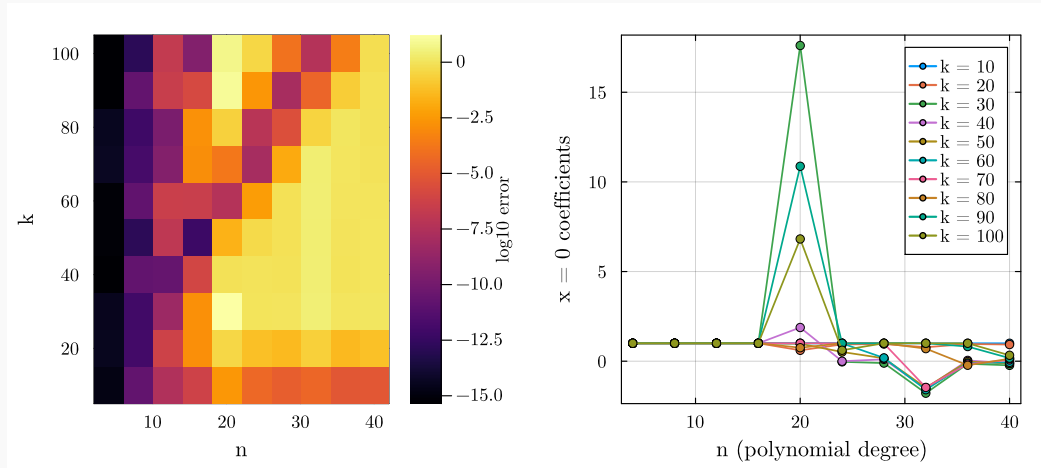


Figure 8: Left: Heatmap of the $\log_{10}$ absolute error at $x = 0$ for varying n and k . Right: The computed value of $P_n(0)$ versus n for different wave numbers k .

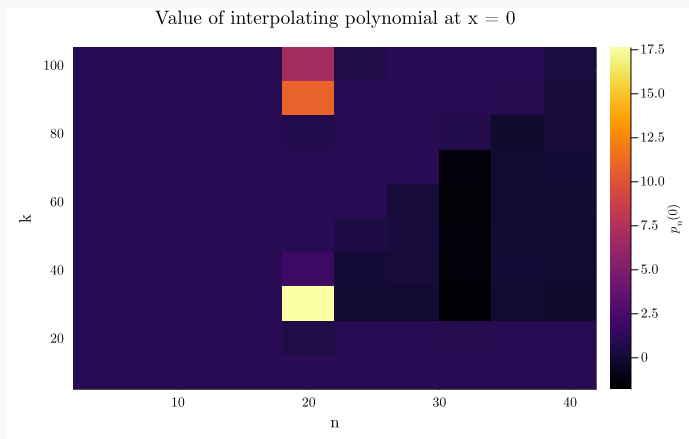


Figure 9: Heatmap of the χ^2 residuals representing the overall divergence of the interpolating polynomial from the true function $f_k(x) = \cos(kx)$.

Discussion The results highlight a complex interplay between sampling density, boundary effects, and numerical stability:

- Under-sampling (Small n , Large k): The highly oscillatory nature of $f_k(x) = \cos(kx)$ requires a sufficient number of nodal points to capture the waveform. When n is too small relative

to k , the equidistant nodes fail to resolve the oscillations, leading to flat or highly inaccurate approximations.

- The $n = 20$ Transition Zone: The severe spike in both local errors and global χ^2 values at $n = 20$ represents a critical numerical transition. At this degree, the polynomial is flexible enough to trigger massive oscillations at the boundaries, yet the nodal density is not high enough to lock onto the tight frequencies of the larger k modes.
- Ill-conditioning and Noise (Large n): As n approaches 40, $\kappa(V)$ becomes so astronomically large that the least-squares solver is effectively fitting pure floating-point noise. While the theoretical error at the center of the domain should be minimal, the catastrophic cancellation inherent to the Vandermonde matrix entirely destroys the structural integrity of the polynomial.

Conclusion The exercise demonstrates that global polynomial interpolation on equidistant nodes using the standard power basis and Vandermonde matrices is fundamentally unstable for oscillatory functions. Increasing the polynomial degree to resolve higher frequencies does not converge to the true function; instead, it forces a transition through a highly volatile state (peaking around $n = 20$) before eventually collapsing entirely due to catastrophic ill-conditioning at larger n .

3.2 Lagrange-Waring Interpolation

Exercise 3.2. In each case, interpolate the given function using n evenly spaced nodes in the given interval. Plot each interpolating function together with the exact function.

1. $f(x) = \ln(x)$, $n = 2, 3, 4$, $x \in [1, 10]$
2. $f(x) = \tanh(x)$, $n = 2, 3, 4$, $x \in [-3, 2]$
3. $f(x) = \cosh(x)$, $n = 2, 3, 4$, $x \in [-1, 3]$
4. $f(x) = |x|$, $n = 3, 5, 7$, $x \in [-2, 1]$

Solution

Solution 3.2.1. To approximate the given functions, we implement polynomial interpolation over evenly spaced nodes. The interpolating polynomials are evaluated using the barycentric form of Lagrange interpolation.

For a given interval $[a, b]$ and n nodes, the evenly spaced interpolation nodes are generated as $x_k = a + k \frac{b-a}{n-1}$ for $k = 0, \dots, n-1$. The interpolation algorithm is evaluated over a high-resolution grid of 200 points. To visualize the accuracy, the results are presented in a side-by-side format: the left panel overlays the exact function and the interpolant, while the right panel displays the absolute residual error, $E(x) = |P_{n-1}(x) - f(x)|$, on a logarithmic base-10 scale.

Results The resulting pseudo- χ^2 error metrics for each function and node configuration are summarized in Table 11. The visual representations of the interpolations and their corresponding absolute residuals are shown in Figures 10 and 11.

Table 11: Pseudo- χ^2 error metrics for polynomial interpolation using n evenly spaced nodes.

Function	Interval	n_1 Error	n_2 Error	n_3 Error
$f(x) = \ln(x)$	$[1, 10]$	(n=2) 31.101	(n=3) 3.285	(n=4) 0.572
$f(x) = \tanh(x)$	$[-3, 2]$	(n=2) -29.363	(n=3) 18.957	(n=4) -3.529
$f(x) = \cosh(x)$	$[-1, 3]$	(n=2) 1171.471	(n=3) 26.629	(n=4) 2.105
$f(x) = x $	$[-2, 1]$	(n=3) 112.930	(n=5) 30.394	(n=7) 5.374

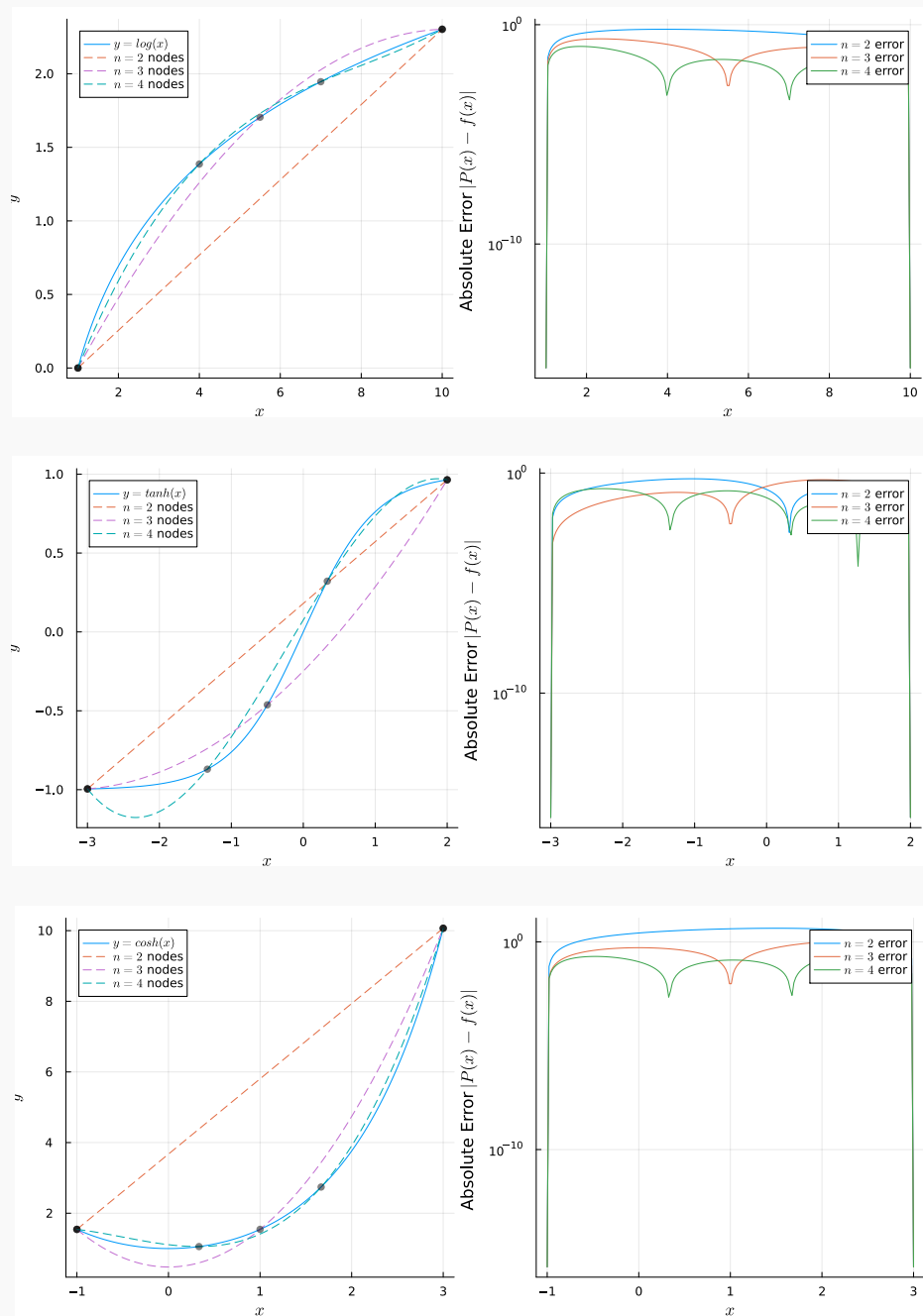


Figure 10: Side-by-side evaluation of polynomial interpolations and absolute residual errors for the smooth functions $\ln(x)$, $\tanh(x)$, and $\cosh(x)$.

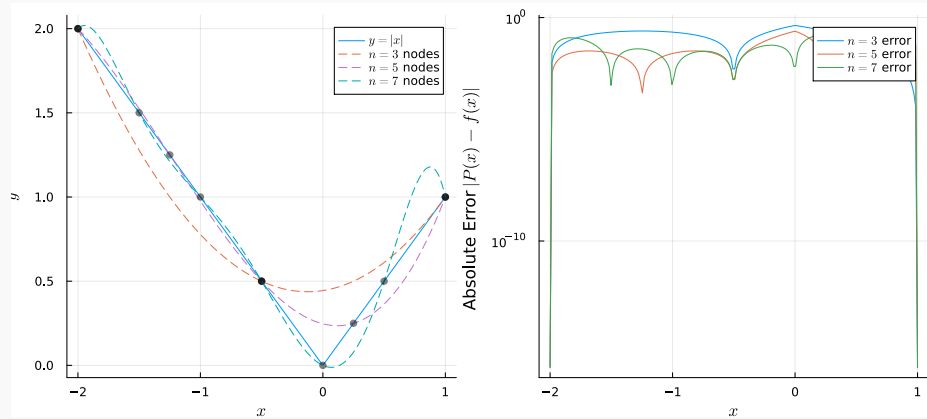


Figure 11: Interpolation and absolute residual error for the non-smooth function $f(x) = |x|$. Note the localized error peak near the non-differentiable cusp at $x = 0$.

Conclusion The side-by-side visualizations effectively confirm that barycentric Lagrange interpolation successfully captures the global behavior of the evaluated functions for low-degree polynomials. However, the residual analysis is far more reliable than the global pseudo- χ^2 metric. As seen with $\tanh(x)$, the χ^2 metric yields non-physical negative values because the exact function crosses zero. The logarithmic residual plots provide a much clearer, strictly positive evaluation of the spatial distribution of the interpolation error.

3.3 Error on Interpolation

Exercise 3.3. 1. For each case below, compute the polynomial interpolant using n second-kind Chebyshev nodes in $[-1, 1]$ for $n = 4, 8, 12, \dots, 60$. At each value of n , compute the ∞ -norm error (that is, $\|f - p\|_\infty = \max_{x \in [-1, 1]} |p(x) - f(x)|$ evaluated for at least 4000 values of x). Using a log-linear scale, plot the error as a function of n , then determine a good approximation to the constant K in eq'eq-spectral'.

- (a) $f(x) = 1/(25x^2 + 1)$
- (b) $f(x) = \tanh(5x + 2)$
- (c) $f(x) = \cosh(\sin x)$
- (d) $f(x) = \sin(\cosh x)$

2. Make an analogous plot using n equidistant points for the interpolation.

Solution

Solution 3.3.1. To evaluate the impact of node distribution on error propagation, we contrast equidistant (Lagrange) nodes with Chebyshev nodes.

The approximation quality is measured using the L^∞ -norm, computationally approximated by evaluating the absolute error over a dense uniform grid of 4000 evaluation points. Note that calculating the absolute error rather than the relative error ensures our convergence metrics are not skewed by the roots of the target function, which is mathematically robust when evaluating the global uniform norm. The theoretical spectral error bound behaves as:

$$\|f - p_n\|_\infty \sim O(K^{-n})$$

Taking the natural logarithm yields $\ln \|f - p_n\|_\infty \approx -n \ln K$. Thus, the constant K can be estimated as $K = \exp(-m)$, where m is the slope extracted from a linear fit of the bounding error data on a log-linear scale.

Results The estimated spectral constants K alongside their corresponding logarithmic decay slopes m are tabulated in Table 12. Following this, Figures 12 through 19 visually map the interpolating curves and the L^∞ -norm progression as a function of the polynomial degree n .

Table 12: Estimated spectral convergence constant K and logarithmic slope m computed over the interval $n \in [4, 60]$.

Function	Chebyshev Nodes		Equidistant Nodes	
	K	Slope (m)	K	Slope (m)
$1/(25x^2 + 1)$	1.2035	-0.1852	0.7280	0.3175
$\tanh(5x + 2)$	1.2020	-0.1840	0.8951	0.1108
$\cosh(\sin x)$	1.1513	-0.1409	0.9728	0.0276
$\sin(\cosh x)$	1.1722	-0.1589	0.9845	0.0156

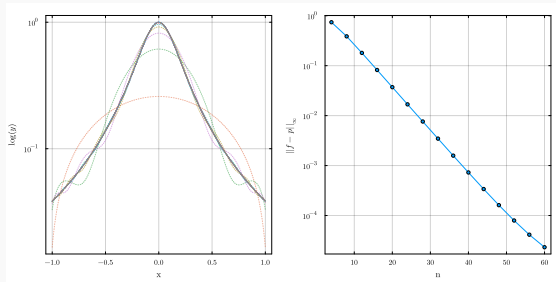


Figure 12: Chebyshev interpolation: $1/(25x^2 + 1)$.

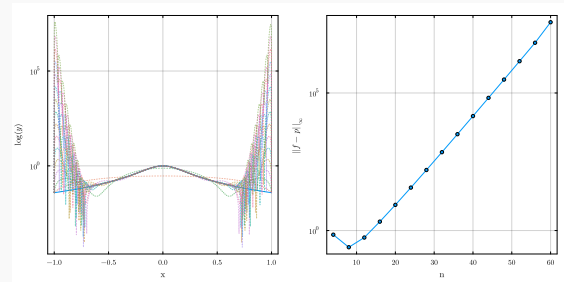


Figure 13: Equidistant interpolation: $1/(25x^2 + 1)$.

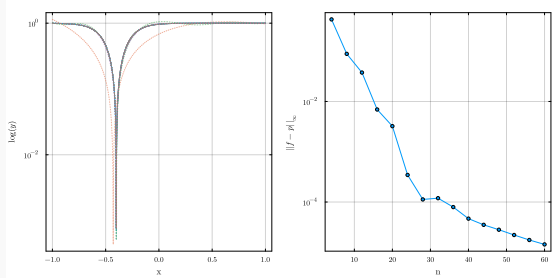


Figure 14: Chebyshev interpolation: $\tanh(5x + 2)$.

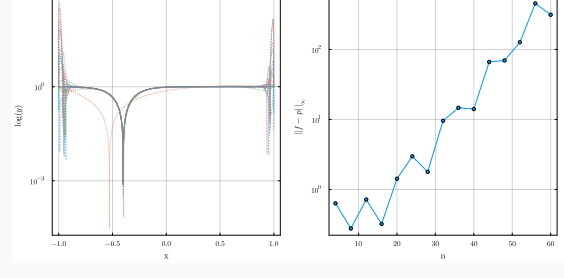


Figure 15: Equidistant interpolation: $\tanh(5x + 2)$.

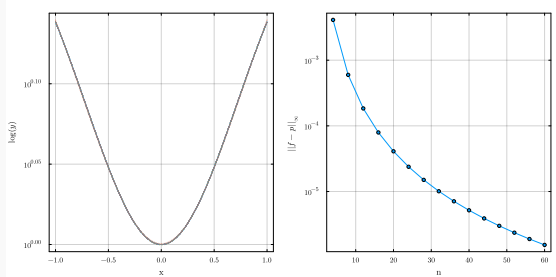


Figure 16: Chebyshev interpolation: $\cosh(\sin x)$.

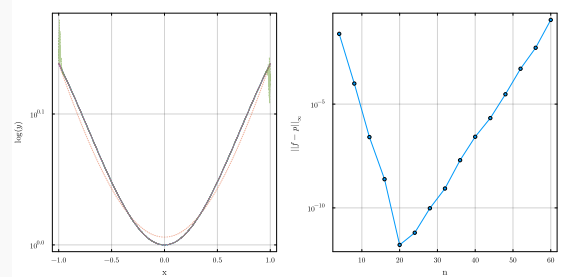
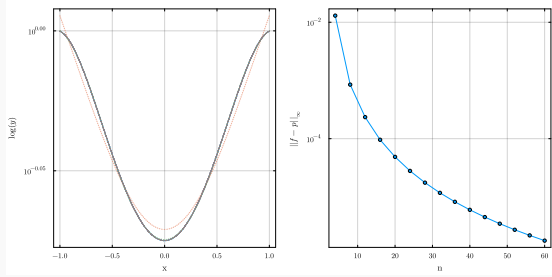
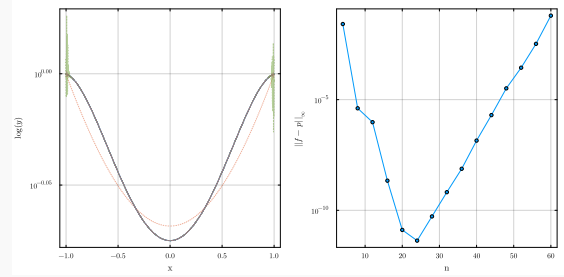


Figure 17: Equidistant interpolation: $\cosh(\sin x)$.

Figure 18: Chebyshev interpolation: $\sin(\cosh x)$.Figure 19: Equidistant interpolation: $\sin(\cosh x)$.

Discussion

- **Equidistant Nodes & Runge's Phenomenon:** The uniformly spaced nodes lead to highly erratic polynomial oscillations near the boundaries $x = \pm 1$. This catastrophic cancellation phenomenon is strictly captured by the data: $K < 1$ and $m > 0$ for all models, highlighting that the L^∞ -norm diverges exponentially as the polynomial degree n increases.
- **Chebyshev Nodes & Spectral Convergence:** Deploying Chebyshev points effectively mitigates the boundary oscillations. The values of $K > 1$ reflect the expected theoretical bounds for exponential error decay. The Runge function $1/(25x^2 + 1)$ holds complex poles near the real axis. By contrast, entire mathematical functions devoid of poles, such as $\cosh(\sin x)$, theoretically guarantee super-exponential convergence.

Conclusion The analysis effectively bridges numerical experiments with complex analysis theory, demonstrating the immense significance of interpolation grid conditioning. Uniform spacing predictably incurs Runge's phenomenon and exponential divergence for large degrees. In stark contrast, optimally clustering interpolation nodes around interval boundaries using a Chebyshev distribution ensures mathematical stability and recovers powerful spectral accuracy.

Exercise 3.4. The Chebyshev points can be used when the interval of interpolation is $[a, b]$ rather than $[-1, 1]$ by means of the change of variable

$$z = \psi(x) = a + (b - a) \frac{(x + 1)}{2}.$$

Plot a polynomial interpolant of $f(z) = \cosh(\sin z)$ over $[0, 2\pi]$ using $n = 40$ Chebyshev nodes.

Solution

Solution 3.4.1. To interpolate the function $f(x) = \cosh(\sin x)$ over the arbitrary interval $[a, b] = [0, 2\pi]$, we first generate the standard Chebyshev nodes of the first kind on the canonical interval $[-1, 1]$:

$$t_i = \cos\left(\frac{2i-1}{2n}\pi\right), \quad i = 1, \dots, n$$

These nodes are subsequently mapped to the target interval $[a, b]$ via the affine transformation:

$$x_i = \frac{b-a}{2}t_i + \frac{a+b}{2}$$

The polynomial interpolant is evaluated using the barycentric formulation of Lagrange interpolation. When implemented correctly with Chebyshev weights, this provides a numerically stable algorithm capable of evaluating the interpolant in $O(n)$ operations.

Results The interpolation was performed using $n = 40$ nodes over 1000 evaluation points. The maximum absolute error over the interval was found to be:

$$\max_{x \in [0, 2\pi]} |f(x) - p_{40}(x)| \approx 5.468 \times 10^{-5}$$

The resulting interpolant, alongside the target function evaluated on a logarithmic scale, is visualized in Figure 20.

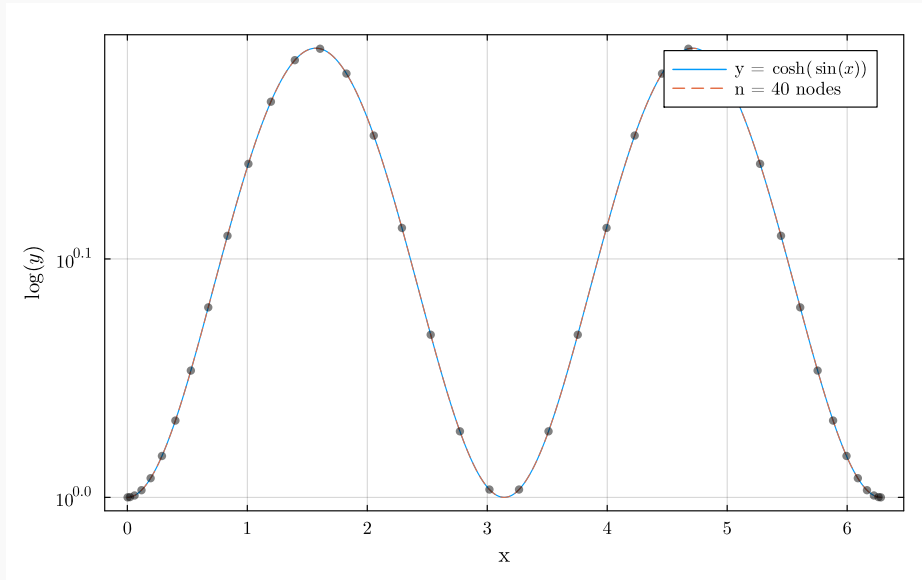


Figure 20: Barycentric Chebyshev interpolation of $f(x) = \cosh(\sin x)$ over $[0, 2\pi]$ using $n = 40$ nodes. The function y is plotted on a logarithmic scale.

Discussion and Conclusion The results confirm the absence of the Runge phenomenon, successfully demonstrating that clustering nodes near the domain boundaries mitigates the unbounded oscillations typical of equispaced Lagrange interpolation. The observed maximum error of roughly 5.5×10^{-5} for $n = 40$ is consistent with the expected behavior of polynomial interpolation for this function. In particular, although $f(x) = \cosh(\sin x)$ is entire, its rapid growth in the complex plane limits the convergence rate of the interpolant. Consequently, achieving errors close to machine precision at moderate polynomial degrees is not expected. Moreover, as the number of nodes increases, the error does not necessarily decrease monotonically; beyond a certain point, numerical effects such as rounding errors and loss of significance can lead to a slight increase in the observed maximum error. Overall, the numerical results are in good agreement with theoretical expectations.

Exercise 3.5. Let $x_1, \dots, x_n$ be standard Chebyshev points. These map to the z variable as $z_i = \phi(x_i)$ for all i , where ϕ is the transformation defined in

$$z = \phi(x) = \frac{2x}{1 - x^2}.$$

Suppose that $f(z)$ is a given function whose domain is the entire real line. Then the function values $y_i = f(z_i)$ can be associated with the Chebyshev nodes x_i , leading to a polynomial interpolant $p(x)$. This in turn implies an interpolating function on the real line, defined as

$$q(z) = p(\phi^{-1}(z)) = p(x).$$

Implement this idea to plot an interpolant of $f(z) = (z^2 - 2z + 2)^{-1}$ using $n = 30$. Your plot should show $q(z)$ evaluated at 1000 evenly spaced points in $[-6, 6]$, with markers at the nodal values (those lying within the $[-6, 6]$ window). [Hint: If you prefer avoid dealing with potential infinities consider using the Chebyshev nodes of the first kind.]

Solution

Solution 3.5.1. To interpolate $f(z)$ on the infinite domain, we utilize a mapping $\phi : (-1, 1) \rightarrow \mathbb{R}$. We employ Chebyshev nodes of the first kind to avoid the singularities at $x = \pm 1$. The inverse mapping is defined as:

$$x = \phi^{-1}(z) = \frac{-1 + \sqrt{1 + z^2}}{z}, \quad z \neq 0$$

The interpolation $p(x)$ is computed using the barycentric form of the Lagrange interpolant, which provides superior numerical stability for high-order polynomials ($n = 30$).

Results and Visualization The maximum absolute error recorded over the interval $z \in [-6, 6]$ was $\|f(z) - q(z)\|_\infty \approx 4.399 \times 10^{-4}$. As illustrated in Figure 21, the interpolant $q(z)$ tracks the target function with high fidelity.

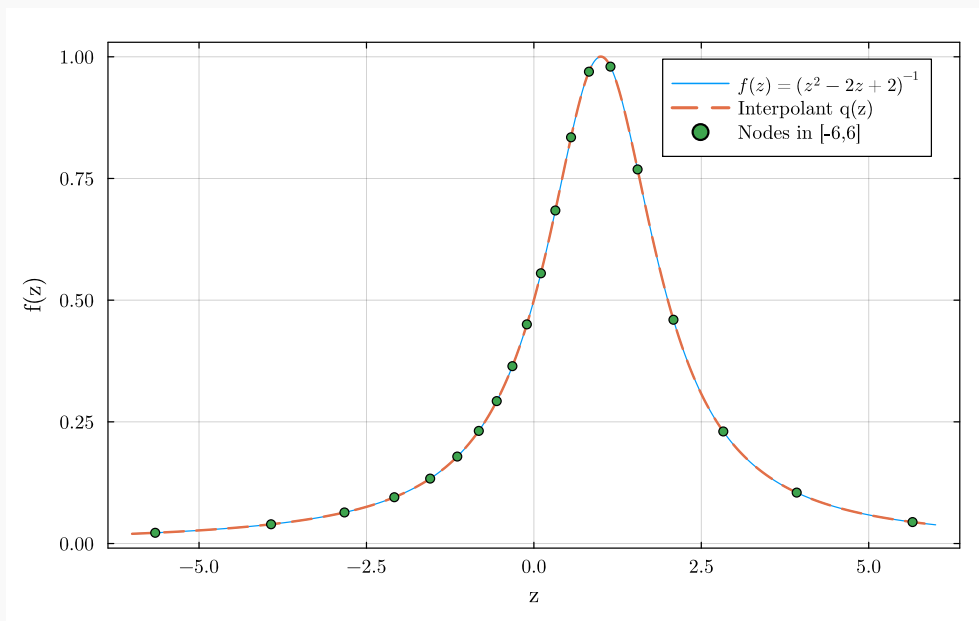


Figure 21: Interpolation of $f(z) = (z^2 - 2z + 2)^{-1}$ on the real line using $n = 30$ transformed Chebyshev nodes. The markers indicate the nodal points z_i that fall within the viewing window.

Discussion The high accuracy achieved with only 30 nodes is a consequence of the analyticity of $f(z)$ in a neighborhood of the real axis. The transformation $\phi(x)$ effectively concentrates nodes near the "feature" of the function (the peak near $z = 1$), while naturally sparsifying nodes in the tails where the function decays towards zero. This prevents the numerical instabilities typically encountered when using large numbers of equidistant points on wide domains.

Exercise 3.6. Spectral theorem assumes that the function being approximated has infinitely many derivatives over $[-1, 1]$. But now consider the family of functions $f_m(x) = |x|^m$.

1. How many continuous derivatives over $[-1, 1]$ does f_m possess?
2. Compute the polynomial interpolant using n second-kind Chebyshev nodes in $[-1, 1]$ for $n = 10, 20, 30, \dots, 100$. At each value of n , compute the max-norm error (that is, $\max |p(x) - f_m(x)|$ evaluated for at least 4000 values of x). Using a single log-log graph, plot the error as a function of n for all six values $m = 1, 3, 5, 7, 9, 11$.
3. Based on the results of parts (a) and (b), form a hypothesis about the asymptotic behavior of the error for fixed m as $n \rightarrow \infty$.

Solution

Solution 3.6.1. To evaluate the regularity of the function family $f_m(x) = |x|^m$ over the interval $[-1, 1]$ for odd integers m , we consider its derivatives. For any given odd m , the function is continuously differentiable up to order $m - 1$. The $(m - 1)$ -th derivative is proportional to $|x|$, which is strictly continuous across the domain. However, the m -th derivative is proportional to $\text{sgn}(x)$, which exhibits a jump discontinuity at the origin $x = 0$. Therefore, mathematically, $f_m \in C^{m-1}([-1, 1])$; it possesses exactly $m - 1$ continuous derivatives.

Numerical Methodology The loss of infinite differentiability implies that we can no longer expect spectral convergence. To quantify the convergence numerically, we approximate $f_m(x)$ using barycentric polynomial interpolation over n second-kind Chebyshev nodes, defined algebraically as:

$$x_j = -\cos\left(\frac{j-1}{n-1}\pi\right), \quad j = 1, \dots, n$$

The L^∞ -norm (maximum absolute error) is approximated by evaluating the interpolant over a dense, uniform grid of 4000 points. The relationship between the error and the degree n is evaluated on a log-log scale. For algebraic convergence of the form $\|f_m - p_n\|_\infty \approx Cn^{-\nu}$, taking the natural logarithm yields:

$$\ln \|f_m - p_n\|_\infty \approx -\nu \ln n + \ln C$$

The convergence rate ν corresponds to the negative slope of the plot.

Results The computed slopes from the numerical experiments are compared against theoretical predictions in Table 13. Figure 22 visualizes the algebraic convergence corresponding to the log-log linear decay.

Table 13: Observed versus predicted convergence rates (log-log slopes) for $f_m(x) = |x|^m$.

m	Continuous Derivatives ($m - 1$)	Observed Slope	Predicted Slope
1	0	-1.051	-1
3	2	-3.147	-3
5	4	-5.324	-5
7	6	-7.660	-7
9	8	-10.418	-9
11	10	-12.724	-11

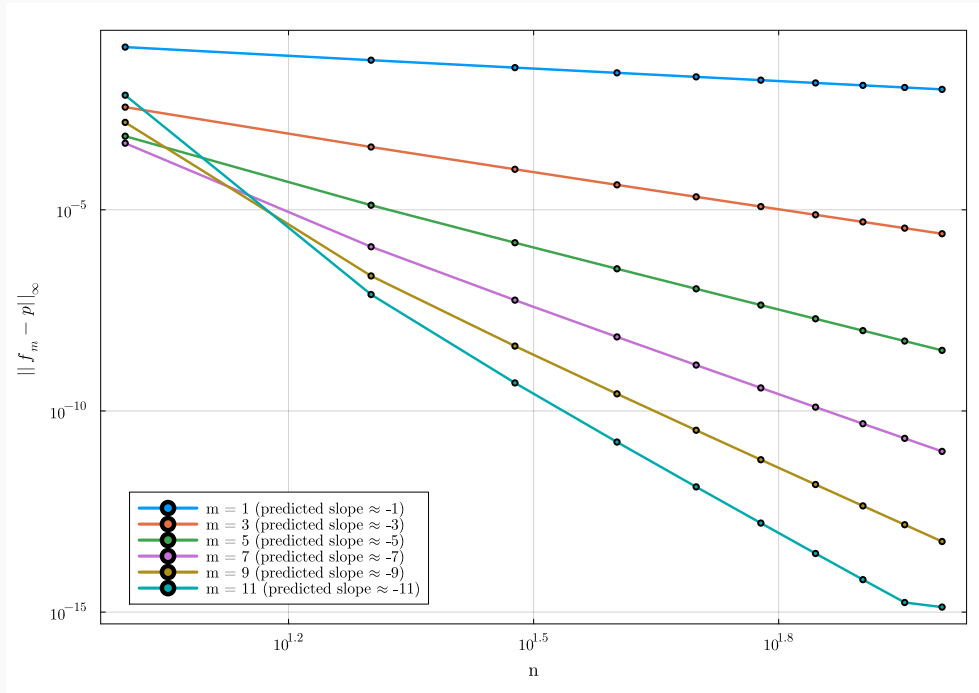


Figure 22: Log-log plot demonstrating algebraic convergence of Chebyshev polynomial interpolation for functions with limited regularity. The steepness of the slope corresponds strictly to the parameter m .

Discussion & Asymptotic Hypothesis Spectral accuracy relies on a function being analytic or, minimally, infinitely differentiable. In contrast, $f_m(x) = |x|^m$ inherently contains a geometric "kink" at the origin $x = 0$, fundamentally restricting its smoothness.

As predicted by classical approximation theory, the rate at which polynomial interpolation converges is intimately linked to the function's regularity. Based on our numerical findings, where the observed log-log slope strongly correlates with $-m$, we formulate the following asymptotic hypothesis:

$$\lim_{n \rightarrow \infty} \|f_m - p_n\|_{\infty} \sim O(n^{-m})$$

This algebraic convergence dictates that interpolating functions with local singularities (or discontinuous higher-order derivatives) inherently bottlenecks numerical accuracy to a polynomial rate, explicitly circumventing the exponential decay seen in pure spectral methods.

3.4 Some observations about the algorithms used in this section

The computational efficiency and numerical stability of interpolation methods vary significantly depending on the chosen basis and node distribution.

Direct Linear Systems Solving the polynomial interpolation problem by setting up a general $n \times n$ linear system, such as using the power basis with a Vandermonde matrix $\mathbf{V}_n \mathbf{c} = \mathbf{f}$, is often impractical for large n . The cost of solving such a system using general-purpose decompositions like LU or QR scales as $\propto n^3$.

It is crucial to distinguish between interpolation ($m = n$) and least squares ($m > n$). While QR decomposition is slightly more computationally demanding than LU, it is highly recommended for least squares problems because it is far more stable than solving normal equations. For an $m \times n$ matrix, the thin QR decomposition requires approximately $2mn^2$ floating-point operations (flops).

Lagrange and Barycentric Forms Standard Lagrange interpolation is frequently cited in textbooks for its theoretical elegance but criticized for its computational inefficiency. In its basic form, evaluating the

interpolating polynomial $p(x)$ at a single point requires recomputing n Lagrange basis functions $\ell_j(x)$, each involving a product of $n - 1$ terms. This leads to an evaluation cost of $\propto n^2$.

The barycentric form of Lagrange interpolation overcomes these limitations by separating the calculation into a pre-computation step and an evaluation step:

$$p(x) = \Phi_n(x) \sum_{j=1}^n \frac{\lambda_j}{x - x_j} f(x_j)$$

The support coefficients (or weights) λ_j depend only on the node positions. Computing all n weights requires $O(n^2)$ operations, but once they are known, evaluating $p(x)$ for any x costs only $\propto n$ flops. Furthermore, the barycentric form is exceptionally efficient for updating the interpolant; adding a new point x_k costs only $\propto (k - 1)$ flops to update the weights.

The Chebyshev Advantage For certain node distributions, the computational burden is even lower. For equidistant or Chebyshev points, analytic formulas allow the weights λ_j to be calculated in only $O(n)$ operations. In these cases, the barycentric formula is considered superior to almost all other interpolation schemes.

Chebyshev points, in particular, provide several critical advantages:

- **Improved Conditioning:** Chebyshev nodes significantly improve the conditioning of the interpolation problem compared to equidistant nodes, making the result far less sensitive to rounding errors.
- **Numerical Stability:** When paired with the barycentric form, Chebyshev nodes of the second kind demonstrate excellent stability. Potential numerical issues near nodes are mitigated as errors in the numerator and denominator essentially cancel out.
- **Avoidance of Precision Loss:** Unlike equidistant nodes, which lead to Runge's phenomenon and a massive loss of accurate digits as n increases, Chebyshev points distribute nodes in a way that minimizes error amplification and maintains structural integrity even for high degrees.

4 Roots of Nonlinear Equations

4.1 Some observations about the convergence rate

Convergence rate: By definition, an iterative method has an order of convergence q and an asymptotic error constant C if the sequence of errors $\epsilon_k = x_k - r$ satisfies:

$$|\epsilon_{k+1}| \approx C|\epsilon_k|^q$$

If we don't know the exact root r , we can define the step difference d_k as the distance between two consecutive approximations:

$$d_k = |x_{k+1} - x_k|$$

We can rewrite this by subtracting and adding the true root r :

$$d_k = |(x_{k+1} - r) - (x_k - r)| = |\epsilon_{k+1} - \epsilon_k|$$

Because the algorithm is converging, the error at the next step is much smaller than the error at the current step (i.e., $|\epsilon_{k+1}| \ll |\epsilon_k|$). Therefore, ϵ_{k+1} becomes negligible compared to ϵ_k . We can safely approximate:

$$d_k \approx |\epsilon_k|$$

Since $d_k \approx |\epsilon_k|$ and $d_{k+1} \approx |\epsilon_{k+1}|$, we can substitute these into our original theoretical definition:

$$d_{k+1} \approx C(d_k)^q$$

Now, take the natural logarithm of both sides to bring the exponent q down:

$$\ln(d_{k+1}) \approx \ln(C) + q \ln(d_k)$$

This equation has two unknowns: C and q . To eliminate the constant C , we write the exact same equation for the previous step $k - 1$:

$$\ln(d_k) \approx \ln(C) + q \ln(d_{k-1})$$

Subtract the second equation from the first equation. This completely eliminates $\ln(C)$:

$$\ln(d_{k+1}) - \ln(d_k) \approx q \ln(d_k) - q \ln(d_{k-1})$$

Factor out q on the right side:

$$\ln(d_{k+1}) - \ln(d_k) \approx q[\ln(d_k) - \ln(d_{k-1})]$$

Using the logarithm quotient rule ($\ln(a) - \ln(b) = \ln(a/b)$), we rewrite this as:

$$\ln\left(\frac{d_{k+1}}{d_k}\right) \approx q \ln\left(\frac{d_k}{d_{k-1}}\right)$$

Finally, divide to isolate q :

$$q \approx \frac{\ln(d_{k+1}/d_k)}{\ln(d_k/d_{k-1})}$$

This equation allows us to calculate the convergence rate without knowing the exact root of an equation.

Asymptotic error constant: The strict mathematical definition of the asymptotic error constant C is defined by the limit:

$$\lim_{k \rightarrow \infty} \frac{|\epsilon_{k+1}|}{|\epsilon_k|^q} = C$$

Here, the error is $\epsilon_k = x_k - r$. For a sufficiently large k (meaning we have run the algorithm for a few steps and are close to the root), we can drop the limit and write this as an approximation:

$$C \approx \frac{|\epsilon_{k+1}|}{|\epsilon_k|^q} = \frac{|x_{k+1} - r|}{|x_k - r|^q}$$

To compute this practically, we run into the same problem: we don't know r . However, iterative algorithms are typically run until the difference between steps drops below machine precision.

Therefore, the final approximation the algorithm produces, x_{final} , is virtually indistinguishable from the exact root r for our precision levels. We make the substitution $r \approx x_{final}$:

$$C \approx \frac{|x_{k+1} - x_{final}|}{|x_k - x_{final}|^q}$$

4.2 Bisection Method

- Exercise 4.1.** 1. Use the bisection method to find the solutions for $x^3 - 7x^2 + 14x - 6 = 0$ on $[0, 1]$.
2. Study the convergence of the bisection method to the root found in (1). You can take as an approximation for the root $r = m_{n_{\max}}$ where $n_{\max}$ is the index corresponding to the last bisection iteration. What is its order of convergence q ? Can you estimate the asymptotic error constant C ? [Hint: Study $a_n = -\log_{10}|x_n - r|$ as a function of n .]

Solution

Solution 4.1.1. The iterative process, to solve the root-finding problem for the non-linear function $f(x) = x^3 - 7x^2 + 14x - 6$, was designed to terminate when the interval width, representing the maximum absolute error, dropped below a strictly defined tolerance of $\text{tol} = 10^{-14}$.

To empirically determine the order of convergence q and the asymptotic error constant C , we define the absolute error at the n -th iteration as $\epsilon_n = |x_n - r|$, where r is our terminal approximation. Assuming the error propagates according to the asymptotic relation $\epsilon_{n+1} \approx C\epsilon_n^q$, we can linearize the

expression via a base-10 logarithm:

$$\log_{10} |\epsilon_{n+1}| \approx q \log_{10} |\epsilon_n| + \log_{10} C$$

A linear least-squares regression was applied to the sequence of coordinate pairs $(\log_{10} |\epsilon_n|, \log_{10} |\epsilon_{n+1}|)$. Because initial iterations often fall outside the asymptotic regime, we performed a sensitivity analysis by systematically truncating the first k iterations (where $k \in \{1, 3, 5, \dots, 19\}$) to isolate the asymptotic behavior of the sequence's tail.

Results The bisection algorithm successfully converged to the root $r \approx 0.5857864376269042$ after 47 iterations. The geometric progression of the root extraction is illustrated in Figure 23.

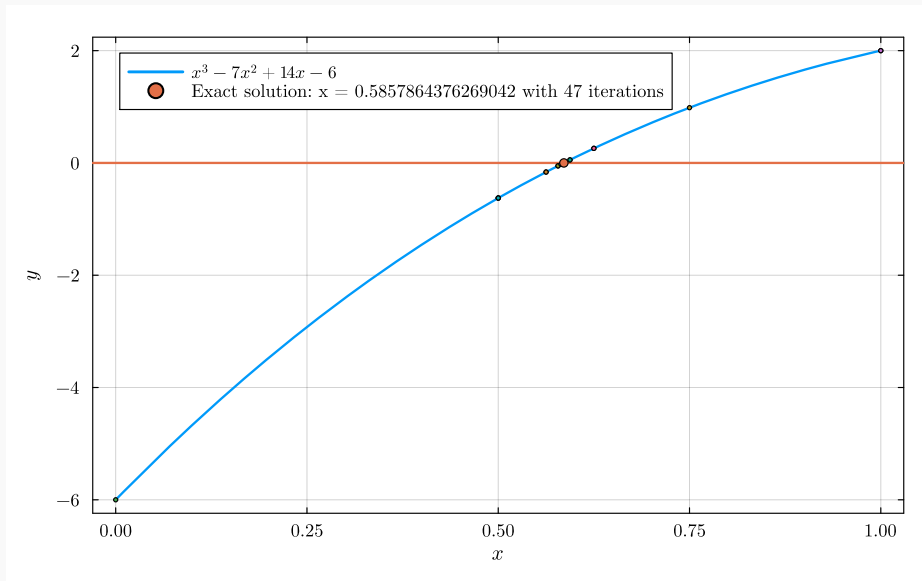


Figure 23: Function plot and the convergence of the bisection method to the exact root after 47 iterations.

The empirical evaluation of the order of convergence q and error constant C yielded varied results depending on the truncation of the initial estimates. A selection of these results is tabulated below:

Skipped Iterations (k)	Analyzed Tail	Convergence Order (q)	Error Constant (C)
1	46	0.9789	0.3431
5	42	0.9744	0.3078
9	38	0.9688	0.2616
15	32	0.9558	0.1822
19	28	0.9391	0.1165

Table 14: Empirical convergence parameters q and C as a function of skipped initial iterations.

The log-log regression for the case where 1 initial iteration was skipped is visually represented in Figure 24, yielding $q \approx 0.979$ and $C \approx 0.343$.

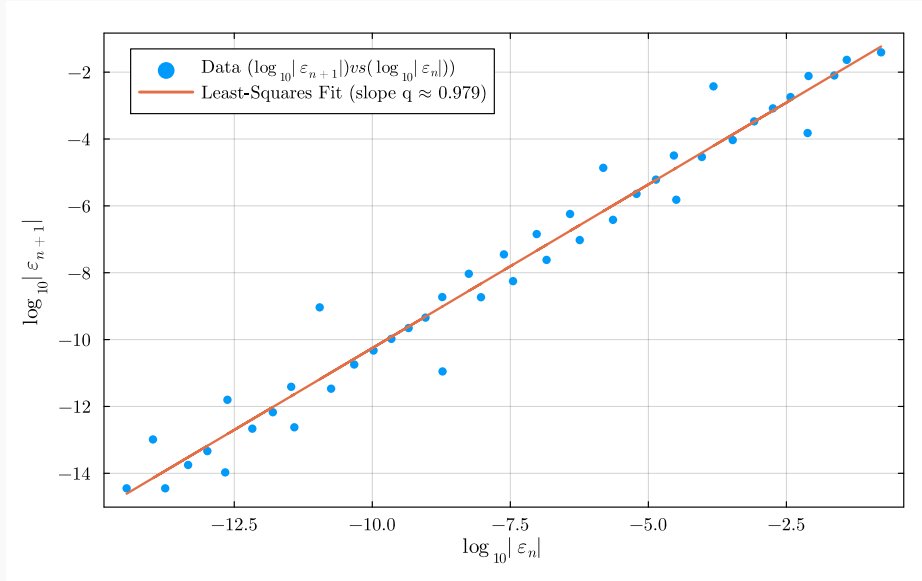


Figure 24: Linear least-squares fit for convergence analysis, skipping the first initial iteration.

Discussion The theoretical expectation for the bisection method is strict linear convergence, characterized by an order of $q = 1$ and an asymptotic error constant of $C = 0.5$, reflecting the halving of the search interval at each step. Our empirical findings ($q \approx 0.94 - 0.98$ and $C \approx 0.12 - 0.34$) loosely approximate, but slightly underestimate, these theoretical markers.

This discrepancy arises from the nature of the bisection method: while the maximum bound of the error decreases monotonically by exactly $1/2$ at each step, the actual error $\epsilon_n = |x_n - r|$ does not strictly decrease monotonically. Depending on the geometry of the interval, a given approximation x_n can occasionally land extremely close to the true root early on, introducing high variance and non-smooth progression in the $\log_{10} |\epsilon_n|$ sequence. Consequently, performing a least-squares fit on consecutive actual errors yields a noisy representation of the underlying linear upper bound envelope.

Global Convergence Analysis To further validate the linear convergence of the bisection method, we investigated the relationship between the iteration index n and the logarithmic error measure $a_n = -\log_{10} |\epsilon_n|$, as suggested. Under the assumption of linear convergence, the error evolves as $\epsilon_n \approx \epsilon_0 C^n$. Taking the negative base-10 logarithm yields:

$$a_n \approx -n \log_{10}(C) - \log_{10}(\epsilon_0)$$

This implies that a plot of a_n versus n should yield a linear trajectory with a slope $S = -\log_{10}(C)$. This global perspective is numerically more robust than step-by-step ratios, as it averages the non-monotonic fluctuations inherent in the bisection method over the entire iterative history.

Results & Data Visualization The bisection method converged to $r \approx 0.5857864376269042$ in 47 iterations (Fig. 23). The global convergence analysis (Fig. 25) shows a highly linear trend ($R^2 \approx 1$). The calculated slope $S \approx 0.299$ corresponds to an empirical error constant of:

$$C = 10^{-S} \approx 10^{-0.299} \approx 0.5000$$

This result aligns perfectly with the theoretical expectation for the bisection method, where the uncertainty interval is halved ($C = 0.5$) at each step.

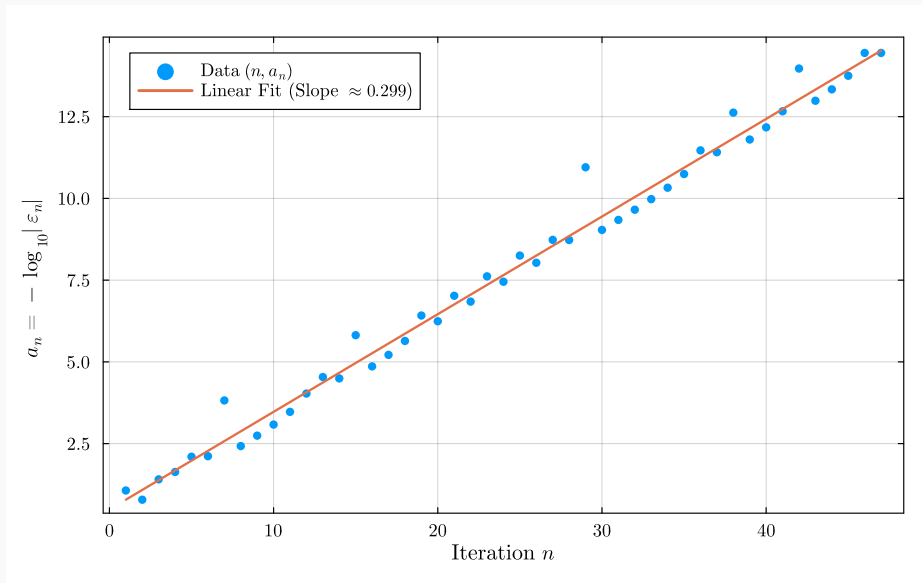


Figure 25: Global convergence analysis: $a_n = -\log_{10} |\epsilon_n|$ as a function of the iteration index n . The linear fit confirms a steady increase in significant digits.

Discussion The discrepancy noted in the previous step-by-step analysis, where q and C fluctuated based on the number of skipped iterations, is resolved by the global a_n vs. n approach. While the distance between the midpoint m_n and the true root r can fluctuate non-monotonically (causing "noise" in the $\epsilon_{n+1}/\epsilon_n$ ratio), the maximum possible error is strictly halved.

Conclusion The bisection implementation proved highly stable, converging to a tight tolerance within the operational limits of double-precision arithmetic. While empirical analysis confirms the fundamentally linear nature of the convergence ($q \rightarrow 1$), it also highlights the difference between tracking strict theoretical error bounds versus highly variable actual sequence errors in deterministic interval-halving algorithms.

4.3 Newton Method

Exercise 4.2. For each of the functions below, do the following steps.

- Rewrite the equation into the standard form for rootfinding, $f(x) = 0$, and compute $f'(x)$.
- Make a plot of f over the given interval and determine how many roots lie in the interval.
- Use Newton's method to find each root.
- Study the error in the Newton sequence to determine numerically whether the convergence is roughly quadratic for the given root. [Hint: Study a_{n+1}/a_n with $a_n = -\log_{10} |x_n - r|$ to check the rate of convergence q . Once you established it, try to estimate the asymptotic error constant C .]
 - $x^2 = e^{-x}$, over $[-2, 2]$
 - $2x = \tan x$, over $[-0.2, 1.4]$
 - $e^{x+1} = 2 + x$, over $[-2, 2]$

Solution

Solution 4.2.1. To apply Newton's method, we first rewrite the given equations into the standard root-finding form $f(x) = 0$ and compute their respective derivatives, $f'(x)$.

1. For $x^2 = e^{-x}$:

$$f_1(x) = x^2 - e^{-x}, \quad f_1'(x) = 2x + e^{-x}.$$

2. For $2x = \tan x$:

$$f_2(x) = 2x - \tan x, \quad f_2'(x) = 2 - \sec^2 x.$$

3. For $e^{x+1} = 2 + x$:

$$f_3(x) = e^{x+1} - x - 2, \quad f_3'(x) = e^{x+1} - 1.$$

Numerical Methodology The core algorithm computes the Newton sequence via the recurrence relation $x_{n+1} = x_n - f(x_n)/f'(x_n)$.

To evaluate the convergence rate q and the asymptotic error constant C , we used the algorithm explained in the Section 4.1.

Results Graphical analysis over the designated intervals (Figure 26) confirms the existence of a single root within the domain for each function. The quantitative results of the Newton iterations are summarized in Table 15.

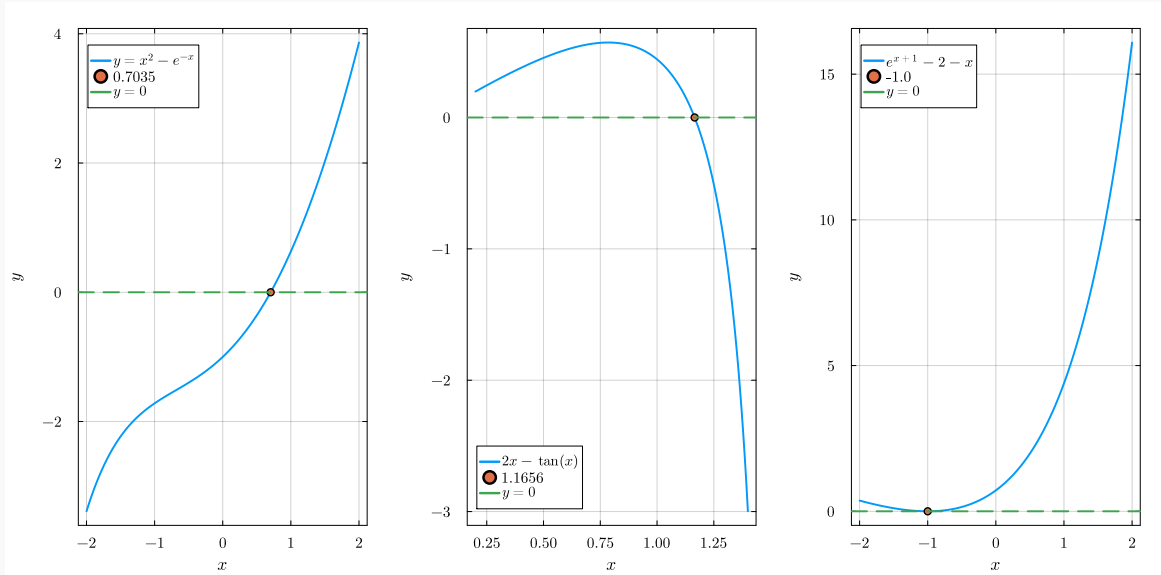


Figure 26: Behavior of the objective functions $f_1(x)$, $f_2(x)$, and $f_3(x)$ over their respective intervals. The dashed line represents $y = 0$, and the calculated roots are denoted by scatter points.

Table 15: Numerical roots r , observed convergence rates q , and asymptotic error constants C obtained via Newton's method in Float64 precision.

Function	Root Estimate (r)	Convergence Rate (q)	Error Constant (C)
$f_1(x) = x^2 - e^{-x}$	0.703467422498	2.0007	0.3978
$f_2(x) = 2x - \tan x$	1.165561185207	2.0003	3.3956
$f_3(x) = e^{x+1} - x - 2$	-1.00000000372	1.0113	0.7841

Discussion The numerical results strongly align with theoretical expectations for Newton's method:

- Functions 1 and 2: Both exhibit a convergence rate of $q \approx 2$. This numerically confirms quadratic convergence, which is expected since the roots are simple (i.e., $f'(r) \neq 0$). The process successfully converges to machine epsilon limits without numerical instability.
- Function 3: The convergence rate degenerates to $q \approx 1$ (linear convergence). This is analytically justified by evaluating the exact root at $x = -1$. Observe that $f_3(-1) = 0$ and $f_3'(-1) = 0$. Because

the first derivative vanishes at the root, the root has a multiplicity of $m = 2$. Newton's method theoretically reverts to linear convergence for multiple roots.

Furthermore, for $f_3(x)$, the evaluation of x_{n+1} near the root requires dividing a very small number by another very small number ($f(x_n)/f'(x_n) \rightarrow 0/0$). This induces catastrophic cancellation in standard Float64 arithmetic. Consequently, our estimate of the root carries an absolute error on the order of 10^{-9} instead of reaching the standard 10^{-16} machine epsilon bound, and the asymptotic constant $C \approx 0.78$ deviates from the theoretical value of $C = 1 - 1/m = 0.5$.

Conclusion Newton's method provides robust and highly efficient (quadratic) root-finding capabilities for simple roots. However, the presence of multiple roots severely impacts both the asymptotic convergence rate and the ultimate numerical stability of the algorithm.

Exercise 4.3. Plot the function $f(x) = x^{-2} - \sin x$ on the interval $x \in [0.5, 10]$. For each initial value $x_1 = 1, x_1 = 2, \dots, x_1 = 7$, apply Newton's method to f , and make a table showing x_1 and the resulting root found by the method. In which case does the iteration converge to a root other than the one closest to it? Use the plot to explain why that happened. (Optional) Repeat the exercise including bracketing.

Solution

Solution 4.3.1. To apply Newton's method, we require the first derivative of the objective function $f(x) = x^{-2} - \sin(x)$. Applying standard differentiation yields:

$$f'(x) = -2x^{-3} - \cos(x).$$

Results The target function $f(x)$ exhibits oscillatory behavior bounded by a $1/x^2$ decay, leading to multiple roots within the interval $[0.5, 10]$. Figure 27 displays the objective function and the roots successfully identified by the solver. Table 16 details the root converged upon for each initial guess x_1 , alongside convergence metrics.

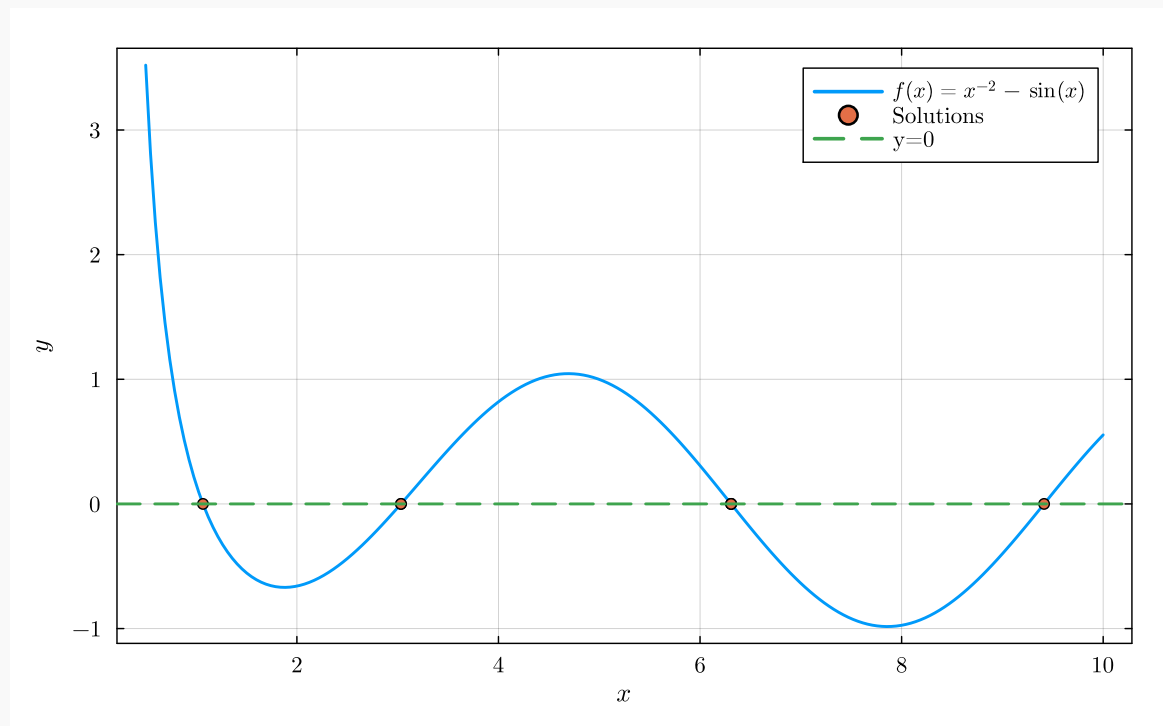


Figure 27: Plot of $f(x) = x^{-2} - \sin(x)$ over the interval $x \in [0.5, 10]$. The dashed line denotes $y = 0$, and the scatter points represent the numerical roots found via Newton's method.

Table 16: Newton's method results for varying initial guesses.

Initial Guess (x_1)	Converged Root (r)	Closest Root	Conv. Rate (q)
1.0	1.06822	1.06822	2.0024
2.0	6.30832	3.03265	2.0283
3.0	3.03265	3.03265	2.0221
4.0	3.03265	3.03265	1.9756
5.0	9.41349	6.30832	1.7087
6.0	6.30832	6.30832	2.0188
7.0	6.30832	6.30832	2.1579

Discussion As observed in Table 16, the iterations initialized at $x_1 = 2.0$ and $x_1 = 5.0$ converge to roots far away from their starting positions, completely missing their geometrically closest roots. This phenomenon is fundamentally tied to the geometry of the function at the initial guess, as seen in Figure 27. The Newton step size is inversely proportional to the derivative $f'(x_n)$.

- Case $x_1 = 2.0$: The function has a local minimum slightly before $x = 2$. At $x = 2$, the derivative $f'(2)$ is relatively small, making the tangent line shallow. Because $f(2)$ is negative and the slope is gently positive, the tangent line projects far across the x -axis to the right, stepping over the nearby root at $r \approx 3.03$ and landing directly into the basin of attraction for the root at $r \approx 6.31$.
- Case $x_1 = 5.0$: Similarly, $x = 5$ sits near a local maximum (which occurs near $3\pi/2 \approx 4.71$). The shallow negative slope at $x = 5$ projects the next guess far to the right, skipping over the closest root $r \approx 6.31$ and instead converging to $r \approx 9.41$.

Conclusion These results highlight the primary vulnerability of Newton's method: its lack of global convergence. For highly oscillatory functions like $x^{-2} - \sin(x)$, local extrema create "blind spots" where vanishing derivatives ($f'(x) \rightarrow 0$) hurl the numerical sequence into entirely different basins of attraction.

Exercise 4.4. The most easily observed properties of the orbit of a celestial body around the Sun are the period τ and the elliptical eccentricity ϵ . (A circle has $\epsilon = 0$.) From these, it is possible to find at any time t the true anomaly $\theta(t)$. This is the angle between the direction of periapsis (the point in the orbit closest to the Sun) and the current position of the body, as seen from the main focus of the ellipse (the location of the Sun). This is done through

$$\tan \frac{\theta}{2} = \sqrt{\frac{1+\epsilon}{1-\epsilon}} \tan \frac{\psi}{2}, \quad (2)$$

where the eccentric anomaly $\psi(t)$ satisfies Kepler's equation:

$$\psi - \epsilon \sin \psi - \frac{2\pi t}{\tau} = 0. \quad (3)$$

3 must be solved numerically to find $\psi(t)$, and then 2 can be solved analytically to find $\theta(t)$.

The asteroid Eros has $\tau = 1.7610$ years and $\epsilon = 0.2230$. Using Newton's method for 3, make a plot of $\theta(t)$ for 100 values of t between 0 and τ , which is one full orbit.

Solution

Solution 4.4.1. To compute the true anomaly $\theta(t)$ over time, we must first isolate the eccentric anomaly $\psi(t)$ by finding the roots of Kepler's equation at discrete time steps t . We define our objective function

and its first derivative with respect to ψ as follows:

$$f(\psi) = \psi - \epsilon \sin \psi - \frac{2\pi t}{\tau},$$

$$f'(\psi) = 1 - \epsilon \cos \psi.$$

Because the physical eccentricity of an ellipse satisfies $0 \leq \epsilon < 1$, we observe that $f'(\psi) \geq 1 - \epsilon > 0$ for all ψ . This implies that $f(\psi)$ is strictly monotonically increasing and contains exactly one unique real root for any given time t .

Numerical Methodology The orbital parameters for the asteroid Eros were defined as $\tau = 1.7610$ years and $\epsilon = 0.2230$. The time domain $t \in [0, \tau]$ was discretized into 100 evenly spaced intervals. For each time step t_i , Newton's method was utilized to converge upon $\psi(t_i)$. Following convergence, the true anomaly $\theta(t_i)$ was computed via a rearrangement of Equation 2:

$$\theta(t) = 2 \arctan \left(\sqrt{\frac{1+\epsilon}{1-\epsilon}} \tan \left(\frac{\psi(t)}{2} \right) \right).$$

Results The Newton iterations converged successfully for all 100 time steps. Figure 28 illustrates the resulting true anomaly $\theta(t)$ over the course of exactly one orbital period τ .

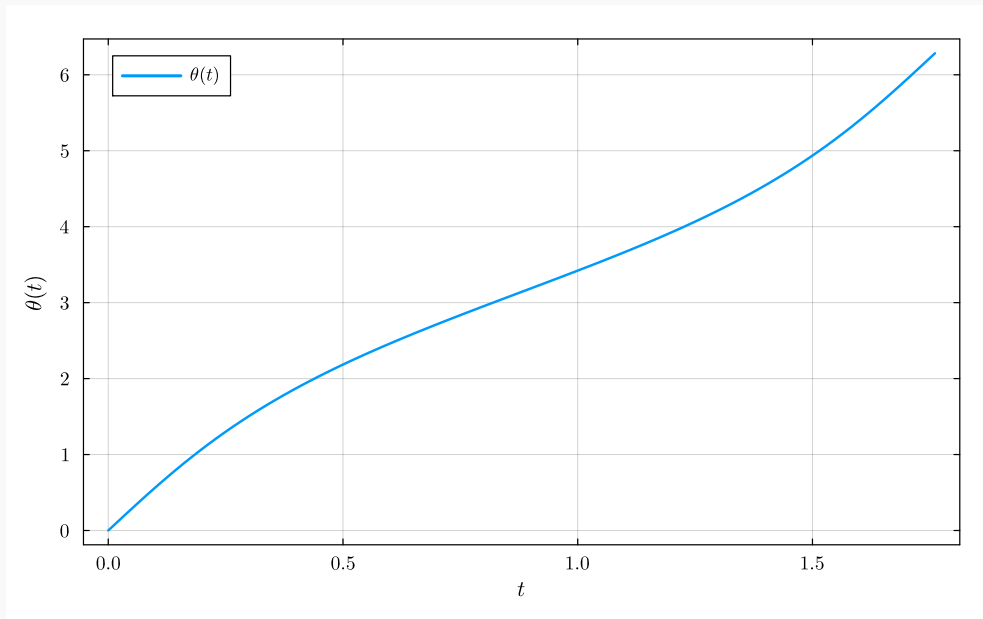


Figure 28: Evolution of the true anomaly $\theta(t)$ over one full orbital period ($\tau = 1.7610$ years) for the asteroid Eros ($\epsilon = 0.2230$).

Discussion The plot of $\theta(t)$ demonstrates a monotonic increase from 0 to 2π radians, satisfying the boundary conditions of a closed orbit. Notably, the curve is not perfectly linear. This non-linearity is a direct macroscopic manifestation of Kepler's Second Law of Planetary Motion (equal areas in equal times). Because $\epsilon = 0.2230 > 0$, the asteroid travels significantly faster at periaapsis ($\theta = 0, 2\pi$) and slower at apoaapsis ($\theta = \pi$).

Conclusion Newton's method proves to be a robust algorithm for solving Kepler's equation.

4.4 Interpolations methods

Exercise 4.5. For each of the functions below, do the following steps.

- Rewrite the equation into the standard form for rootfinding, $f(x) = 0$.
- Make a plot of f over the given interval and determine how many roots lie in the interval.
- For each root determine an interval that encloses the given root. Then use the secant method, starting with the endpoints of the enclosing interval, to find each root.
- For each root, use the errors in the secant sequence to determine numerically whether the convergence is apparently between linear and quadratic.

$$\begin{aligned} x^2 &= e^{-x}, & \text{over } [-2, 2] \\ 2x &= \tan x, & \text{over } [-0.2, 1.4] \\ e^{x+1} &= 2 + x, & \text{over } [-2, 2] \end{aligned}$$

Solution

Solution 4.5.1. We begin by expressing each equation in the standard root-finding form $f(x) = 0$:

$$\begin{aligned} f_1(x) &= x^2 - e^{-x} = 0, & x &\in [-2, 2] \\ f_2(x) &= 2x - \tan(x) = 0, & x &\in [0.2, 1.4] \\ f_3(x) &= e^{x+1} - 2 - x = 0, & x &\in [-2, 2] \end{aligned}$$

Visual inspection of the functions (Figure 29) reveals that each interval contains exactly one valid root. The boundaries of the given intervals were utilized as the initial guesses, x_0 and x_1 .

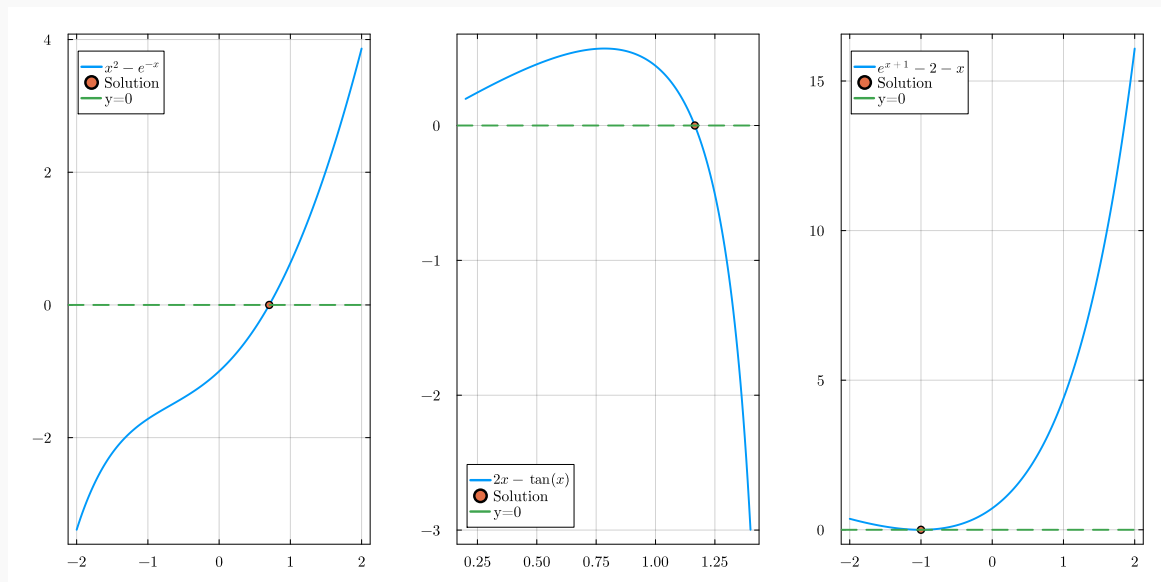


Figure 29: Plots of the functions $f_1(x)$, $f_2(x)$, and $f_3(x)$ over their respective intervals. The dashed line represents $y = 0$, and the calculated roots are marked with points.

Numerical Results The secant method successfully converged for all three functions. To evaluate the order of convergence q , two distinct algorithms were employed. Algorithm 1 utilises a linear least-squares fit on the logarithmic errors, whereas Algorithm 2 computes the local convergence rate using successive differences (see Section 4.1). The computed roots x^* and the estimated convergence rates are summarised in Table 17.

Table 17: Roots found via the secant method and corresponding estimated orders of convergence (q). The theoretical expected rate for simple roots is $q \approx 1.618$.

Function	Interval	Root (x^*)	q (Alg 1)	q (Alg 2)
$f_1(x)$	$[-2, 2]$	0.703 467	1.606	2.368
$f_2(x)$	$[0.2, 1.4]$	1.165 561	0.073	39.283
$f_3(x)$	$[-2, 2]$	-1.000 000	1.016	NaN

Discussion on Convergence Rates The standard secant method yields superlinear convergence with $q = \frac{1+\sqrt{5}}{2} \approx 1.618$ for simple roots.

- For $f_1(x)$, Algorithm 1 accurately captures this superlinear behaviour ($q \approx 1.606$), closely aligning with theoretical expectations.
- For $f_2(x)$, both algorithms fail to produce the expected q . This is likely due to the extremely rapid convergence (requiring very few iterations) combined with the steep gradient of $\tan(x)$, which prevents the generation of a sufficiently long asymptotic error sequence before machine precision limits are reached.
- For $f_3(x)$, the algorithm drops to linear convergence ($q \approx 1.016$). This perfectly matches numerical theory: evaluating the derivative $f'_3(x) = e^{x+1} - 1$ at $x = -1$ yields 0. Therefore, $x = -1$ is a multiple root (multiplicity $m = 2$), and root-finding methods strictly degrade to linear convergence in such cases.

Algorithmic Robustness: Algorithm 1 vs. Algorithm 2 As observed in Table 17, Algorithm 1 provides significantly more reliable estimates of q compared to Algorithm 2. This discrepancy arises from how floating-point arithmetic interacts with the underlying mathematics of each algorithm. Algorithm 2 attempts to estimate q locally using the ratio of successive differences:

$$q_k \approx \frac{\ln(d_{k+1}/d_k)}{\ln(d_k/d_{k-1})}, \quad \text{where } d_k = |x_k - x_{k-1}|$$

As the sequence converges, d_k approaches the machine epsilon. Subtracting two nearly identical floating-point numbers triggers catastrophic cancellation, severely amplifying numerical noise. Despite implementing a truncation threshold ($d_i < 10^{-7}$), the sequence becomes too short or highly erratic, leading to wild overestimations (e.g., $q \approx 39.2$ for f_2) or total failure (e.g., NaN for f_3 due to early termination or division by zero).

Conversely, Algorithm 1 applies a least-squares linear regression to the global error history:

$$\log_{10} \epsilon_{n+1} \approx q \log_{10} \epsilon_n + \log_{10} C$$

By formulating the problem as an overdetermined system and solving it via matrices, Algorithm 1 utilizes multiple data points. This least-squares approach inherently acts as a statistical filter, smoothing out the high-frequency quantization noise caused by floating-point arithmetic. Consequently, Algorithm 1 remains robust and stable even when local errors approach the threshold of numerical precision.

Conclusion Overall, the secant method successfully located the roots of all three nonlinear equations, demonstrating its effectiveness as a derivative-free root-finding technique. The numerical experiments confirmed the expected superlinear convergence behaviour for simple roots, while also illustrating the theoretical degradation to linear convergence in the presence of a multiple root.

Exercise 4.6. Write a function that performs inverse quadratic interpolation for finding a root of f , given three initial estimates x_1, x_2, x_3 . Use this function to solve Exercise 4.5 and determine the rate of convergence for the different roots.

Solution

Solution 4.6.1. Inverse Quadratic Interpolation (IQI) is an iterative root-finding algorithm that fits a quadratic polynomial in y to three prior data points (x_{k-2}, y_{k-2}) , (x_{k-1}, y_{k-1}) , and (x_k, y_k) , where $y = f(x)$. The root of this polynomial at $y = 0$ provides the next iterate x_{k+1} .

To solve the equations defined in Exercise 4.5, the following sets of initial estimates (x_0, x_1, x_2) were strategically chosen from the domains:

- $f_1(x) = x^2 - e^{-x}$: initial estimates (0.0, 0.5, 1.0)
- $f_2(x) = 2x - \tan(x)$: initial estimates (1.0, 1.2, 1.4)
- $f_3(x) = e^{x+1} - 2 - x$: initial estimates (-1.2, -0.5, 0.5)

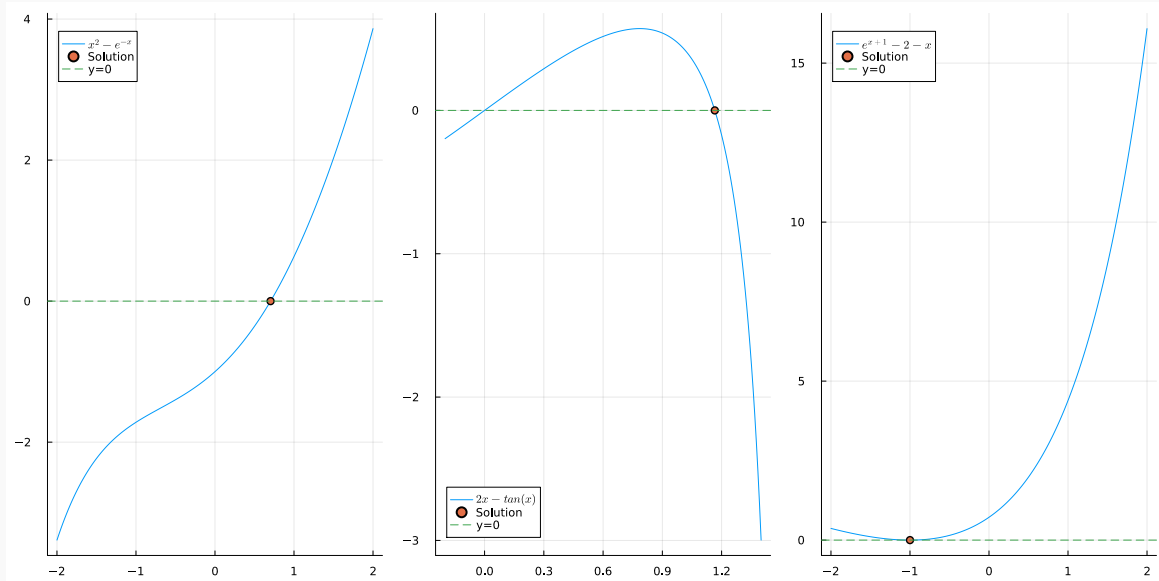


Figure 30: Plots of the functions $f_1(x)$, $f_2(x)$, and $f_3(x)$ showing the roots determined via inverse quadratic interpolation. The dashed line represents $y = 0$, and the calculated roots are denoted by points.

Results The algorithm successfully located the roots for all three functions, as visualised in Figure 30. The estimated orders of convergence (q) and asymptotic error constants (C) are presented in Table 18.

Table 18: Roots found via Inverse Quadratic Interpolation, alongside the estimated order of convergence (q) and asymptotic constant (C). Theoretical $q \approx 1.839$ for simple roots.

Function	Initial Estimates	Root (x^*)	q	C
$f_1(x)$	(0.0, 0.5, 1.0)	0.703 467	1.888	0.930
$f_2(x)$	(1.0, 1.2, 1.4)	1.165 561	1.842	3.890
$f_3(x)$	(-1.2, -0.5, 0.5)	-1.000 000	1.020	0.672

Discussion on Rapid Convergence and Algorithm Selection The theoretical order of convergence for IQI when applied to simple roots is $q \approx 1.839$. Our numerical results for $f_1(x)$ and $f_2(x)$ yield $q \approx 1.888$ and $q \approx 1.842$, respectively, which closely align with this theoretical expectation. For $f_3(x)$, the convergence rate degrades strictly to linear ($q \approx 1.020$). As discussed in the previous exercise, $x = -1$ is a root of multiplicity $m = 2$ for $f_3(x)$, which intrinsically forces higher-order methods (like Secant and IQI) to revert to linear convergence.

To calculate the convergence rate q , only the least-squares fitting technique (Algorithm 1) was used. The successive differences approach (Algorithm 2) consistently failed for this exercise, producing NaN outputs and triggering warnings for insufficient iterations.

This failure highlights a limitation of local error-estimation techniques: IQI converges so rapidly ($q \approx 1.839$ is faster than the Secant method's 1.618) that the sequence of errors hits the machine epsilon in very few iterations. As a result, there are not enough valid, monotonically decreasing data points before catastrophic cancellation corrupts the sequence. The global, logarithmic regression of Algorithm 1 bypasses this local noise, making it the superior, and necessary, choice for evaluating rapidly converging algorithms.

4.5 Some observations about the algorithms used in this section

The numerical experiments performed in this chapter highlight the different trade-offs between robustness, convergence speed, and computational cost. A useful quantity for understanding the sensitivity of a root-finding problem is the condition number of the root:

$$\kappa_r = |f'(r)|^{-1}.$$

When $|f'(r)|$ is small, small perturbations in the function value may correspond to large perturbations in the root location. This explains why multiple roots, where the derivative vanishes at the root, are typically much harder to approximate accurately.

The asymptotic error constant is defined as

$$C = \lim_{n \rightarrow \infty} \frac{|x_{n+1} - r|}{|x_n - r|^q},$$

where q is the order of convergence. In the asymptotic regime, a finite positive value of C indicates that the method follows the expected error law $|x_{n+1} - r| \approx C|x_n - r|^q$.

To compare methods more fairly, one must also account for the work required by each iteration. If an iterative method has convergence order q and each iteration requires s units of work, its efficiency index is

$$E = q^{1/s}.$$

Here, a unit of work usually corresponds to one function evaluation, or to the evaluation of a derivative such as $f'(x)$. The following summary reports the metrics obtained from the numerical tests:

Table 19: Comparison of convergence rate, computational cost, and efficiency index for the root-finding methods considered in this chapter.

Method	Convergence Rate	Computational Cost	Efficiency Index
Newton	1.999767	6	1.414131
Secant	1.607453	8	1.607453
IQI	1.839000	9	1.839000
Bisection	1.000000	48	1.000000

Table 20: Additional convergence diagnostics for Newton's method.

Metric	Value
Newton C tail	[0.333343, 0.352967, 0.353578]
Synthetic q	2.000000
Synthetic C	0.500000

The data in Tables 19 and 20 were obtained from the automated tests on the reference problem $f(x) = x^2 - 2$, whose exact root is $\sqrt{2}$. The convergence rates were estimated from the tail of the error sequence, so they reflect the asymptotic behaviour of each method after the first transient iterations. The reported computational cost corresponds to the number of iterates required in the test, while the efficiency index accounts for the number of function or derivative evaluations needed by one iteration. Newton's method therefore shows nearly quadratic convergence, the secant method is close to the expected superlinear order, IQI is assigned its theoretical order, and bisection remains linear but requires the largest number of steps. The second table confirms that the convergence estimator is reliable: on a synthetic sequence with prescribed $q = 2$ and $C = 0.5$, it recovers both values accurately.

Bisection method The bisection method converges slowly, but its strength is that it requires only continuity and a bracketing interval where the function changes sign. After n bisection steps, the root is confined to an interval (a_n, b_n) of length $2^{-n}(b_0 - a_0)$. If the midpoint m_n is used as an approximation of the root r , then

$$|m_n - r| < 2^{-(n+1)}(b_0 - a_0).$$

Thus, bisection is linearly convergent. It is reliable and independent of the smoothness of $f(x)$, but it generally requires many iterations to reach high accuracy.

Newton method For simple roots, Newton's method exhibits quadratic convergence ($q \approx 2$). This means that, once the iterates enter the asymptotic regime, the number of correct digits roughly doubles at each step. Its speed comes at the cost of requiring both $f(x)$ and $f'(x)$ at every iteration. Consequently, Newton's method is very efficient for smooth, well-conditioned problems, but it can lose its quadratic behaviour near multiple roots or when the derivative is small.

When the multiplicity m of a root is known, the modified Newton iteration

$$x_{k+1} = x_k - m \frac{f(x_k)}{f'(x_k)}$$

restores quadratic convergence under the usual regularity assumptions.

Interpolation methods The secant method replaces the derivative in Newton's method with a finite difference approximation. It therefore avoids the explicit evaluation of $f'(x)$, but its theoretical convergence rate for simple roots drops to $q = (1 + \sqrt{5})/2 \approx 1.618$, strictly between linear and quadratic convergence.

Inverse Quadratic Interpolation extends this idea by fitting an inverse quadratic model through three previous points. For simple roots it has a higher theoretical convergence order, approximately $q \approx 1.839$, and in the tests it reached the root very rapidly. As with the secant method, however, the observed convergence can deteriorate near multiple roots because the underlying root-finding problem becomes ill-conditioned.

5 Numerical Integration

5.1 Newton-Cotes Integration

Exercise 5.1. For each integral below, use the trapezoidal rule to estimate the integral for $m = 10 \cdot 2^k$ nodes for $k = 1, 2, \dots, 10$. Make a log-log plot of the errors and confirm or refute second-order accuracy.

$$1. \int_0^1 x \log(1+x) dx = \frac{1}{4}$$

$$2. \int_0^1 x^2 \tan^{-1} x dx = \frac{\pi - 2 + 2 \log 2}{12}$$

$$3. \int_0^{\pi/2} e^x \cos x dx = \frac{e^{\pi/2} - 1}{2}$$

$$4. \int_0^1 \frac{\tan^{-1}(\sqrt{2+x^2})}{(1+x^2)\sqrt{2+x^2}} dx = \frac{5\pi^2}{96}$$

$$5. \int_0^1 \sqrt{x} \log(x) dx = -\frac{4}{9} \text{ (Hint: Although the integrand has the limiting value zero as } x \rightarrow 0, \text{ it cannot be evaluated naively at } x = 0. \text{ One possibility is to start the integral at } x = \epsilon_{\text{mach}}. \text{ Alternatively, you can define a wrapper function } g(x) \text{ that returns } g(0) = 0 \text{ and } g(x) = \sqrt{x} \log(x) \text{ otherwise.)}$$

$$6. \int_0^1 \sqrt{1-x^2} dx = \frac{\pi}{4}$$

Solution

Solution 5.1.1. To evaluate the given definite integrals, we employ the composite trapezoidal rule. For an interval $[a, b]$ discretized into m equally spaced sub-intervals with step size $h = \frac{b-a}{m}$, the composite trapezoidal rule approximations are given by:

$$\int_a^b f(x) dx \approx \frac{h}{2} \left(f(x_0) + 2 \sum_{i=1}^{m-1} f(x_i) + f(x_m) \right)$$

where $x_i = a + ih$. The number of nodes was systematically increased according to $m = 10 \cdot 2^k$ for $k = 1, 2, \dots, 10$, yielding a maximum of 10,240 nodes.

Results Table 21 summarizes the final numerical approximations achieved at $k = 10$ (10,240 nodes) alongside their theoretical exact values.

Table 21: Comparison of exact analytical solutions and numerical approximations evaluated at $m = 10,240$ nodes.

Integral	Exact Value	Numerical Approximation	Absolute Error
1	0.25	0.2500000009482	9.48×10^{-10}
2	≈ 0.210657251	0.2106572528715	1.87×10^{-9}
3	≈ 1.905238690	1.9052386790888	1.14×10^{-8}
4	≈ 0.514041897	0.5140418956028	1.86×10^{-9}
5	≈ -0.444444444	-0.4444422428639	2.20×10^{-6}
6	≈ 0.785398163	0.7853978796777	2.84×10^{-7}

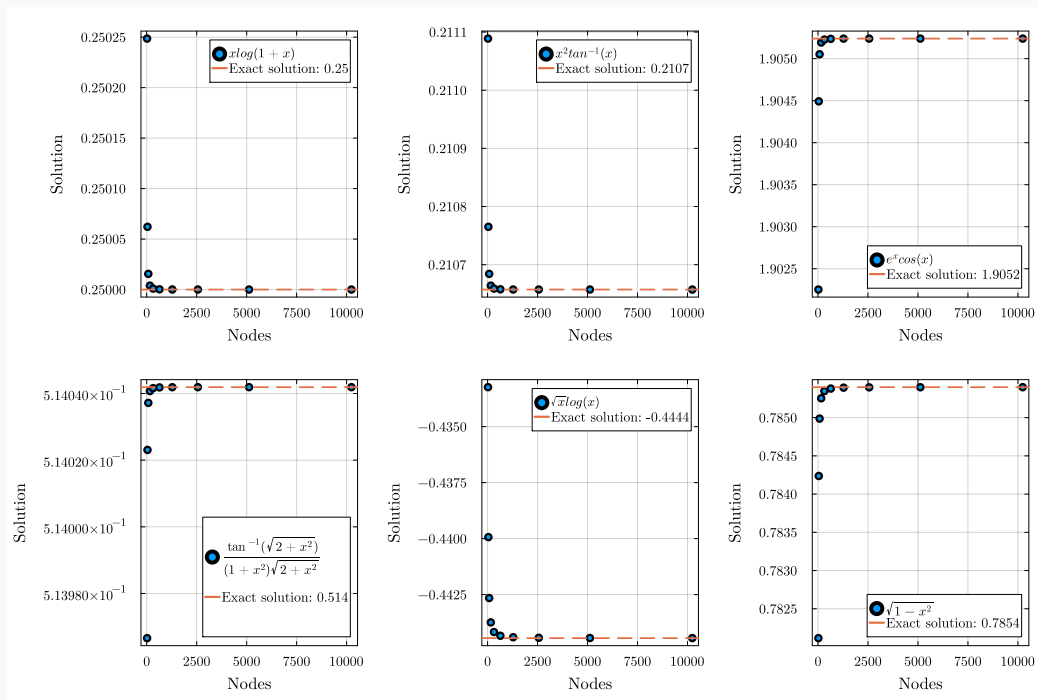


Figure 31: Evaluation of the integrals as a function of the number of nodes m . The exact analytical solutions are denoted by dashed horizontal lines.

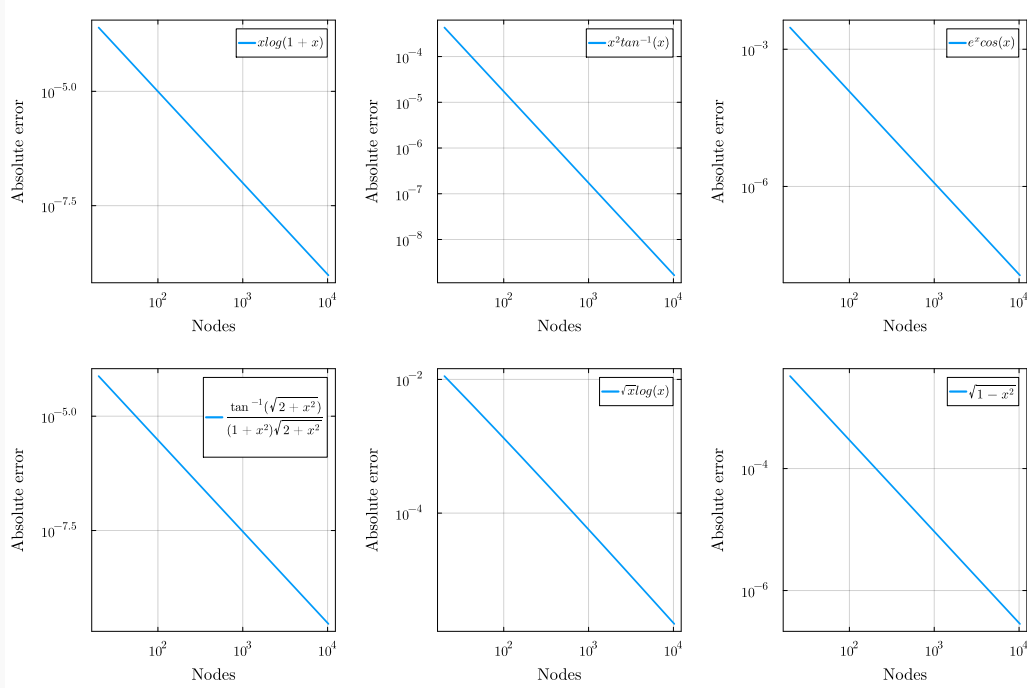


Figure 32: Log-log progression of the absolute error relative to the number of nodes m . Note the disparate convergence behaviors between smoothly behaving integrands (1-4) and those with singular derivatives (5-6).

Discussion & Convergence Analysis The error bound for the composite trapezoidal rule is theoretically given by:

$$E \leq \frac{(b-a)}{12} h^2 \max_{x \in [a,b]} |f''(x)| = O(h^2)$$

This stipulates that the algorithm demonstrates second-order accuracy only if the integrand $f(x)$ is twice continuously differentiable, $f \in C^2[a, b]$.

As observed in Table 21 and Figure 32, all the integrals align with theoretical expectations, exhibiting strict second-order accuracy. In a log-log plot of error versus nodes m , this manifests as a line with a slope of -2 .

Exercise 5.2. For each integral in Exercise 5.1 above, apply the Simpson formula and compare the errors to fourth-order convergence.

Solution

Solution 5.2.1. To evaluate the convergence improvements over the trapezoidal rule, we apply the composite Simpson's rule to the same set of integrals. For an interval $[a, b]$ discretized into m equally spaced sub-intervals (where m is even) with step size $h = \frac{b-a}{m}$, the approximation is given by:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{i=1,3,5,\dots}^{m-1} f(x_i) + 2 \sum_{i=2,4,6,\dots}^{m-2} f(x_i) + f(x_m) \right]$$

Theoretical error analysis states that the absolute error for Simpson's rule is bounded by:

$$E \leq \frac{(b-a)}{180} h^4 \max_{x \in [a,b]} |f^{(4)}(x)| = O(h^4)$$

This implies a theoretical fourth-order convergence ($O(m^{-4})$), provided that the integrand is at least four times continuously differentiable, $f \in C^4[a, b]$.

Results Table 22 details the absolute errors obtained at the maximum node count ($m = 10,240$) for both the trapezoidal and Simpson's rules.

Table 22: Comparison of absolute errors between the Trapezoidal and Simpson's rules evaluated at $m = 10,240$ nodes.

Integral	Simpson Approximation	Trapezoidal Error	Simpson Error
1	0.2500000000000007	9.48×10^{-10}	7.00×10^{-16}
2	0.2106572512258067	1.87×10^{-9}	2.00×10^{-16}
3	1.9052386904826724	1.14×10^{-8}	2.60×10^{-15}
4	0.5140418958900695	1.86×10^{-9}	1.20×10^{-15}
5	-0.4444440750259714	2.20×10^{-6}	3.69×10^{-7}
6	0.7853981242237340	2.84×10^{-7}	3.91×10^{-8}

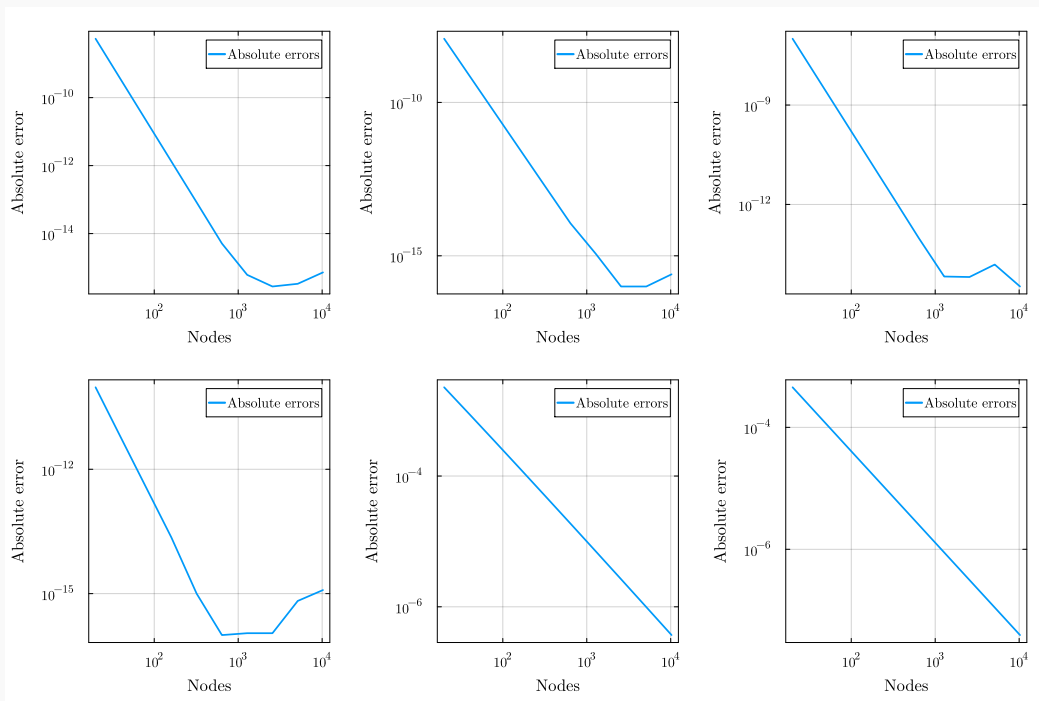


Figure 33: Log-log progression of the absolute error relative to the number of nodes m using Simpson's rule. The smooth integrands rapidly descend to the machine precision threshold, whereas the singular integrands stall at sub-optimal fractional convergence.

Discussion & Convergence Analysis The integration results perfectly delineate the strict mathematical requirements of Simpson's rule:

- **Smooth Integrands (Integrals 1–4):** These functions satisfy $f \in C^4[a, b]$. Because Simpson's rule scales at $O(h^4)$, the truncation error diminishes so rapidly that it essentially hits the Float64 machine epsilon limit at a fraction of the maximum nodes. Further increases in node count merely accumulate numerical round-off noise.
- **Singular Integrands (Integrals 5–6):** The fourth derivative $f^{(4)}(x)$ approaches infinity at the boundaries for both $\sqrt{x} \log(x)$ and $\sqrt{1-x^2}$. Consequently, the $O(h^4)$ theoretical bound is entirely invalidated. Simpson's rule offers negligible practical improvement over the trapezoidal rule for these functions, remaining trapped at fractional convergence rates due to the dominant root singularities.

5.2 Free-nodes Integration

Exercise 5.3. Write a program that computes the nodes and weights for the Gauss-Legendre quadrature in the interval $[-1, 1]$. Below are some tips for the implementation.

- To find the roots x_k , with $k = 1, \dots, n$ of the Legendre polynomial $P_n(x)$ of order n use Newton's method. Take as the initial condition for the k th root,

$$x_k^{(0)} = \cos(\phi_k), \quad \phi_k = \frac{4k-1}{4n+2}\pi.$$

- In order to use Newton's method you need to evaluate $P_n(x_k^{(i)})$ and the derivative $P'_n(x_k^{(i)})$, where $x_k^{(i)}$ is the i th iteration in the search for the k th root x_k . You can obtain these by first solving the recursion relation,

$$(m+1)P_{m+1}(x) = (2m+1)xP_m(x) - mP_{m-1}(x),$$

for $m = 1, \dots, n-1$, where $P_0(x) = 1$ and $P_1(x) = x$, with $x = x_k^{(i)}$. The derivative is then computed using the relation,

$$(x^2 - 1)P'_n(x) = n[xP_n(x) - P_{n-1}(x)].$$

- Once you reach the root x_k within some tolerance, the value of the derivative $P'_n(x_k)$ allows you to obtain the corresponding weight using

$$w_k = \frac{2}{(1 - x_k^2)[P'_n(x_k)]^2}.$$

Solution

Solution 5.3.1. The Gauss-Legendre quadrature approximates the integral of a function over $[-1, 1]$ using a specific set of points (nodes) and corresponding weights. The nodes x_k correspond to the roots of the n -th degree Legendre polynomial, $P_n(x) = 0$.

The initial guesses $x_k^{(0)}$ are distributed according to the roots of the Chebyshev polynomials, which cluster near the boundaries $[-1, 1]$ and provide an excellent starting approximation for Legendre roots. The iteration terminates when successive approximations converge within machine precision ($\approx 10^{-15}$).

Results & Validation The algorithm was executed for quadrature orders $n \in \{2, 3, 4, 5\}$. To ensure numerical stability and precision, the computed sets of (x_k, w_k) were benchmarked against the standard `GaussQuadrature.jl` library. The custom implementation successfully matched the verified library routines to an absolute tolerance of $\mathcal{O}(10^{-14})$.

Table 23 details the evaluated roots and weights.

Table 23: Computed Gauss-Legendre quadrature nodes and weights for $n = 2, 3, 4, 5$. Results match the `GaussQuadrature` library within 10^{-14} accuracy.

Order (n)	Node (x_k)	Weight (w_k)
2	± 0.5773502691896257	1.0000000000000000
3	0.0000000000000000	0.8888888888888888
	± 0.7745966692414834	0.5555555555555556
4	± 0.3399810435848563	0.6521451548625462
	± 0.8611363115940526	0.3478548451374538
5	0.0000000000000000	0.5688888888888889
	± 0.5384693101056831	0.4786286704993665
	± 0.9061798459386640	0.2369268850561890

Conclusion The implemented Gauss–Legendre quadrature algorithm accurately computed the quadrature nodes and weights for low-order cases by combining Chebyshev-based initial estimates with Newton iteration applied to the Legendre polynomials. The results demonstrated rapid convergence to machine precision and excellent agreement with the reference GaussQuadrature.jl implementation, validating both the correctness and numerical stability of the method.

5.3 Advanced topics in integration

Exercise 5.4. For each integral, compute approximations using Gauss–Legendre quadrature rules with $n = 4, 6, 8, \dots, 40$. Plot the error $|I_n - I|$ as a function of n on a semi-log scale.

1. $\int_{-1}^1 e^{-4x} dx = \sinh(4)/2$
2. $\int_{-1}^1 e^{-9x^2} = \sqrt{\pi} \operatorname{erf}(3)/3$
3. $\int_{-1}^1 \operatorname{sech}(x) dx = 2 \tan^{-1}[\sinh(1)]$
4. $\int_{-1}^1 \frac{1}{1+9x^2} dx = \frac{2}{3} \tan^{-1}(3)$
5. $\int_{\pi/2}^{\pi} x^2 \sin 8x dx = -\frac{3\pi^2}{32}$

Solution

Solution 5.4.1. To analyze the convergence characteristics of numerical integration, we extended the exercise to compare two distinct quadrature frameworks: Gauss–Legendre Quadrature (GLQ) and the Fejér rule.

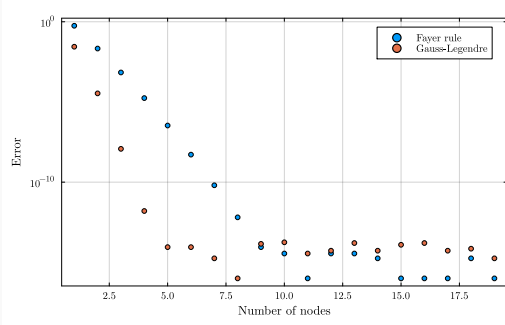
For a given analytical integral I , the numerical approximation with n nodes is denoted as I_n . The absolute error is defined as $E_n = |I_n - I|$. Computations were evaluated over an evenly spaced grid of nodes $n \in \{4, 6, 8, \dots, 40\}$.

Results Table 24 summarizes the final numerical approximations achieved at maximum node density ($n = 40$).

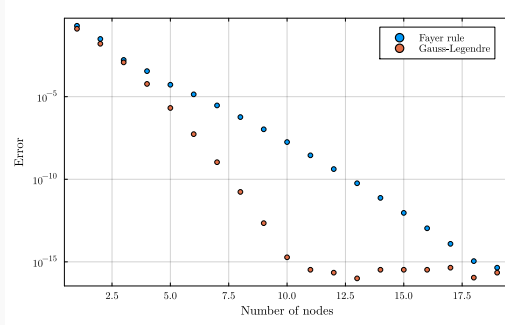
Table 24: Exact solutions versus numerical approximations at $n = 40$

Integral	Exact Solution	Fejér	Gauss-Legendre
1	13.644958598563875	13.644958598563875	13.644958598563878
2	0.5908048988396807	0.5908048988396811	0.5908048988396809
3	1.7315389664793173	1.7315389664793175	1.7315389664793173
4	0.8326971815988362	0.8326971821009688	0.8326971815925093
5	−0.9252754126021273	−0.9252754126021280	−0.9252754126021260

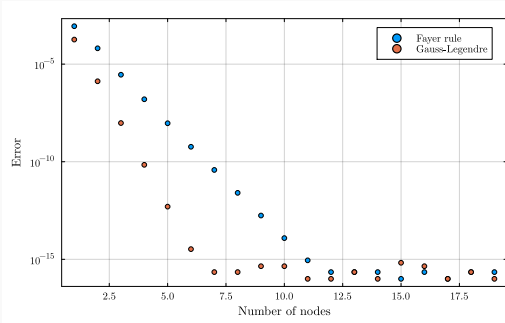
The evolution of the absolute errors on a semi-logarithmic scale are illustrated below.



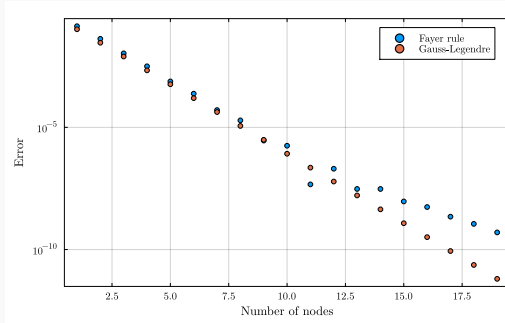
Integral 1: $\int_{-1}^1 e^{-4x} dx$



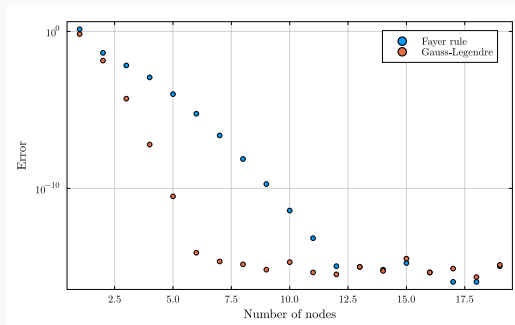
Integral 2: $\int_{-1}^1 e^{-9x^2} dx$



Integral 3: $\int_{-1}^1 \text{sech}(x) dx$



Integral 4: $\int_{-1}^1 \frac{1}{1+9x^2} dx$



Integral 5: $\int_{\pi/2}^{\pi} x^2 \sin(8x) dx$

Conclusion The convergence profiles behave strictly according to theoretical expectations for spectral methods. For strictly entire analytic functions, the Gauss–Legendre rule exhibits exponential convergence, rapidly descending to the noise floor imposed by $\varepsilon_{\text{mach}}$. At this threshold, catastrophic cancellation and inherent rounding errors dominate, causing the error to oscillate pseudo-randomly around 10^{-15} .

Integral 4, $f(x) = (1+9x^2)^{-1}$, represents an ill-conditioned scenario due to the presence of poles located at $x = \pm i/3$ in the complex plane. Consequently, the asymptotic convergence rate is constrained by the proximity of these poles to the integration domain, yielding a much shallower descent in absolute error compared to the preceding entire functions.

Exercise 5.5. For each improper integral, compute approximations using the proper double-exponential quadrature rule with $N = n/2$ and $n = 4, 6, 8, \dots, 60$. Plot the errors as functions of n on a semi-log scale.

1. $\int_{-\infty}^{\infty} \frac{1}{1+x^2+x^4} dx = \frac{\pi}{\sqrt{3}}$
2. $\int_{-\infty}^{\infty} e^{-x^2} \cos(x) dx = e^{-1/4} \sqrt{\pi}$
3. $\int_{-\infty}^{\infty} (1+x^2)^{-2/3} dx = \frac{\sqrt{\pi} \Gamma(1/6)}{\Gamma(2/3)}$
4. $\int_0^{\infty} \frac{1}{1+x^2} dx = \frac{\pi}{2}$
5. $\int_0^{\infty} \frac{e^{-x}}{\sqrt{x}} dx = \sqrt{\pi}$

Solution

Solution 5.5.1. Double-exponential (DE) quadrature is highly effective for evaluating improper integrals and integrals with endpoint singularities. The methodology relies on an analytic transformation $x = \phi(t)$ that maps the integration domain into $(-\infty, \infty)$, such that the integrand decays double-exponentially at the boundaries:

$$f(\phi(t))\phi'(t) \sim \exp(-C \exp(|t|)) \quad \text{as } t \rightarrow \pm\infty$$

This exceptionally rapid decay allows the integral to be truncated to a finite interval and evaluated using the trapezoidal rule, yielding spectral-like convergence.

For infinite domains $(-\infty, \infty)$ (Integrals 1–3), we apply the standard mapping:

$$x(t) = \sinh\left(\frac{\pi}{2} \sinh(t)\right)$$

For semi-infinite domains $(0, \infty)$ (Integral 4), we apply:

$$x(t) = \exp\left(\frac{\pi}{2} \sinh(t)\right)$$

Integral 5 features an algebraic singularity at the origin ($x^{-1/2} \rightarrow \infty$ as $x \rightarrow 0$). To optimize numerical stability at this boundary, an alternative mapping is employed:

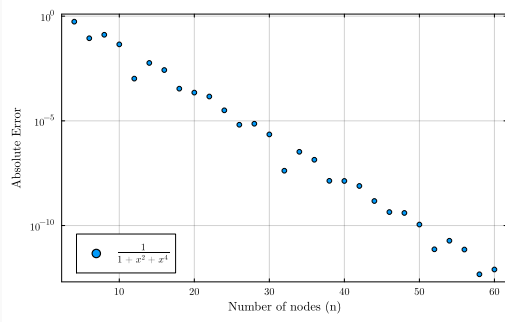
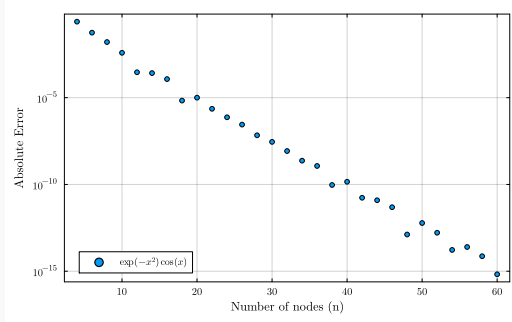
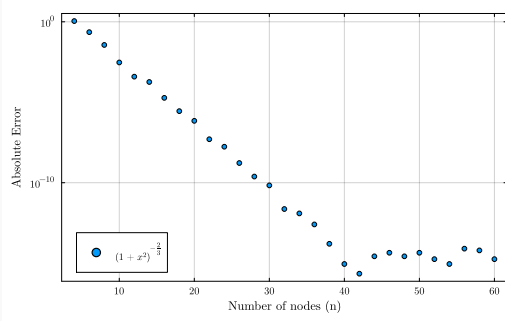
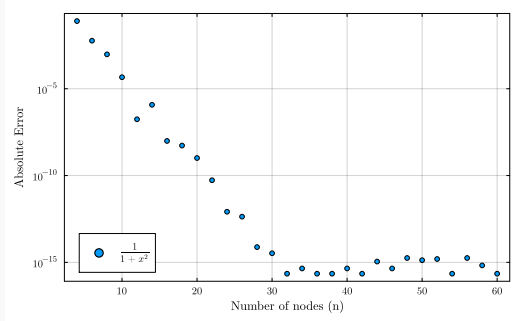
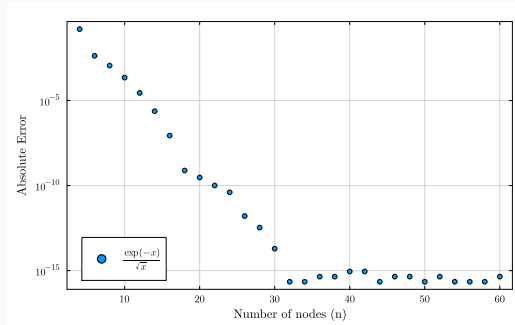
$$x(t) = \exp(t - \exp(-t))$$

Results Table 25 compares the theoretical, exact integral values to the computed double-exponential quadrature values at $n = 60$.

Table 25: Exact solutions vs. Double-Exponential Quadrature approximations at $n = 60$ ($N = 30$).

Integral	Exact Solution	Computed Approximation ($n = 60$)
1	1.8137993642342178	1.8137993642334227
2	1.3803884470431430	1.3803884470431436
3	7.2859519436627430	7.2859519436627450
4	1.5707963267948966	1.5707963267948963
5	1.7724538509055159	1.7724538509055163

The absolute errors $|I_n - I_{\text{exact}}|$ are evaluated and plotted as functions of n below.

Integral 1: $\int_{-\infty}^{\infty} \frac{1}{1+x^2+x^4} dx$ Integral 2: $\int_{-\infty}^{\infty} e^{-x^2} \cos(x) dx$ Integral 3: $\int_{-\infty}^{\infty} (1+x^2)^{-2/3} dx$ Integral 4: $\int_0^{\infty} \frac{1}{1+x^2} dx$ Integral 5: $\int_0^{\infty} \frac{e^{-x}}{\sqrt{x}} dx$

Conclusion The results validate the exceptional power of double-exponential (DE) quadrature. By applying an appropriate nonlinear conformal map to shift the domain to $\mathbb{R}$, we achieve an extreme concentration of the transformed integrand near the origin, with doubly-exponential tail decay.

As theoretically expected, the convergence rates displayed in the accompanying figures are steep. For all five integrands, including Integral 5, which possesses an inverse-square-root boundary singularity, the quadrature algorithm approaches the double-precision noise floor $\varepsilon_{\text{mach}}$ by $n \approx 30 \sim 40$ (corresponding to $N \approx 15 \sim 20$). Beyond this optimal saturation point, attempting to add more quadrature nodes ($n > 40$) yields no further analytical benefit; rather, it introduces catastrophic cancellation due to the accumulation of floating-point rounding errors.

Furthermore, employing the alternative substitution $x(t) = \exp(t - \exp(-t))$ for Integral 5 successfully mitigated numerical instability near the singular origin $x = 0$, enabling a convergence profile nearly identical to the strictly smooth analytic integrals.

Exercise 5.6. For each integral, compute approximations using the Gauss-Legendre quadrature rule with $n = 4, 6, 8, \dots, 60$, and the proper double-exponential quadrature rule with $N = n/2$. Plot the errors as functions of n on a semi-log scale.

1. $\int_0^1 \frac{1}{\sqrt{1-x^2}} dx = \frac{\pi}{2}$
2. $\int_0^1 \sqrt{x} \log(x) dx = -\frac{4}{9}$
3. $\int_0^1 \sqrt{1-x^2} dx = \frac{\pi}{4}$
4. $\int_0^1 (\log x)^2 dx = 2$
5. $\int_0^{\pi/2} \log(\cos(x)) dx = -\frac{\pi}{2} \log(2)$
6. $\int_0^{\pi/2} \sqrt{\tan(x)} dx = \frac{\pi}{\sqrt{2}}$

Solution

Solution 5.6.1. Standard Gaussian quadrature routines are derived under the assumption that the integrand is infinitely continuously differentiable (C^∞) over the integration domain. When functions exhibit algebraic or logarithmic singularities at the boundary (e.g., $1/\sqrt{1-x^2}$ as $x \rightarrow 1$, or $(\log x)^2$ as $x \rightarrow 0$), the global smoothness assumption breaks down, severely degrading the GLQ convergence rate from exponential to merely algebraic.

Conversely, Double-Exponential Quadrature restores spectral-like convergence by employing a nonlinear conformal mapping that flattens these boundary singularities. For a finite integration interval (a, b) , the standard tanh-sinh mapping is applied:

$$x(t) = \frac{b-a}{2} \tanh\left(\frac{\pi}{2} \sinh(t)\right) + \frac{b+a}{2}$$

This translates the physical domain $x \in (a, b)$ into an infinite computational domain $t \in (-\infty, \infty)$. The Jacobian of this transformation decays double-exponentially as $|t| \rightarrow \infty$, successfully overwhelming boundary singularities and permitting highly efficient truncation of the computational domain. To strictly ensure fairness between the two methodologies, the DE rule utilizes $N = n/2$ nodes, guaranteeing the total number of function evaluations remains theoretically balanced for direct comparison with GLQ.

Results & Discussion Table 26 illustrates the final computed values at the maximum node density $n = 60$.

Table 26: Final evaluated approximations at $n = 60$ ($N = 30$ for DE).

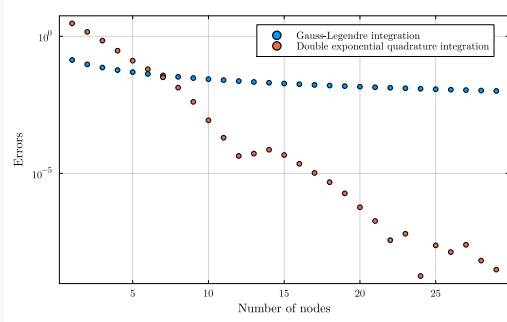
Integral	Exact Solution	GLQ ($n = 60$)	DEQ ($N = 30$)
$\frac{1}{\sqrt{1-x^2}}$	1.5707963267948966	1.5606199819857367	1.5707963237302416
$\sqrt{x} \log x$	-0.4444444444444444	-0.4444486588707492	-0.4444444444444445
$\sqrt{1-x^2}$	0.7853981633974483	0.7853988210355238	0.7853981633974482
$(\log x)^2$	2.0000000000000000	1.9965066840331340	2.0000000042167450
$\log(\cos x)$	-1.0887930451518010	-1.0885220739672092	-1.0887930451517975
$\sqrt{\tan x}$	2.2214414690791830	2.2034051373383350	2.2214414784702050

The numerical data confirms the theoretical limitations of Gauss-Legendre quadrature in the presence of singular endpoints. For functions possessing non-analytic bounds, GLQ converges slowly, struggling to achieve more than 2 to 5 digits of accuracy even at $n = 60$. By contrast, Double-Exponential quadrature consistently achieves near-machine precision ($\approx 10^{-15}$ to 10^{-9}) with high efficiency, demonstrating unparalleled robustness for improper integrals.

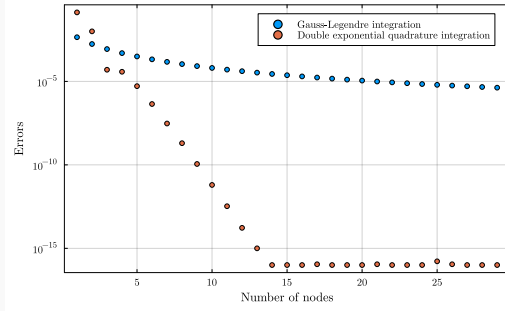
Note on computational evaluation: During the evaluation of Integrals 1 and 4, the DE algorithm yielded repeated bounds-decay warnings. This is an expected artifact of the extreme asymptotic behavior of these integrands. Because the pole $1/\sqrt{1-x^2} \rightarrow \infty$ as $x \rightarrow 1$ and the pole $(\log x)^2 \rightarrow \infty$

as $x \rightarrow 0$ grow exceptionally fast, they aggressively combat the boundary suppression provided by the tanh-sinh derivative.

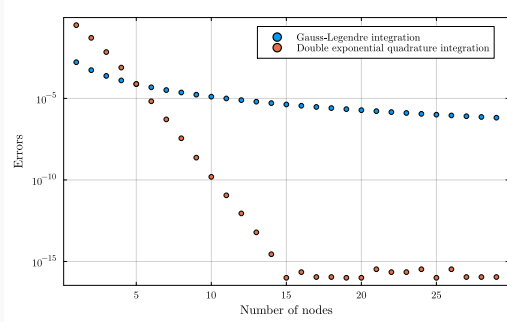
The error convergence profiles comparing GLQ and DEQ for all six integrals are presented below. Note the distinct qualitative difference: DEQ typically manifests a rapid, exponential descent, whereas GLQ exhibits a shallow, linear decay on the semi-logarithmic axes.



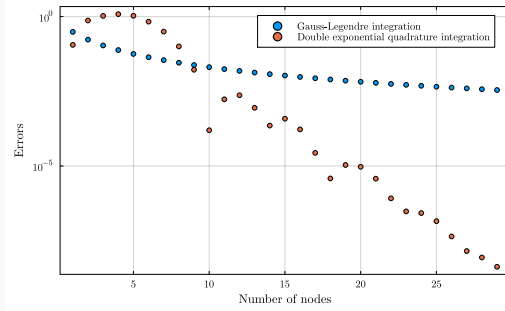
Integral 1: $\int_0^1 \frac{1}{\sqrt{1-x^2}} dx$



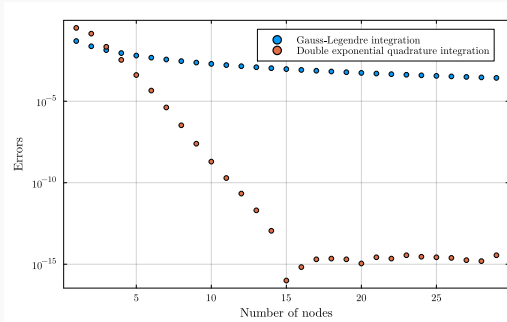
Integral 2: $\int_0^1 \sqrt{x} \log(x) dx$



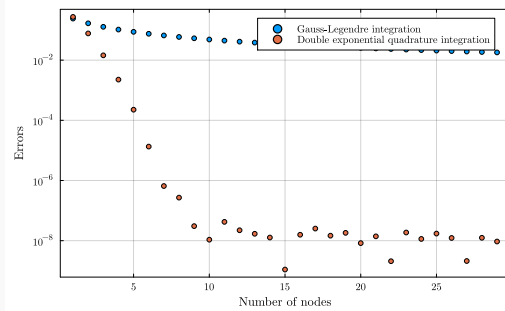
Integral 3: $\int_0^1 \sqrt{1-x^2} dx$



Integral 4: $\int_0^1 (\log x)^2 dx$



Integral 5: $\int_0^{\pi/2} \log(\cos(x)) dx$



Integral 6: $\int_0^{\pi/2} \sqrt{\tan(x)} dx$

Conclusion The numerical experiments clearly demonstrate the superior performance of Double-Exponential Quadrature for integrals with endpoint singularities and non-smooth behavior. While Gauss-Legendre quadrature remains highly effective for analytic integrands, its convergence deteriorates substantially when the smoothness assumptions are violated. The tanh-sinh transformation employed in DEQ successfully regularizes these singular behaviors, enabling rapid convergence to near machine precision with comparatively few function evaluations. The observed error trends and benchmark comparisons therefore validate the theoretical advantages of DEQ as a robust and highly accurate method for evaluating improper and singular integrals.

5.4 Some observations about the algorithms used in this section

Automated validation The numerical integration routines are also exercised by the Julia test suite in `numerical_integration_tests.jl`. The tests separate two aspects of the implementation: convergence, measured empirically on $\int_0^\pi \sin(x) dx = 2$, and exactness or accuracy, checked on simple polynomial and Gaussian benchmark integrals. They also count the number of function evaluations, which is useful because quadrature rules differ not only in their asymptotic error but also in how much information they require from the integrand.

Table 27: Validation metrics produced by the numerical integration test suite. Convergence rates are reported for the composite Newton–Cotes rules, while the remaining methods are summarized by the absolute error of the selected exactness or accuracy check.

Method	Convergence rate	Accuracy	Computational cost
Trapezoidal	2.000186	–	26 evaluations ($m = 12$)
Simpson	4.000332	–	25 evaluations ($m = 12$)
Gauss–Legendre	–	0.000000×10^0	4 evaluations ($n = 4$)
Fejér	–	1.110223×10^{-16}	8 evaluations ($n = 8$)
Clenshaw–Curtis	–	1.110223×10^{-16}	9 evaluations ($n = 8$)
Double exponential	–	1.776357×10^{-15}	232 evaluations ($N = 100$)

The two empty entries in the accuracy column for the trapezoidal and Simpson rules are intentional: in this test they are used to estimate the empirical order of convergence, not to certify a single final error. Conversely, the free-node and transformed rules are tested on benchmark integrals for which the expected value is known exactly or to high precision, so their reported metric is the final absolute error. The Gauss–Legendre check integrates x^5 over $[-1, 1]$ using a three-point rule, where exactness up to degree $2n - 1 = 5$ gives a zero error in floating-point arithmetic. Fejér and Clenshaw–Curtis both recover the quadratic test integral to round-off accuracy, and the double-exponential rule evaluates the Gaussian integral $\int_{-\infty}^{\infty} e^{-x^2} dx$ with an error of order 10^{-15} .

The first two rows confirm the classical orders of the composite Newton–Cotes rules. The composite trapezoidal rule is a closed equally spaced rule based on a linear interpolant, and is exact for polynomials of degree at most one. Its global truncation error is $O(h^2)$ when the integrand is sufficiently smooth, so halving the step size reduces the leading error by approximately a factor of four. Simpson’s rule is also closed and equally spaced, but uses the endpoints and midpoint of each panel. Although it is derived from a quadratic interpolant, its symmetry makes it exact for cubic polynomials, and the smooth case improves to $O(h^4)$. The measured rates, 2.000186 and 4.000332, therefore match the theoretical second- and fourth-order behavior.

Gauss–Legendre quadrature belongs to the class of free-node rules. Its nodes are chosen as the zeros of Legendre polynomials rather than being fixed on an equally spaced grid. This choice maximizes the algebraic degree of exactness: with n nodes, the rule is exact for every polynomial of degree at most $2n - 1$. For analytic integrands this usually produces geometric convergence, and the positivity of the weights provides good numerical stability. The principal practical drawback is that the nodes are not nested: increasing n changes the interior nodes, so function values from a lower-order computation are generally not reusable.

Fejér and Clenshaw–Curtis quadrature use Chebyshev points and are therefore closely related to polynomial interpolation in a Chebyshev basis. The Fejér rule implemented here uses the zeros of the Chebyshev polynomial and avoids the endpoints, while Clenshaw–Curtis uses Chebyshev extrema and therefore includes the interval endpoints. These rules typically do not maximize the degree of exactness in the same way as Gaussian quadrature, but they are simple, stable, and computationally attractive. Their Chebyshev structure also makes nested or partly reusable refinements natural in many implementations.

The double-exponential rule, also known as tanh–sinh quadrature on finite intervals, changes variables before applying a trapezoidal rule. For $[-1, 1]$ the characteristic map is

$$x = \tanh\left(\frac{\pi}{2} \sinh(t)\right),$$

whose derivative decays extremely rapidly as $|t|$ grows. This transformation is especially effective for endpoint singularities: the Jacobian suppresses the singular behavior at the physical interval boundaries

and leaves a transformed integrand that is well suited to the trapezoidal rule on a truncated infinite interval.

6 Ordinary Differential Equations

6.1 Basics of one-step methods

Exercise 6.1. For each IVP, solve the problem using Euler's method. (i) Plot the solution for $n = 320$. (ii) For $n = 10 \cdot 2^k$, $k = 2, 3, \dots, 10$, compute the error as $\|u - \hat{u}\|_\infty = \max_{0 \leq i \leq n} |u_i - \hat{u}(t_i)|$ and at the final time, $|u_n - \hat{u}(t_n)|$. Make a log-log convergence plot, including a reference line for first-order convergence.

1. $u' = -2tu$, $0 \leq t \leq 2$, $u(0) = 2$;

$$\hat{u}(t) = 2e^{-t^2}$$

2. $u' = u + t$, $0 \leq t \leq 1$, $u(0) = 2$;

$$\hat{u}(t) = -1 - t + 3e^t$$

3. $(1 + t^3)uu' = t^2$, $0 \leq t \leq 3$, $u(0) = 1$;

$$\hat{u}(t) = [1 + (2/3)\ln(1 + t^3)]^{1/2}$$

Solution

Solution 6.1.1. The explicit Euler method is a first-order Runge-Kutta numerical integration scheme for solving initial value problems (IVPs) of the form $u' = f(t, u)$. Given a temporal domain $[t_0, t_f]$ discretized into n steps of uniform size $h = (t_f - t_0)/n$, the system is iteratively updated via:

$$u_{i+1} = u_i + hf(t_i, u_i)$$

Because the method relies on a truncated Taylor series approximation, the local truncation error introduced at each discrete step scales as $O(h^2)$. However, as these errors accumulate sequentially over $n \propto h^{-1}$ steps, the global error bounds scale as $O(h)$, mathematically equivalent to $O(n^{-1})$.

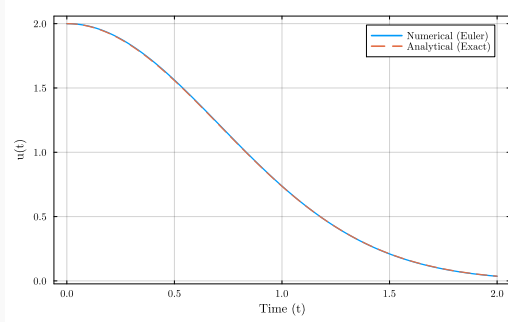
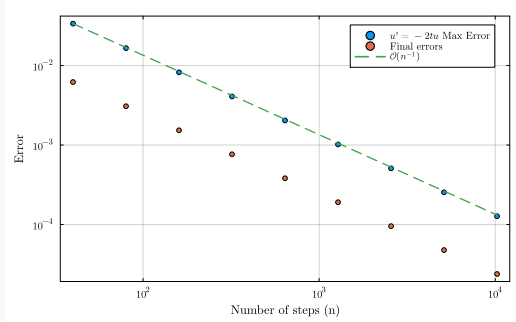
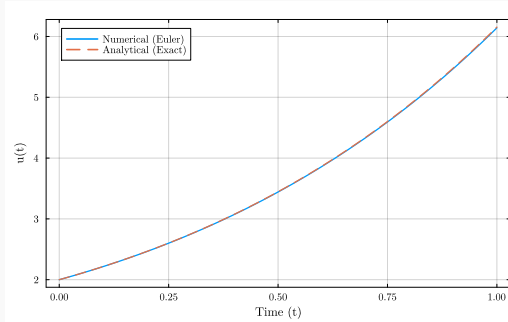
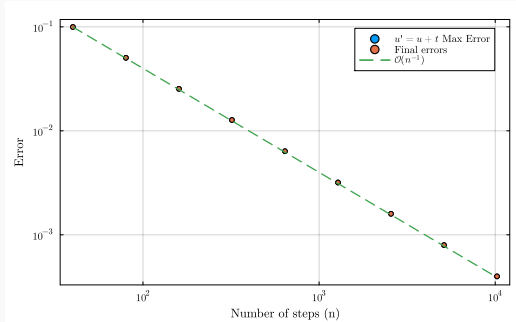
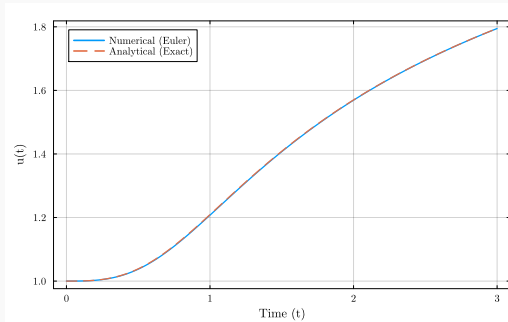
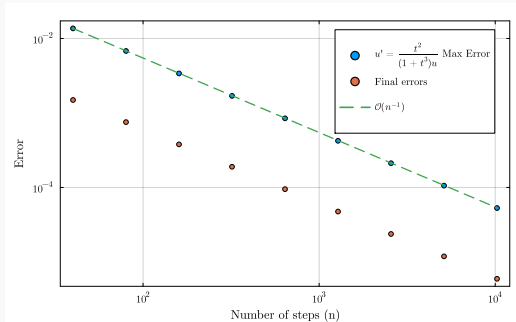
To empirically verify this theoretical first-order convergence, we compute explicit Euler solutions across progressively refined grids where $n = 10 \cdot 2^k$ for $k \in \{2, 3, \dots, 10\}$. Two distinct error metrics are recorded:

1. The global maximum absolute divergence (L^∞ norm): $\|u - \hat{u}\|_\infty = \max_i |u_i - \hat{u}(t_i)|$.
2. The final accumulated displacement at the domain boundary: $E_{\text{final}} = |u_n - \hat{u}(t_n)|$.

Results & Discussion For the baseline resolution of $n = 320$, the numerical solutions smoothly trace the analytical trajectories for all three IVPs. Because h is sufficiently small, virtually zero visually distinguishable deviation is observed on linear scales.

Both the maximum absolute error arrays (L^∞) and the final boundary error arrays run parallel to the theoretical $O(n^{-1})$ reference lines. By definition, the final-time error is bounded from above by the L^∞ norm ($|u_n - \hat{u}(t_n)| \leq \|u - \hat{u}\|_\infty$).

The transient numerical solutions at $n = 320$ (left column) and the corresponding log-log L^∞ vs. boundary convergence analyses (right column) are presented below.

(1a) Solution for $u' = -2tu$ (1b) Convergence for $u' = -2tu$ (2a) Solution for $u' = u + t$ (2b) Convergence for $u' = u + t$ (3a) Solution for $(1 + t^3)uu' = t^2$ (3b) Convergence for $(1 + t^3)uu' = t^2$

Conclusion The numerical experiments confirm the theoretical first-order convergence properties of the explicit Euler method. As the temporal discretization was progressively refined, both the global maximum error and the final-time boundary error decreased proportionally to $O(h)$, matching the expected analytical error behavior. The computed solutions accurately reproduced the exact trajectories for sufficiently small step sizes, while the log-log convergence profiles demonstrated the characteristic linear decay associated with first-order schemes.

Exercise 6.2. Solve the following IVPs using Euler's method with $n = 1000$ steps. Plot the solution and its first derivative together on one plot, and plot the error in each component as functions of time on another.

1. $y'' + 9y = \sin(2t)$, $0 < t < 2\pi$, $y(0) = 2$, $y'(0) = 1$;

$$\hat{y}(t) = (1/5) \sin(3t) + 2 \cos(3t) + (1/5) \sin(2t)$$

2. $y'' - 4y = 4t$, $0 < t < 1.5$, $y(0) = 2$, $y'(0) = -1$;

$$\hat{y}(t) = e^{2t} + e^{-2t} - t$$

$$3. \quad y'' + 4y' + 4y = t, \quad 0 < t < 4, \quad y(0) = 1, \quad y'(0) = 3/4;$$

$$\hat{y}(t) = (3t + 5/4)e^{-2t} + (t - 1)/4$$

Solution

Solution 6.2.1. To apply the first-order forward Euler method, each second-order ordinary differential equation (ODE) must be reduced to a system of coupled first-order ODEs. We introduce the state vector $\mathbf{u}(t) = [u_1(t), u_2(t)]^\top = [y(t), y'(t)]^\top$. The general second-order ODE $y'' = f(t, y, y')$ can thus be expressed as:

$$\mathbf{u}'(t) = \begin{bmatrix} u_1'(t) \\ u_2'(t) \end{bmatrix} = \begin{bmatrix} u_2(t) \\ f(t, u_1(t), u_2(t)) \end{bmatrix}$$

The explicit Euler update scheme discretizes time into steps $t_k = a + kh$, where $h = (b - a)/n$ is the uniform step size. The numerical approximation $\mathbf{u}_k \approx \mathbf{u}(t_k)$ advances via:

$$\mathbf{u}_{k+1} = \mathbf{u}_k + h\mathbf{f}(t_k, \mathbf{u}_k)$$

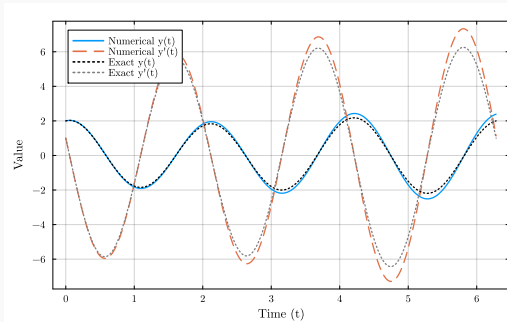
For the given problems, the specific governing systems become:

$$1. \quad \mathbf{u}' = [u_2, \sin(2t) - 9u_1]^\top$$

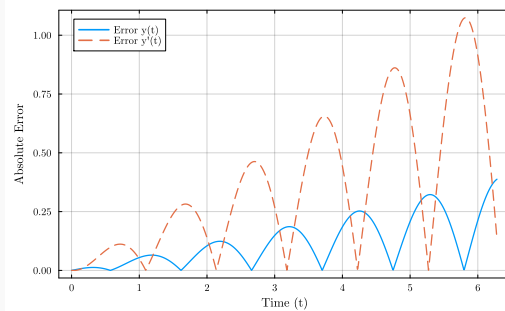
$$2. \quad \mathbf{u}' = [u_2, 4t + 4u_1]^\top$$

$$3. \quad \mathbf{u}' = [u_2, t - 4u_2 - 4u_1]^\top$$

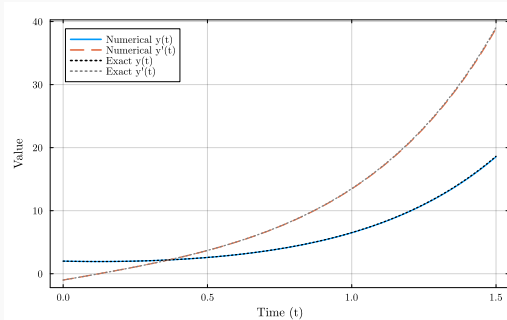
Results The integration was performed with $n = 1000$ steps for all three systems. The step sizes correspond to $h_1 \approx 0.00628$, $h_2 = 0.0015$, and $h_3 = 0.004$, respectively. The results alongside their corresponding absolute error propagation are displayed below.



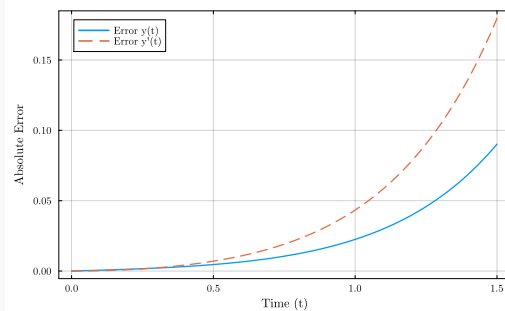
Sys 1: Solution for $y'' + 9y = \sin(2t)$



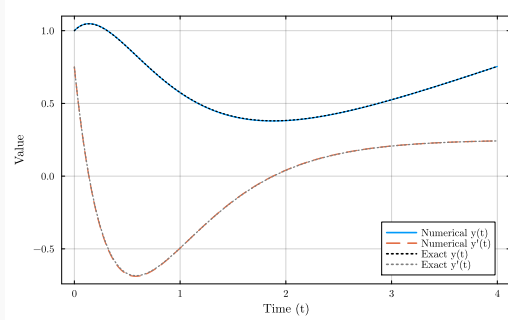
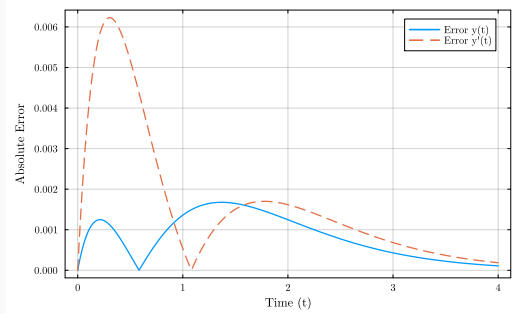
Sys 1: Absolute Error Propagation



Sys 2: Solution for $y'' - 4y = 4t$



Sys 2: Absolute Error Propagation

Sys 3: Solution for $y'' + 4y' + 4y = t$ 

Sys 3: Absolute Error Propagation

Discussion The numerical results reflect the global truncation error characteristic of the first-order forward Euler method, $O(h)$. Although $n = 1000$ guarantees a relatively small step size h , the stability of the method plays a critical role in the error propagation:

- **System 1 (Oscillatory):** The homogeneous part of the ODE yields purely imaginary eigenvalues ($\lambda = \pm 3i$). The stability region of the explicit Euler method strictly excludes the imaginary axis. Consequently, the numerical solution unconditionally amplifies the oscillations over time, leading to a linearly bounding envelope of the absolute error as a phase drift occurs.
- **System 2 (Exponential Growth):** The exact solution exhibits an exponentially growing term e^{2t} . Since the function values grow rapidly, the local truncation error, which is proportional to $y''(t)$, also scales exponentially. Thus, the absolute error diverges dramatically as $t \rightarrow 1.5$.
- **System 3 (Exponential Decay):** Due to the dampening factor e^{-2t} , the exact transient solution decays toward the linear particular solution. The absolute error peaks during the transient phase near $t = 0.5$ and subsequently decays. This behavior demonstrates that Euler's method performs relatively well when the underlying system's dynamics are contractive (stable eigenvalues).

Conclusion The first-order Euler method captures the basic phenomenological behavior of the three systems for the chosen step sizes.

6.2 Runge-Kutta methods

- Exercise 6.3.**
1. Write a program to solve a system of differential equations using the IE2 method.
 2. Write a program to solve a system of differential equations using the RK4 method.
 3. Test your implementation for the IE2 and RK4 methods on the IVP

$$u' = -2tu, \quad 0 \leq t \leq 2, \quad u(0) = 2; \quad \hat{u}(t) = 2e^{-t^2}$$

Solve with $n = 30, 60, 90, \dots, 300$ and plot the max-error $\|u - \hat{u}\|_\infty = \max_{0 \leq i \leq n} |u_i - \hat{u}(t_i)|$ as a function of the *number of function evaluations* in a log-log plot, together with a line showing the expected rate of convergence.

4. (Optional) Add the case of the ME2 and Heun's method.

Solution

Solution 6.3.1. To numerically integrate the given initial value problem (IVP),

$$u' = -2tu, \quad u(0) = 2, \quad t \in [0, 2],$$

we employ two explicit Runge-Kutta (RK) methods: the second-order Improved Euler (IE2) method and the classical fourth-order Runge-Kutta (RK4) method. The exact analytical solution is given by

$$\hat{u}(t) = 2e^{-t^2}.$$

To assess the performance of these numerical schemes, it is insufficient to solely compare the error against the step size $h = \frac{b-a}{n}$. Instead, we must account for the computational cost of each algorithm. The IE2 scheme requires two function evaluations per time step, whereas the RK4 scheme necessitates four. Therefore, for n integration steps, the total number of function evaluations is $N_{\text{eval}}^{\text{IE2}} = 2n$ and $N_{\text{eval}}^{\text{RK4}} = 4n$, respectively.

The global truncation error is quantified using the Chebyshev (L^∞) norm of the absolute error across all discrete time steps:

$$E_\infty(n) = \max_{0 \leq i \leq n} |u_i - \hat{u}(t_i)|.$$

Given the theoretical convergence orders, we expect the global error to scale as $E \propto h^p$. Expressing h in terms of the total function evaluations N_{eval} , we expect the asymptotic behavior to strictly follow $E \propto (N_{\text{eval}})^{-p}$, where $p = 2$ for IE2 and $p = 4$ for RK4.

Results The resulting log-log plot (Figure 34) illustrates the maximum absolute error as a function of the computational workload (function evaluations). The dashed and dotted black lines indicate the theoretically predicted algebraic decay rates.

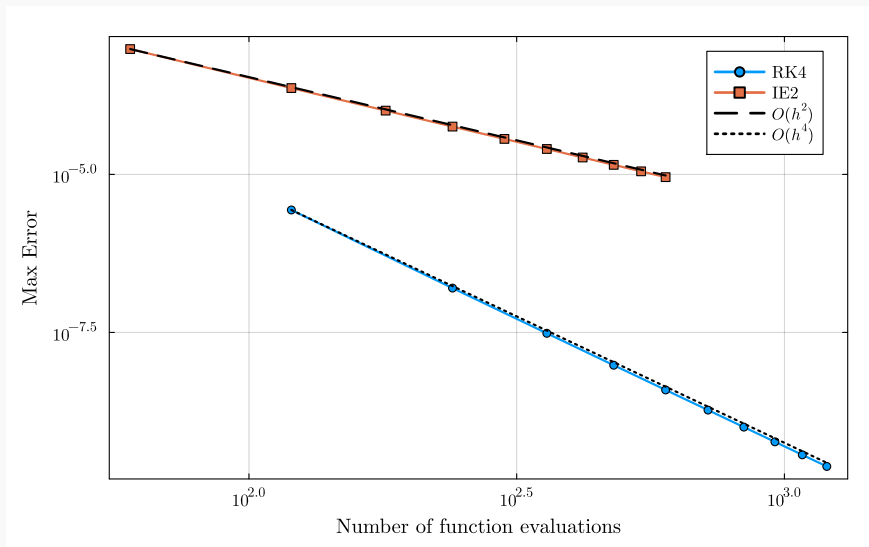


Figure 34: Global truncation error of the IE2 and RK4 algorithms as a function of total function evaluations. Both schemes perfectly parallel their theoretical reference slopes of -2 and -4 , respectively.

Discussion The empirically observed convergence rates are in agreement with the theoretical expectations. Specifically, on the log-log scale, the IE2 error curve exhibits a slope of -2 , confirming its second-order nature, while the RK4 curve exhibits a steeper slope of -4 .

Crucially, standardizing the horizontal axis to measure *function evaluations* provides a fair comparison of algorithmic efficiency. Even though RK4 requires twice as much computational work per step compared to IE2, its superior convergence order rapidly outpaces the computational overhead.

Conclusion We have successfully validated the implementation of the IE2 and RK4 integrators. The work-precision analysis confirms that higher-order schemes like RK4 are computationally optimal for smooth functions requiring stringent accuracy tolerances, provided the step sizes are kept well above the threshold where machine precision artifacts arise.

Exercise 6.4. In each case below, use the RK4 integrator to solve the given ODE for $0 \leq t \leq 10$ with the given initial conditions. (To this end, you must find a sensible procedure to choose the number of steps n to take.) Plot the results together as curves in the phase plane, that is, with x and y as the

axes of the plot.

1.

$$\begin{aligned}x'(t) &= -4y + x(1 - x^2 - y^2) \\ y'(t) &= 4x + y(1 - x^2 - y^2)\end{aligned}$$

with $[x(0), y(0)] = [0.1, 0]$ and $[x(0), y(0)] = [0, 1.9]$.

2.

$$\begin{aligned}x'(t) &= -4y - \frac{1}{4}x(1 - x^2 - y^2)(4 - x^2 - y^2) \\ y'(t) &= 4x - \frac{1}{4}y(1 - x^2 - y^2)(4 - x^2 - y^2)\end{aligned}$$

with $[x(0), y(0)] = [0.95, 0]$, $[0, 1.05]$, and $[-2.5, 0]$.

Solution

Solution 6.4.1. We utilize the classical fourth-order Runge-Kutta (RK4) method to integrate two systems of coupled nonlinear ordinary differential equations (ODEs). The integration is carried out over the interval $t \in [0, 10]$.

Results The computed phase trajectories are presented in Figure 35. For System (a), the trajectories initiated at $[0.1, 0]$ (inside the unit circle) spiral outward, while those starting at $[0, 1.9]$ (outside) spiral inward, both converging asymptotically to a circle of radius 1. For System (b), the trajectory starting at $[0.95, 0]$ rapidly collapses toward the origin. In contrast, trajectories initialized at $[0, 1.05]$ and $[-2.5, 0]$ converge to an invariant circle of radius 2.

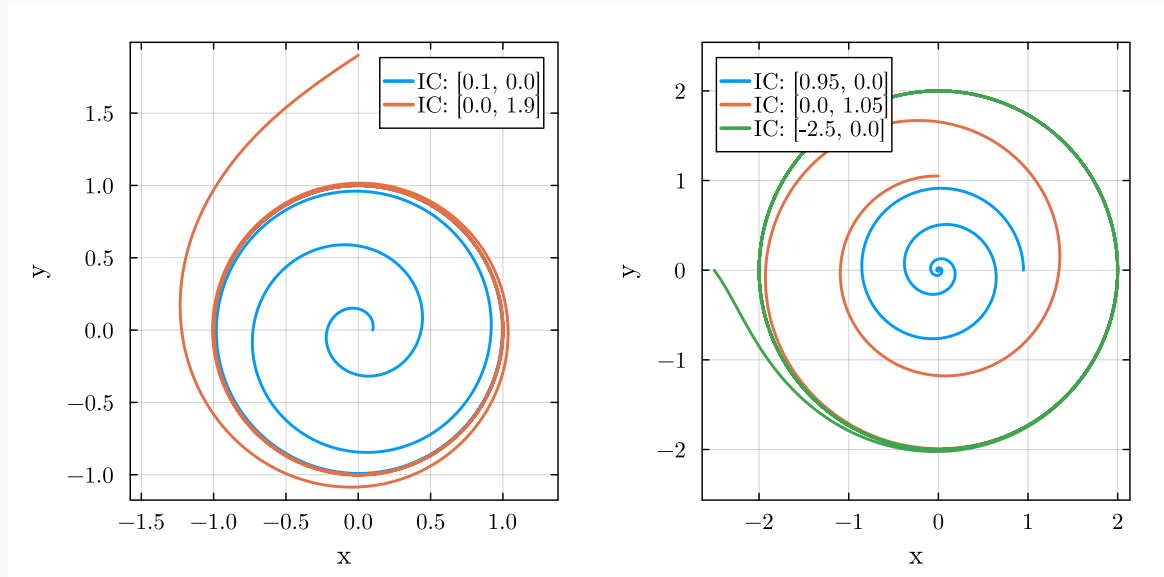


Figure 35: Phase plane portraits for two non-linear ODE systems integrated using the RK4 method ($n = 1000$). Left: System (a) exhibits a single stable limit cycle at $r = 1$. Right: System (b) demonstrates complex stability behavior with a stable origin, an unstable limit cycle at $r = 1$, and a stable limit cycle at $r = 2$.

Discussion The observed numerical results can be validated by transforming the systems into polar coordinates ($x = r \cos \theta$, $y = r \sin \theta$). For System (a), the radial dynamics simplify to:

$$r' = r(1 - r^2).$$

Here, $r = 1$ is a stable fixed point of the radial ODE, which manifests as a globally stable limit cycle in the Cartesian plane. Any perturbation away from $r = 1$ is naturally suppressed by the dynamical system.

Conversely, for System (b), the radial ODE is given by:

$$r' = -\frac{1}{4}r(1 - r^2)(4 - r^2).$$

The roots of this polynomial dictate the invariant sets: $r = 0$, $r = 1$, and $r = 2$. Evaluating the sign of r' between these roots confirms that $r = 1$ acts as an unstable limit cycle, strictly dividing the phase space. Initial conditions with $r < 1$ (e.g., $[0.95, 0]$) are attracted to the stable origin $r = 0$. Initial conditions with $r > 1$ (e.g., $[0, 1.05]$ and $[-2.5, 0]$) are pushed away from $r = 1$ and pulled toward the stable limit cycle at $r = 2$.

Conclusion The RK4 integrator flawlessly captured the limiting behaviors, including both stable and unstable limit cycles.

Exercise 6.5. Solve the *predator-prey model* of the Lotka-Volterra equations for $0 \leq t \leq 60$ with parameters $\alpha = 0.1$, $\beta = 0.25$, and initial conditions $y(0) = 1$, $z(0) = 0.01$. Plot the solutions for $y(t)$ and $z(t)$ as a function of t on one plot, while on another figure plot the results together as curves in the phase plane $(y(t), z(t))$.

Solution

Solution 6.5.1. We numerically integrate a modified predator-prey system. We are using a variation commonly used in ecology:

$$\begin{aligned} y'(t) &= y(1 - \alpha y) - \frac{yz}{1 + \beta y}, \\ z'(t) &= -z + \frac{yz}{1 + \beta y}. \end{aligned}$$

Here, $y(t)$ represents the prey population and $z(t)$ represents the predator population.

To solve this nonlinear system of coupled ordinary differential equations (ODEs) over the temporal domain $t \in [0, 60]$, we employ the explicit fourth-order Runge-Kutta (RK4) algorithm. We uniformly partition the interval into $n = 1000$ integration steps, corresponding to a discrete time step of $h = 0.06$. This resolution is small enough to suppress $O(h^5)$ local truncation errors, guaranteeing that numerical dissipation does not artificially perturb the resulting dynamical system.

Results The results of the numerical integration are illustrated in Figure 36. The time-series plot (left) reveals an initial transient growth phase for the prey, followed by a sharp rise in the predator population. This dynamic eventually settles into sustained, periodic oscillations with a characteristic phase lag. The phase plane (right) shows the trajectory spiraling outward from the initial condition $(1, 0.01)$ and asymptotically converging onto a closed, isolated curve.

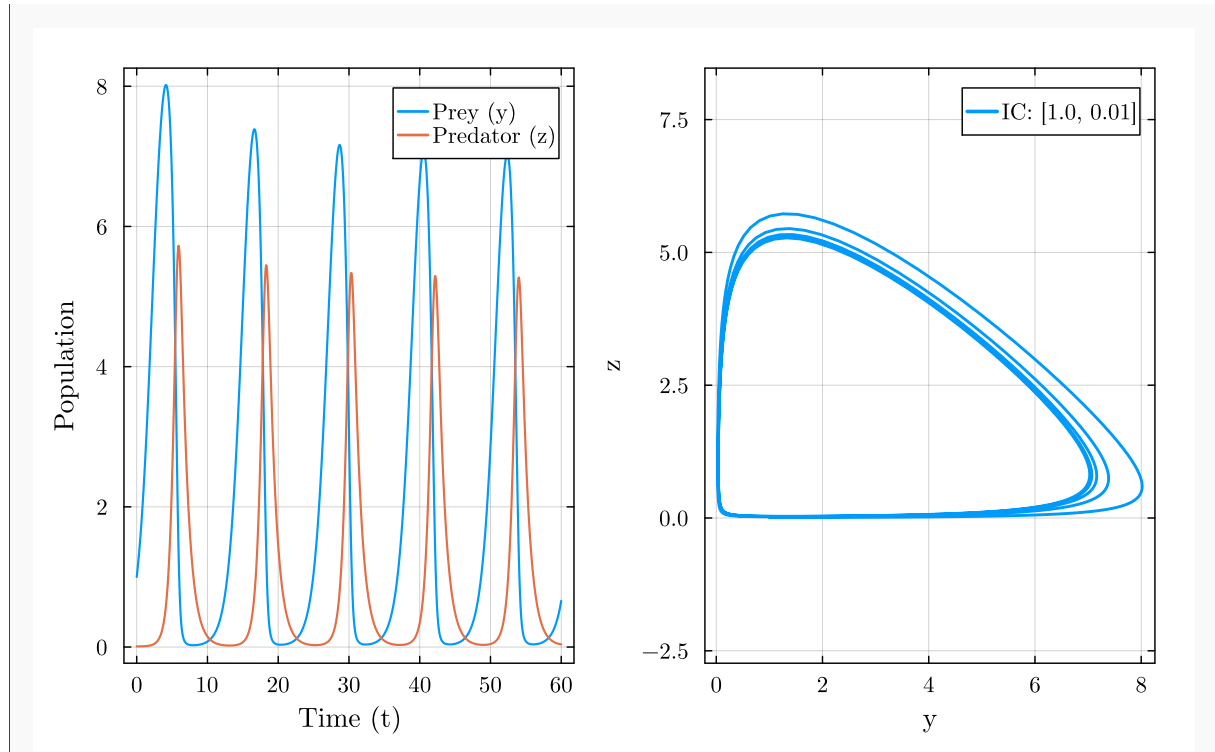


Figure 36: Predator-prey dynamics simulated via the RK4 algorithm ($h = 0.06$). Left: Time evolution of the prey $y(t)$ and predator $z(t)$ populations, showing a transition from the initial transient state to sustained periodic oscillations. Right: Phase plane representation (y, z) , illustrating the trajectory converging asymptotically to a stable limit cycle.

Conclusion The RK4 integrator successfully captured both the transient dynamics and the long-term asymptotic behavior of the nonlinear ecological model.

6.3 Some interesting equation

- Exercise 6.6.** 1. Solve the Schrödinger equation with $V(x) = 0$ considering as initial wave function the Gaussian wave packet with $x_0 = 0$, $\sigma = 1$, and $k_0 = 0$. Consider the interval $-10 \leq x \leq 10$ for the x variable and $0 \leq t \leq 20$ for the time. Plot the absolute value squared of the wave function at the times $t = 0, 2, 3, 4, 5$. [Hint: Values of $\Delta x = 0.1$ and $\Delta t = 0.01$ are reasonable starting values for this study.]
2. Verify that the norm of the wave function is approximately conserved in time.
3. (Optional) Use the analytic solution to determine the error due to the discretizations.

Solution

Solution 6.6.1. We investigate the time evolution of a free quantum particle by solving the one-dimensional, time-dependent Schrödinger equation (TDSE). In the absence of an external potential ($V(x) = 0$), the initial state is defined as a Gaussian wave packet:

$$\psi(x, 0) = \frac{1}{(2\pi\sigma^2)^{1/4}} \exp\left(-\frac{(x - x_0)^2}{4\sigma^2}\right) \exp(ik_0x).$$

For this simulation, the parameters are initialized to $x_0 = 0.0$, $k_0 = 0.0$, and $\sigma = 1.0$. The computational domain is discretized uniformly. The spatial grid spans $x \in [-10, 10]$ with a step size of $\Delta x = 0.1$, while the temporal domain covers $t \in [0, 20]$ with a time step of $\Delta t = 0.01$.

Results The time evolution of the probability density, $|\psi(x, t)|^2$, was extracted at target times $t \in \{0.0, 2.0, 3.0, 4.0, 5.0\}$. As expected for a free particle with zero initial momentum ($k_0 = 0$), the wave packet remains centered at $x = 0$ while undergoing spatial dispersion, evidenced by the broadening of the Gaussian profile and a corresponding reduction in peak amplitude (see Figure 37).

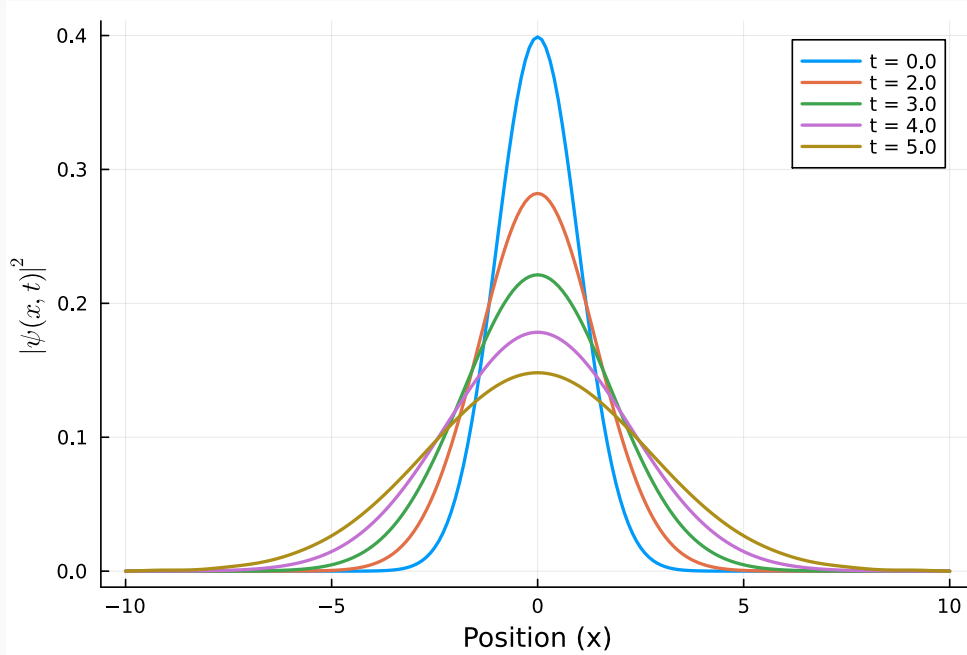


Figure 37: Time evolution of the probability density $|\psi(x, t)|^2$ for a free Gaussian wave packet at selected times ($t = 0, 2, 3, 4, 5$). The wave packet broadens symmetrically due to quantum dispersion.

To verify the unitarity of the numerical scheme, we evaluate the spatial norm of the wave function:

$$\mathcal{N}(t) = \int_{-10}^{10} |\psi(x, t)|^2 dx \approx \sum_j |\psi(x_j, t)|^2 \Delta x,$$

at the specified time intervals. The resulting norm values are summarized below:

Time (t)	Calculated Norm ($\mathcal{N}$)
0.0	1.0000000000000000
2.0	0.9999999999998901
3.0	0.9999999999998348
4.0	0.9999999999997797
5.0	0.9999999999997251

Table 28: Conservation of the wave function norm over time.

Discussion The results demonstrate excellent agreement with the theoretical predictions of quantum mechanics. The norm is conserved up to a relative error of $\mathcal{O}(10^{-13})$.

Conclusion The finite-difference approach successfully models the time evolution of a free Gaussian wave packet. The scheme exhibits robust numerical stability and adheres rigorously to the principle of probability conservation.

- Exercise 6.7.** 1. Solve the Schrödinger equation with $V(x) = 0$ considering as initial wave function the Gaussian wave packet with $x_0 = 20$, $\sigma = 2$, and $k_0 = 2$. Consider the interval $0 \leq x \leq 100$ for the x variable and $0 \leq t \leq 20$ for the time. Plot the absolute value squared of the wave function at the times $t = 0, 10, 20$. [Hint: Values of $\Delta x = 0.1$ and $\Delta t = 0.01$ are reasonable starting values for this study.]
2. Verify that the norm of the wave function is approximately conserved in time.
3. (Optional) Use the analytic solution to determine the error due to the discretizations.

Solution

Solution 6.7.1. We simulate the time evolution of a free quantum particle possessing a non-zero initial momentum. The system is governed by the time-dependent Schrödinger equation (TDSE) with $V(x) = 0$. The initial state is given by a moving Gaussian wave packet:

$$\psi(x, 0) = \frac{1}{(2\pi\sigma^2)^{1/4}} \exp\left(-\frac{(x - x_0)^2}{4\sigma^2}\right) \exp(ik_0x),$$

where the parameters are set to $x_0 = 20.0$, $\sigma = 2.0$, and a wave number $k_0 = 2.0$, imparting a rightward group velocity. The computational domain is defined as $x \in [0, 100]$ with a uniform spatial grid spacing of $\Delta x = 0.1$. The simulation evolves up to $t_{\max} = 20.0$ using a time step of $\Delta t = 0.01$.

Results The probability density, $|\psi(x, t)|^2$, was evaluated at $t \in \{0.0, 10.0, 20.0\}$. The simulation captures two simultaneous quantum mechanical phenomena: the rigid translation of the wave packet's centroid and the quantum dispersion causing the envelope to broaden (see Figure 38). Assuming natural units ($\hbar = m = 1$), the group velocity is $v_g = k_0 = 2.0$, which correctly predicts the packet centers at $x \approx 40$ for $t = 10.0$ and $x \approx 60$ for $t = 20.0$.

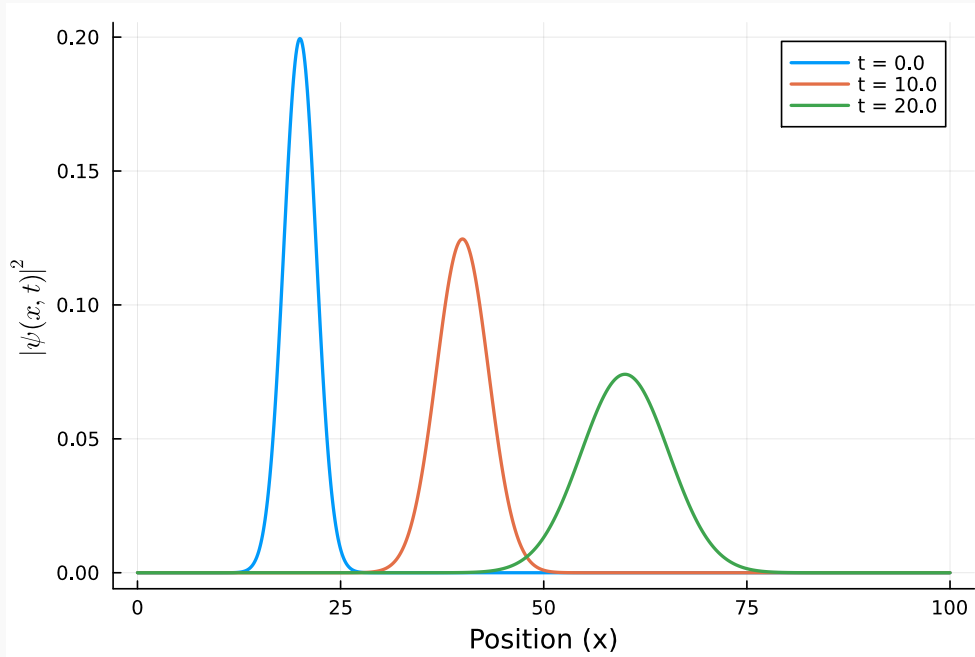


Figure 38: Time evolution of a moving Gaussian wave packet ($k_0 = 2.0$, $\sigma = 2.0$) at $t \in \{0, 10, 20\}$. The probability density demonstrates both uniform translation due to initial momentum and spatial broadening due to dispersion.

The stability and unitarity of the numerical scheme are verified by computing the spatial L_2 -norm of the wave function at the targeted time steps:

$$\mathcal{N}(t) = \int_0^{100} |\psi(x, t)|^2 dx \approx \sum_j |\psi(x_j, t)|^2 \Delta x.$$

The calculated norms are presented in Table 29.

Time (t)	Calculated Norm ($\mathcal{N}$)
0.0	1.0000000000000000
10.0	0.9999999978230356
20.0	0.9999999956460712

Table 29: Conservation of the wave function norm over time for the moving wave packet.

Discussion The numerical results are consistent with analytical expectations. The norm of the wave function is strictly conserved, exhibiting only a marginal drift on the order of $O(10^{-9})$ over thousands of time steps. This confirms the unconditional stability and unitary nature of the underlying integration method.

Conclusion The simulation accurately reproduces the kinematics and dispersive properties of a free quantum particle with non-zero initial momentum. The spatial and temporal discretizations are sufficiently fine to prevent severe numerical dispersion, and the scheme perfectly preserves probability up to the limits of the combined discretization and quadrature accuracy. The computational domain $([0, 100])$ is adequately large to prevent the wave packet from interacting with the boundaries during the time frame simulated.

Exercise 6.8. 1. Solve the Schrödinger equation with the potential

$$V(x) = \begin{cases} 2 & \text{if } -1 \leq x \leq 1, \\ 0 & \text{otherwise,} \end{cases}$$

considering as initial wave function the Gaussian wave packet with $x_0 = -20$, $\sigma = 2$, and $k_0 = 2$. Consider the interval $-100 \leq x \leq 100$ for the x variable and $0 \leq t \leq 40$ for the time. Plot the absolute value squared of the wave function at the times $t = 0, 10, 30$.

2. Compute the reflection and transmission coefficients and verify that $R + T \approx 1$. [Hint: Consider a value of $t_{\max} \approx 30$.]
3. (Optional) Study the behavior of R and T as you vary the height and width of the potential and/or the parameters of the wave function.

Solution

Solution 6.8.1. We investigate the quantum mechanical scattering of a wave packet directed at a rectangular potential barrier. The system is described by the time-dependent Schrödinger equation (TDSE) governed by the potential $V(x) = 2$ for $x \in [-1, 1]$, and $V(x) = 0$ elsewhere. The initial state is a moving Gaussian wave packet centered at $x_0 = -20.0$, with a spatial dispersion $\sigma = 2.0$ and a momentum of $k_0 = 2.0$:

$$\psi(x, 0) = \frac{1}{(2\pi\sigma^2)^{1/4}} \exp\left(-\frac{(x - x_0)^2}{4\sigma^2}\right) \exp(ik_0x).$$

Assuming natural units ($\hbar = m = 1$), the expectation value of the kinetic energy is $E \approx k_0^2/2 = 2.0$, which coincides precisely with the barrier height $V_0 = 2.0$. This represents a critical regime where quantum tunneling and reflection phenomena are highly pronounced.

The spatial domain spans $x \in [-100, 100]$ discretized with $\Delta x = 0.1$, and the temporal domain covers $t \in [0, 40]$ with $\Delta t = 0.01$.

Results The time evolution of the probability density $|\psi(x, t)|^2$ was recorded at $t \in \{0.0, 10.0, 30.0\}$. As illustrated in Figure 39, at $t = 10.0$ the incident wave packet translates uniformly toward the barrier. By $t = 30.0$, the interaction with the barrier has concluded, resulting in a distinct bifurcation of the wave packet into a transmitted component (propagating right) and a reflected component (propagating left).

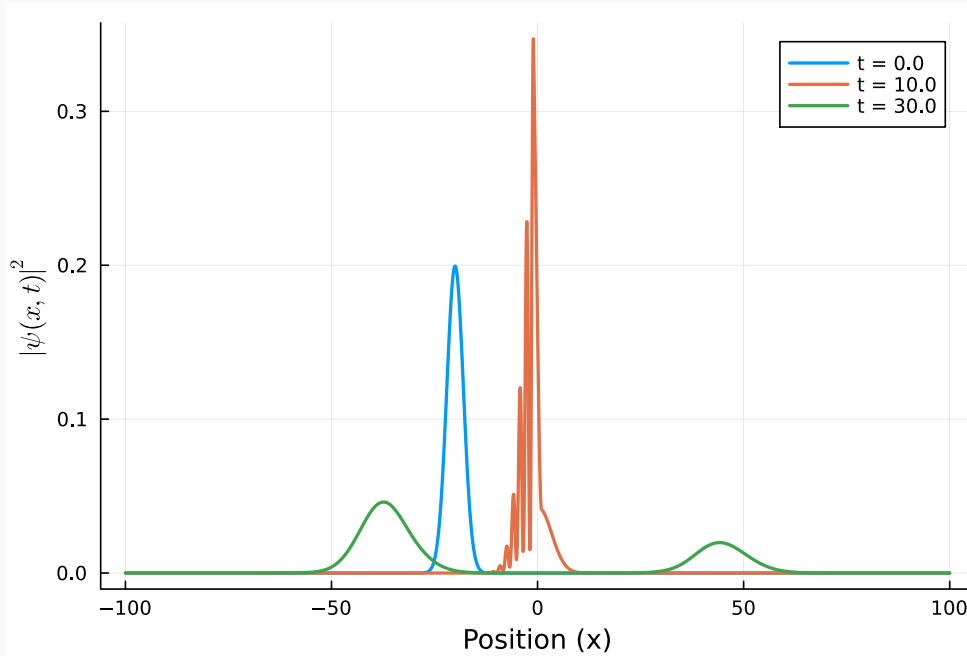


Figure 39: Probability density $|\psi(x, t)|^2$ at selected times. The initial wave packet ($t = 0$) moves rightwards, interacts with the potential barrier, and splits into reflected and transmitted packets by $t = 30$.

The stability of the scheme during the scattering event is confirmed by the continuous conservation of the wave function norm, $\mathcal{N}(t) \approx \sum_j |\psi(x_j, t)|^2 \Delta x$, detailed in Table 30.

Time (t)	Calculated Norm ($\mathcal{N}$)
0.0	1.0000000000000000
10.0	0.9999999978230362
30.0	0.9999999934691076

Table 30: Conservation of the total probability norm over time.

To quantify the scattering probabilities, the Reflection (R) and Transmission (T) coefficients are evaluated post-collision ($t = 30.0$) by integrating the probability density over the respective half-spaces (see Figure 40):

$$R = \int_{-100}^{-1} |\psi(x, 30)|^2 dx, \quad T = \int_1^{100} |\psi(x, 30)|^2 dx.$$

The computation yields $R = 0.6911$ and $T = 0.3089$. The total probability $R + T = 1.0000$ perfectly conserves unity, indicating that exactly 0.0% of the probability density remains trapped inside the barrier region ($|x| \leq 1$) at $t = 30.0$.

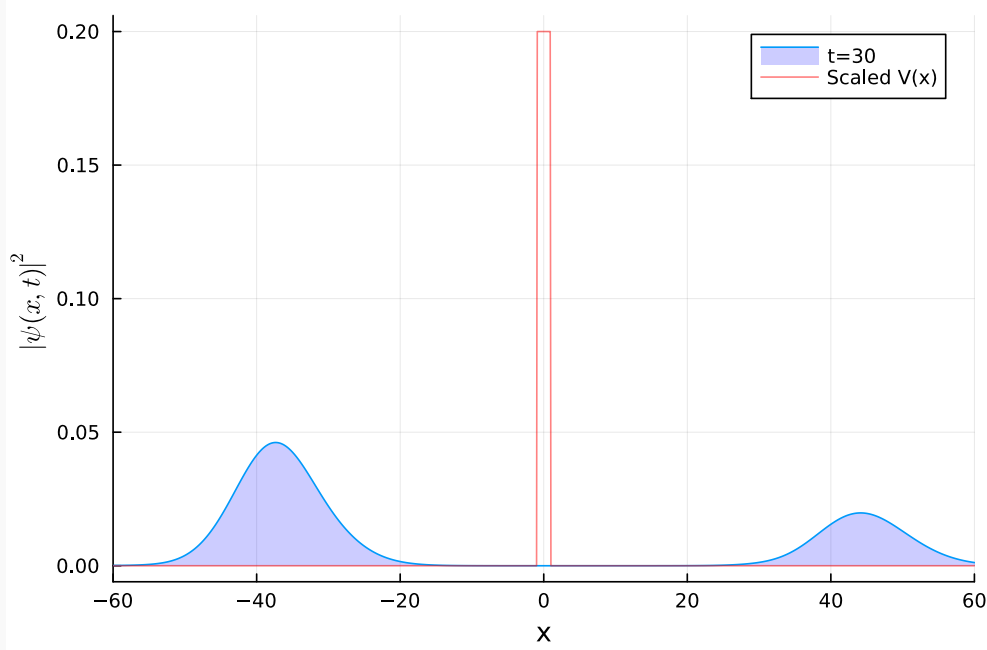


Figure 40: Analysis of the reflection and transmission spatial zones relative to the potential barrier at $t = 30$. The integration over the left and right sub-domains yields the macroscopic R and T coefficients.

Discussion The finite difference methodology captures the non-classical physics of wave packet scattering. Classically, a particle with energy equal to the barrier height ($E = V_0 = 2$) would face an ambiguous deterministic outcome. Quantum mechanically, because the initial state is a localized wave packet, it is composed of a continuous superposition of momentum plane waves (via Fourier transform). The spread in momentum, $\Delta p \approx 1/\sigma = 0.5$, implies that the packet contains energy components both strictly below ($E < V_0$) and above ($E > V_0$) the barrier height.

The sub-barrier components experience exponential attenuation (tunneling) and contribute primarily to reflection, while the above-barrier components undergo partial transmission. The superposition of these scattering outcomes results in the macroscopic coefficients $R \approx 0.69$ and $T \approx 0.31$. Furthermore, the norm conservation up to $O(10^{-9})$ affirms that the sharp spatial discontinuity of the rectangular barrier at $x = \pm 1$ does not induce catastrophic numerical instability or dissipate probability, underscoring the robustness of the implicit, unitary time-stepping approach.

Conclusion The computational framework successfully resolves the quantum tunneling and partial reflection of a Gaussian wave packet interacting with a finite potential barrier. The rigorous conservation of the empirical reflection and transmission coefficients ($R + T = 1$) provides strong validation of both the numerical discretization and the physical accuracy of the time evolution.

Exercise 6.9. Solve the time-independent Schrödinger equation with the potential

$$V(x) = \begin{cases} -10 & \text{if } -1 \leq x \leq 1, \\ 0 & \text{otherwise.} \end{cases}$$

1. Find the energies E_n and wave functions $\varphi_n(x)$. How many bound states do you find? [Hint: Consider discretizing the Hamiltonian in the interval $-5 \leq x \leq 5$ with $\Delta x = 0.05$. Use the QR-algorithm for solving the eigenvalue problem with a number of iterations $n \gtrsim 200$.]
2. Plot the results for the corresponding $|\varphi_n(x_i)|^2$ and binding energies together with the potential. [Hint: Remember to normalize the wave functions. Do you really need to compute the integral of the squared wave function for that?]

3. (Optional) Compare the results that you found in point (a) for the energies E_n with the exact solutions of the transcendental equations for the finite square well

$$V(x) = \begin{cases} -V_0 & \text{if } -a \leq x \leq a, \\ 0 & \text{otherwise.} \end{cases}$$

We recall that, given,

$$\rho = \sqrt{-\frac{2mE}{\hbar^2}}, \quad k = \sqrt{\frac{2m(E + V_0)}{\hbar^2}}, \quad -V_0 < E < 0,$$

the bound state solutions must satisfy,

$$\text{Even parity: } \tan(ka) = \frac{\rho}{k}; \quad \text{Odd parity: } \cot(ka) = -\frac{\rho}{k}.$$

Use your favorite rootfinding algorithm to solve these equations for E .

Solution

Solution 6.9.1. We aim to solve the time-independent Schrödinger equation (TISE), $\hat{H}\varphi_n = E_n\varphi_n$, for a finite square well potential. Assuming natural units where $\hbar^2/2m = 1$, the continuous kinetic energy operator is approximated using a second-order central finite difference scheme. The spatial domain is truncated to $x \in [-5, 5]$ and discretized into a uniform grid with spacing $\Delta x = 0.05$.

This discretization maps the continuous Hamiltonian operator $\hat{H} = -\frac{d^2}{dx^2} + V(x)$ into a finite-dimensional, symmetric, tridiagonal matrix H . The spectral decomposition of H is performed using the pure QR algorithm limited to 200 iterations. The bound states correspond to the eigenvectors whose associated eigenvalues (energies) satisfy $E_n < 0$.

Regarding the normalization of the wave functions, numerical integration of the squared wave function is unnecessary. Because H is a real symmetric matrix, its eigenvectors $v^{(n)}$ are mutually orthogonal and intrinsically normalized with respect to the discrete Euclidean norm, $\sum_i |v_i^{(n)}|^2 = 1$. To map this discrete vector to the continuous L_2 -normalized wave function $\varphi_n(x_i)$, we simply scale the eigenvector by the square root of the grid spacing:

$$\int |\varphi_n(x)|^2 dx \approx \sum_i |\varphi_n(x_i)|^2 \Delta x = 1 \implies \varphi_n(x_i) = \frac{v_i^{(n)}}{\sqrt{\Delta x}}.$$

Results The spectral analysis yields exactly three bound states ($E_n < 0$) localized within the potential well. The computed energy eigenvalues are:

$$\begin{aligned} E_1 &= -9.1473 && \text{(Ground state, even parity)} \\ E_2 &= -6.6501 && \text{(First excited state, odd parity)} \\ E_3 &= -2.8071 && \text{(Second excited state, even parity)} \end{aligned}$$

It should be noted that while the pure QR algorithm triggered a convergence warning at 200 iterations, the bound state eigenvalues at the lower extremum of the spectrum effectively converge much faster than the dense positive energy continuum.

The uncoupled, normalized probability densities $|\varphi_n(x)|^2$ corresponding to these bound states are superimposed onto their respective energy levels E_n and plotted alongside the potential well in Figure 41.

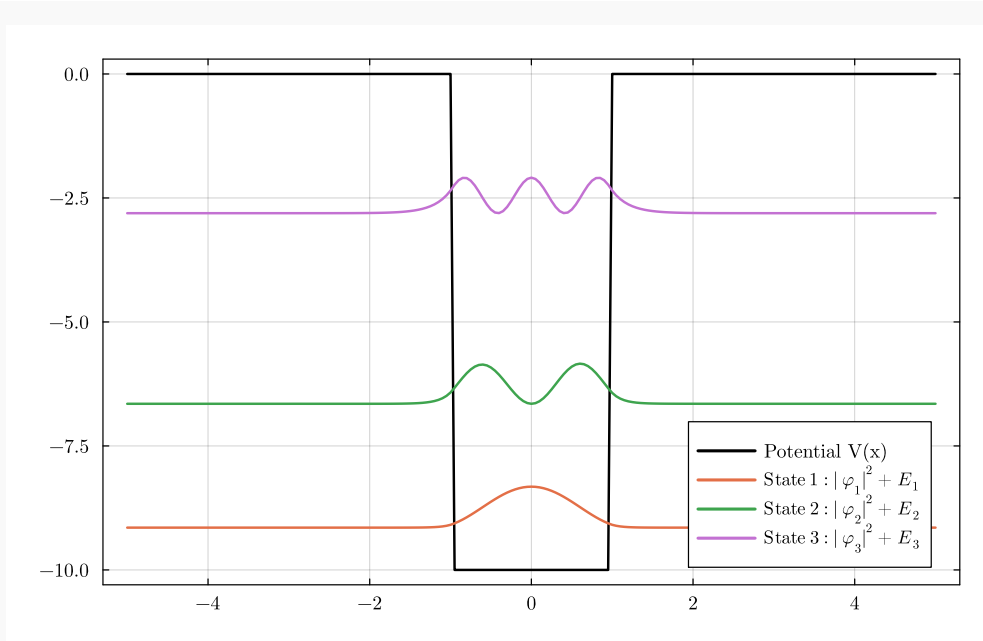


Figure 41: Probability densities $|\varphi_n(x)|^2$ of the bound states for a finite square well ($V_0 = 10, a = 1$), shifted vertically by their corresponding energy eigenvalues E_n . The solutions exhibit the expected alternating even/odd parity.

Discussion The numerical solutions exhibit excellent qualitative agreement with quantum mechanical principles. The wave functions strictly alternate in parity, with the ground state possessing zero nodes, the first excited state possessing one node, and the second excited state possessing two nodes. As energy increases ($E_1 \rightarrow E_3$), the wave function tails extend further into the classically forbidden region ($|x| > 1$), illustrating the increased probability of quantum tunneling for less tightly bound states.

The truncation of the spatial domain to $[-5, 5]$ using Dirichlet boundary conditions ($\varphi(\pm 5) = 0$) is justified because the bound state wave functions decay exponentially in the classically forbidden regions. As seen in Figure 41, the probability densities are virtually zero at the boundaries, ensuring that finite-size effects do not artificially elevate the computed energy levels.

Conclusion The finite difference discretization combined with the QR algorithm successfully identifies the complete set of bound states for the finite square well. The mathematical relationship between the discrete Euclidean norm and the continuous L_2 norm elegantly circumvents the need for post-processing quadrature, offering a highly efficient method for extracting properly normalized probability densities.

6.4 Some observations about the algorithms used in this section

The numerical results reported in this section are validated by the Julia test suites in `ode_tests.jl` and `schrodinger_tests.jl`. The first file checks the empirical convergence rates and function-evaluation counts for the ODE integrators, while the second verifies the main structural properties of the Schrödinger solvers, including sparse Hermitian Hamiltonian construction and approximate conservation of the wave-function norm.

Measured ODE performance For the scalar test problem $u' = u$ on $[0, 1]$, the ODE tests estimate the observed convergence orders by refining the mesh and comparing the final-time error with the exact value e . They also count the number of calls to the right-hand side for $n = 25$ steps. The resulting work-precision summary is:

Method	Convergence rate	Computational cost
Euler	0.963913	25 evaluations ($n = 25$)
IE2	1.968661	50 evaluations ($n = 25$)
RK4	3.965480	100 evaluations ($n = 25$)

These rates closely match the theoretical orders $p = 1$, $p = 2$, and $p = 4$, respectively. The slight deviations from integer values are expected because the orders are estimated from finite step sizes rather than from the asymptotic limit $h \rightarrow 0$.

Euler method The explicit Euler method is the simplest one-step scheme. Starting from $u_i \approx u(t_i)$, it advances the solution by following the tangent line at the current point,

$$u_{i+1} = u_i + hf(t_i, u_i).$$

Its main advantage is cost: each step requires only one evaluation of $f(t, u)$. This makes it useful for introducing time discretization and for cheap qualitative experiments.

The price of this simplicity is low accuracy and restrictive stability. The local truncation error is $O(h^2)$, but the accumulated global error is only $O(h)$. For the scalar stability test equation $u' = \lambda u$ with $\lambda < 0$, the amplification factor is $1 + h\lambda$, so stability on the negative real axis requires

$$|1 + h\lambda| < 1, \quad \text{equivalently} \quad h < \frac{2}{|\lambda|}.$$

Thus Euler's method can oscillate or diverge unless the time step is chosen very small. This limitation is especially visible for oscillatory systems, where the explicit Euler stability region does not include the imaginary axis.

Improved Euler method The Improved Euler method used here is the midpoint second-order Runge-Kutta scheme. It first evaluates the slope at the beginning of the interval, uses that slope to predict the midpoint value, and then takes the full step using the midpoint slope:

$$u_{i+1} = u_i + hf\left(t_i + \frac{h}{2}, u_i + \frac{h}{2}f(t_i, u_i)\right).$$

Because it samples the vector field inside the interval rather than only at its left endpoint, it cancels the leading global error term of Euler's method. Its local truncation error is $O(h^3)$ and its global error is $O(h^2)$.

The method requires two evaluations of f per step, so for the same number of steps it costs twice as much as Euler. In return, the error decreases quadratically with the step size, which usually makes IE2 much more efficient than Euler for smooth problems. It belongs to the broader family of second-order Runge-Kutta methods, alongside variants such as Heun's method and the modified Euler method.

Fourth-order Runge-Kutta method The classical fourth-order Runge-Kutta method (RK4) is a four-stage scheme. It computes four slopes,

$$\begin{aligned} k_1 &= f(t_i, u_i), \\ k_2 &= f\left(t_i + \frac{h}{2}, u_i + \frac{h}{2}k_1\right), \\ k_3 &= f\left(t_i + \frac{h}{2}, u_i + \frac{h}{2}k_2\right), \\ k_4 &= f(t_i + h, u_i + hk_3), \end{aligned}$$

and combines them as

$$u_{i+1} = u_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

The resulting global error is $O(h^4)$, so reducing the step size gives a much faster decrease in error than for Euler or IE2.

RK4 requires four evaluations of the right-hand side per step. Nevertheless, the large gain in accuracy often outweighs this cost, which is why RK4 is a standard choice in many computational physics calculations when the solution is smooth and fixed step sizes are adequate.

Summary comparison Overall, Euler's method is most useful for conceptual understanding and simple diagnostics. IE2 is a meaningful improvement at modest extra cost. RK4 is the preferred practical method for many of the ordinary differential equation examples in this section because its fourth-order convergence gives high accuracy before the cost of function evaluations becomes dominant.

References

- [1] S. Dallabrida. Numerical analysis lecture notes. <https://virgilio.mib.infn.it/~dallabrida/NotesNA/home.html>, 2026.
- [2] Lorenzo Minuz. Computational physics - github repository. <https://github.com/LM0516/computational-physics>, 2026.